M04 — Patrones de arquitectura con ASP.NET Core 8 · Teoría

Autor del curso: JAIME GALLO

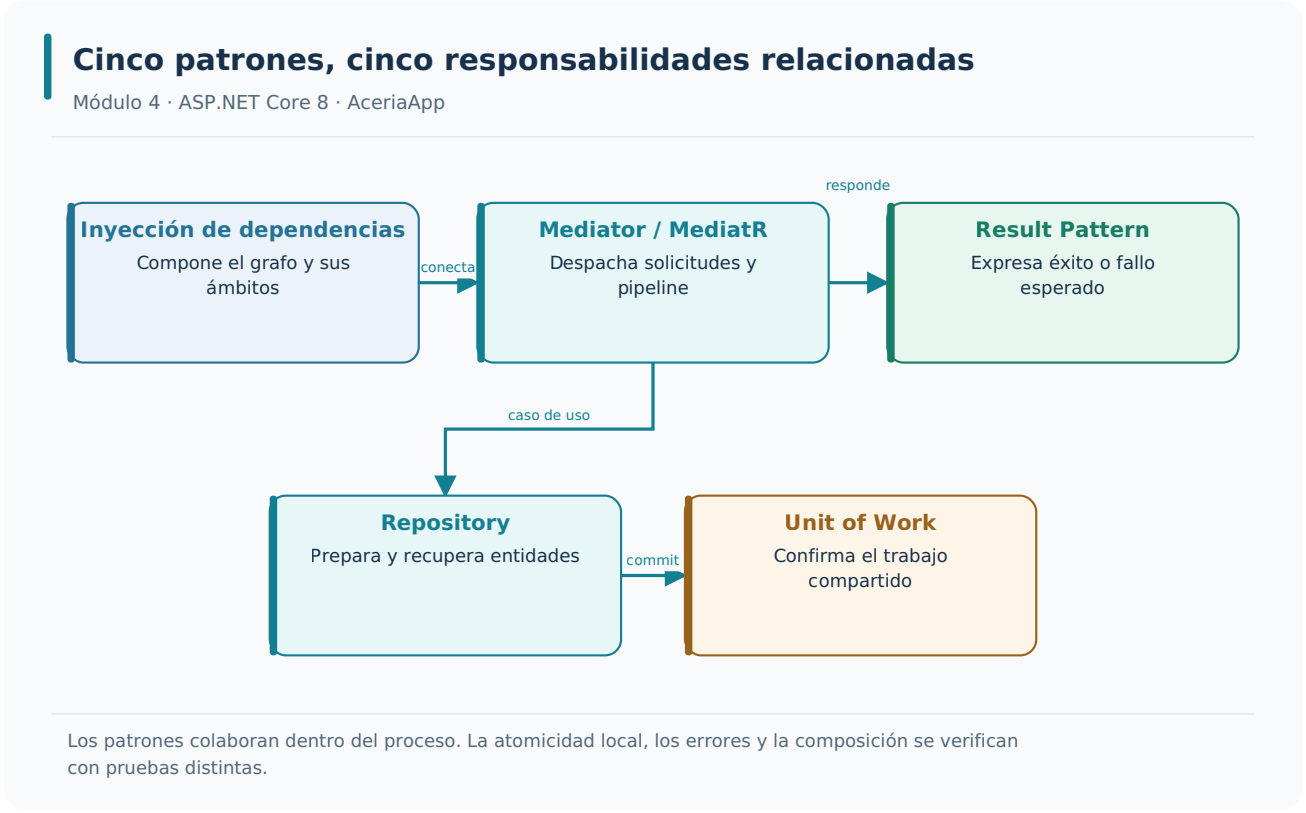
Cómo estudiar el módulo

Los cinco puntos desarrollan Repository, Unit of Work, Mediator con MediatR, Result Pattern e inyección de dependencias. Cada punto contiene diez bloques, ejemplos explicados línea a línea, ampliaciones aplicadas a AceriaApp, diagramas y un resumen de conceptos. Las explicaciones **Línea N** cuentan las líneas no vacías de cada ejemplo; cada explicación ocupa su propia viñeta.

El punto de partida es `M03_ARQUITECTURA_HEXAGONAL/PROYECTO/3.4-implementacion-aspnet-core`: la API `AceriaApp.Api` y las capas `AceriaApp.Hexagonal.Dominio`, `AceriaApp.Hexagonal.Aplicacion` y `AceriaApp.Hexagonal.Infraestructura`. La entidad de referencia es `OrdenHexagonal` y el contexto es `HexagonalDbContext`. Los ejemplos breves de cada bloque son variantes que se estudian por separado; las prácticas indican los archivos y nombres exactos de cada snapshot acumulativo.

Recorrido funcional del módulo. El repositorio de trabajo `IOrdenTrabajoRepositorio` expone `ObtenerAsync` y `AgregarAsync` sin confirmar; `IUnitOfWork.CommitAsync` se implementa mediante `EfUnitOfWork`; `OrdenesTransaccionales` coordina operaciones; MediatR 12.4.1 conecta `CrearOrdenCommand`, `ObtenerOrdenQuery` y `LoggingBehavior`; `OrdenesConResultado` expresa fallos de negocio y `AddPatronesOrdenes` reúne los registros. El repositorio inmediato de M3 se conserva como una capacidad distinta. Un catálogo arquitectónico explica decisiones; las pruebas funcionales verifican además lo que ejecutan los adaptadores.

Convenciones de los ejemplos. Los fragmentos usan los namespaces habituales de .NET, EF Core, MediatR, FluentValidation y xUnit/Moq según el concepto. Las interfaces breves de repositorio son alternativas de diseño, no definiciones para pegar todas juntas. Los ejemplos simples de `Result<T>` usan `IsSuccess`, `Value`, `Success` y `Failure`; la implementación de las prácticas usa `EsExito`, `Valor`, `Exito` y `Fallo`, además de `Error` y `TipoError`. La semántica es la misma y la API concreta se distingue al aplicarla al proyecto. La ruta funcional final es `/api/v1/patrones/ordenes`, con consulta `/{id:guid}` y transición `/{id:guid}/finalizar`.



Punto 4.1 — Repository

Módulo 4: Patrones de arquitectura

Objetivos de aprendizaje:

- Comprender el patrón Repository y su propósito en una arquitectura hexagonal.
- Identificar las responsabilidades de un repositorio y las que no le corresponden.
- Comprender la diferencia entre un repositorio específico y un repositorio genérico.
- Comprender la relación entre Repository, puertos de salida y adaptadores.
- Añadir al proyecto `AceriaApp` las clases de validación de repositorios y un test unitario que verifique la validación.

Desarrollo teórico

Bloque 1 — Definición del patrón Repository

El patrón Repository es una abstracción que media entre la capa de dominio o aplicación y la capa de persistencia. Su propósito es ocultar los detalles de acceso a datos detrás de una interfaz que se expresa en términos del dominio. El repositorio actúa como una colección en memoria de objetos del dominio. Los clientes del repositorio no conocen si los datos provienen de una base de datos relacional, de un archivo o de un servicio remoto. La interfaz del repositorio se define en la capa de aplicación o en la capa de dominio, y su implementación se sitúa en la capa de infraestructura.

```
Patrón Repository:

Aplicación → IOrdenRepositorio (puerto)
                    ↑
                    | implementa
```

```
                    |
Infraestructura → OrdenRepositorioSqlServer (adaptador)
```

- **Línea 1:** `Aplicación → IOrdenRepositorio (puerto)` → la aplicación define el puerto.
- **Línea 2:** `↑` → la flecha indica que la infraestructura implementa el puerto.
- **Línea 3:** `| implementa` → la infraestructura implementa el puerto.
- **Línea 4:** `Infraestructura → OrdenRepositorioSqlServer (adaptador)` → la infraestructura contiene el adaptador.

El patrón Repository se caracteriza por tres propiedades. La primera es que la interfaz se expresa en términos del dominio: los métodos reciben y devuelven entidades del dominio, no filas de base de datos. La segunda es que oculta la tecnología de persistencia: el cliente no conoce si los datos se guardan en SQL Server, en un archivo o en memoria. La tercera es que agrupa operaciones de persistencia de una entidad o, habitualmente en DDD, de una raíz de agregado: el contrato expresa las necesidades del caso de uso. Las tres propiedades permiten que el dominio sea independiente de la tecnología de persistencia.

La aplicación del patrón Repository tiene dos consecuencias prácticas. La primera es que el dominio y la aplicación se prueban con repositorios en memoria sin necesidad de base de datos. La segunda es que la abstracción reduce el impacto de sustituir la persistencia, aunque diferencias en consultas, consistencia y transacciones pueden exigir revisar el contrato. La combinación de ambas reduce el acoplamiento entre capas y facilita la evolución del sistema. El repositorio es la pieza que materializa el puerto de salida de persistencia.

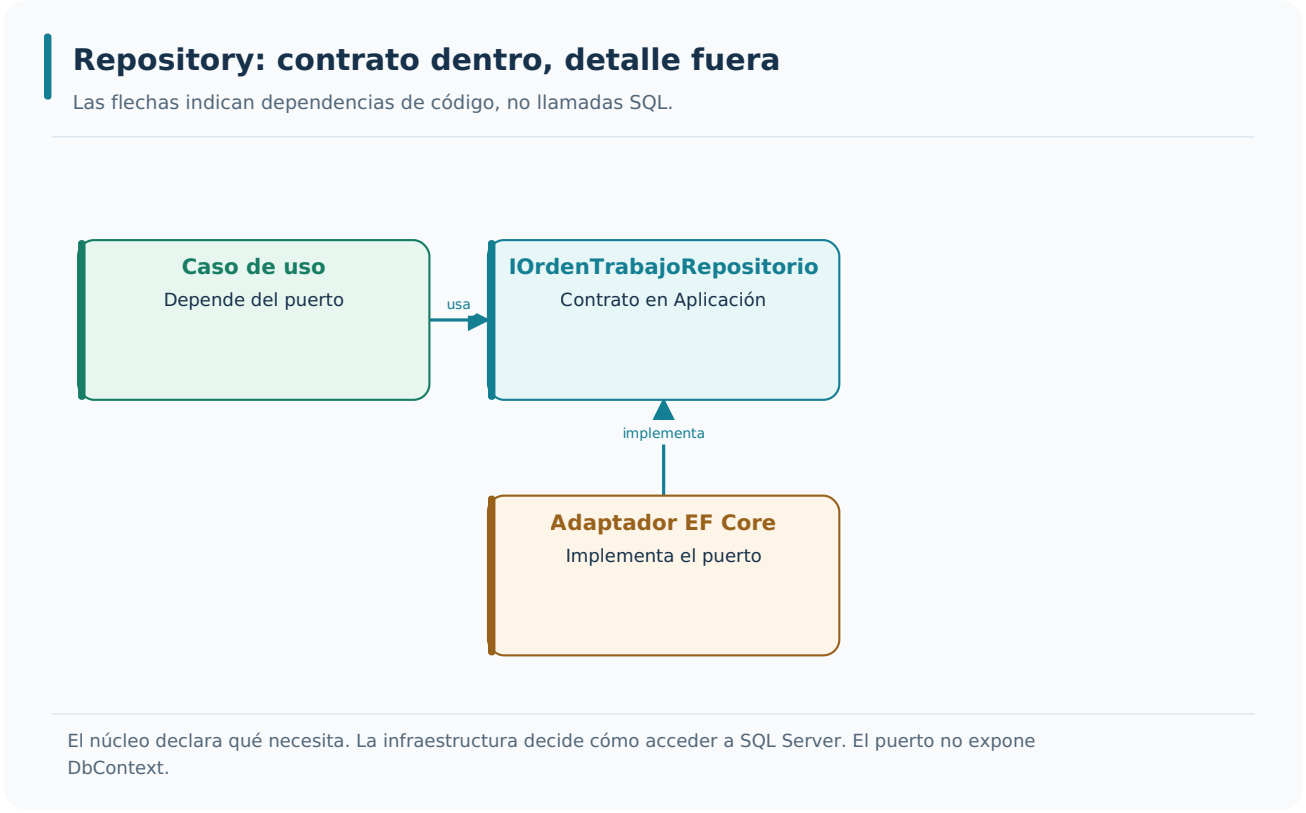
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El contrato debe explicar qué necesita la aplicación antes de describir cómo guarda SQL Server. La palabra Repository nombra el patrón; IOrdenRepositorio nombra un puerto concreto.

Ejemplo concreto en AceriaApp. M3 ya contiene IOrdenRepositorio y OrdenRepositorioSqlServer. M4 añade un puerto de trabajo que permite estudiar por separado la preparación de cambios y su confirmación.

Riesgo, alternativa y límite. Un repositorio no convierte automáticamente dos motores distintos en intercambiables. La paginación, las transacciones y la concurrencia siguen requiriendo decisiones explícitas.

Comprobación técnica y aprendizaje. Localiza la interfaz en aplicación y el adaptador en infraestructura. El proyecto de dominio debe compilar sin referenciar EF Core ni ASP.NET Core.



Interpretación del gráfico: El puerto se define dentro y el adaptador apunta hacia su contrato. Las flechas muestran dependencias de código. La figura se distribuye en SVG vectorial y PNG.

Bloque 2 — Responsabilidades de un repositorio

Un repositorio tiene tres responsabilidades principales. La primera es recuperar entidades del almacenamiento: el repositorio ofrece métodos para obtener una entidad por identificador, obtener una colección de entidades o buscar entidades por criterios. La segunda es añadir entidades al almacenamiento: el repositorio ofrece métodos para insertar entidades nuevas. La tercera es actualizar o eliminar entidades existentes: el repositorio ofrece métodos para modificar o eliminar entidades. Cada responsabilidad se expresa con métodos específicos en la interfaz del repositorio.

```
public interface IOrdenRepositorio
{
    Task<Orden?> GetByIdAsync(Guid id);
    Task<List<Orden>> ListarAsync();
    Task AddAsync(Orden orden);
    Task UpdateAsync(Orden orden);
    Task DeleteAsync(Guid id);
}
```

- **Línea 1:** `public interface IOrdenRepositorio` → interfaz del repositorio.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `Task<Orden?> GetByIdAsync(Guid id);` → recupera una entidad por identificador.
- **Línea 4:** `Task<List<Orden>> ListarAsync();` → recupera la colección completa.
- **Línea 5:** `Task AddAsync(Orden orden);` → añade una entidad.
- **Línea 6:** `Task UpdateAsync(Orden orden);` → actualiza una entidad.
- **Línea 7:** `Task DeleteAsync(Guid id);` → elimina una entidad por identificador.
- **Línea 8:** `}` → cierra el cuerpo.

Las tres responsabilidades se expresan con cinco métodos típicos. El método `GetByIdAsync` recupera una entidad concreta. El método `ListarAsync` recupera la colección. El método `AddAsync` inserta una

entidad nueva. El método `UpdateAsync` actualiza una entidad existente. El método `DeleteAsync` elimina una entidad. Son cinco operaciones habituales, no una plantilla obligatoria. Un repositorio puede omitir actualización o eliminación si el dominio no permite esas acciones y añadir las búsquedas que necesita.

La definición de las responsabilidades tiene dos consecuencias prácticas. La primera es que el repositorio es una abstracción del almacenamiento, no del dominio: no contiene reglas de negocio. La segunda es que las consultas deben servir al contrato: un JOIN puede ser necesario para reconstruir un agregado; las consultas de informes pueden separarse en puertos de lectura. La combinación de ambas permite que el repositorio sea simple y mantenible. El contrato evita exponer detalles como SQL, DbSet o IQueryable cuando se busca independencia del proveedor.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Los métodos se eligen por necesidades de negocio. No añadas Delete solo porque aparece en una plantilla CRUD si la orden debe conservarse por trazabilidad.

Ejemplo concreto en AceriaApp. El puerto heredado ofrece ListarAsync, ObtenerAsync, AgregarAsync y GuardarAsync. El puerto de trabajo de M4 usa ObtenerAsync y AgregarAsync para los casos desarrollados.

Riesgo, alternativa y límite. Actualizar un objeto seguido por EF Core no siempre requiere un método Update. Un contrato debe aclarar quién sigue la entidad y cuándo se confirma.

Comprobación técnica y aprendizaje. Comprueba ausencia con null, altas con una entidad válida y listas vacías con una colección vacía. Son contratos distintos que no deben confundirse con errores de SQL.

Bloque 3 — Lo que un repositorio no debe hacer

Un repositorio no debe contener reglas de negocio. Las reglas de negocio residen en las entidades del dominio. Un repositorio no debe validar invariantes: la validación de invariantes se realiza en el constructor de la entidad o en métodos específicos. Un repositorio de agregados no debería decidir la forma de una respuesta HTTP. Un puerto de lectura sí puede devolver una proyección definida por aplicación. Tampoco existe una prohibición de usar JOINs: infraestructura necesita las consultas que permitan cumplir el contrato; los informes con necesidades distintas pueden tener un servicio de consulta propio.

```
Lo que un repositorio no debe hacer:

- Contener reglas de negocio.
- Validar invariantes.
- Diseñar respuestas HTTP desde persistencia.
- Exponer consultas ajenas al propósito del contrato.
- Gestionar transacciones de múltiples repositorios.
```

- **Línea 1:** `- Contener reglas de negocio.` → las reglas residen en el dominio.
- **Línea 2:** `- Validar invariantes.` → la validación reside en la entidad.
- **Línea 3:** `- Diseñar respuestas HTTP desde persistencia.` → el contrato de transporte se decide en el adaptador de entrada.
- **Línea 4:** `- Exponer consultas ajenas al propósito del contrato.` → un JOIN interno es válido; los informes pueden tener un puerto propio.
- **Línea 5:** `- Gestionar transacciones de múltiples repositorios.` → la transacción la gestiona la unidad de trabajo.

La delimitación de responsabilidades tiene dos consecuencias prácticas. La primera es que el repositorio se mantiene simple y cohesionado. La segunda es que las reglas de negocio y las validaciones no se duplican entre el repositorio y el dominio. La combinación de ambas permite que el

repositorio sea sustituible sin afectar a la lógica de negocio. La disciplina en la delimitación es lo que distingue un repositorio correcto de un repositorio que acumula responsabilidades.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La frontera se define por responsabilidad, no por el número de JOINs. Recuperar un agregado puede exigir varias tablas sin incorporar reglas de negocio al repositorio.

Ejemplo concreto en AceriaApp. La regla Cantidad mayor que cero pertenece a OrdenHexagonal. El repositorio puede traducir sus propiedades a columnas, pero no decidir otra capacidad máxima diferente a aplicación.

Riesgo, alternativa y límite. Un índice único y una validación de negocio pueden proteger aspectos relacionados. La comprobación previa no elimina carreras; las restricciones de almacenamiento también deben tratarse adecuadamente.

Comprobación técnica y aprendizaje. Revisa si el adaptador decide HTTP 400, 404 o 409. Si lo hace, devuelve al adaptador de entrada esa traducción y conserva información suficiente sobre el resultado.

Bloque 4 — Repositorio específico frente a repositorio genérico

Un repositorio específico se define para una entidad concreta y expone solo los métodos que esa entidad necesita. Un repositorio genérico se define para varias entidades y expone métodos comunes a todas ellas. El repositorio específico puede expresar mejor las operaciones del negocio; ambos enfoques admiten pruebas. El repositorio genérico reduce la duplicación de código. La elección depende de los contratos necesarios, del coste de mantener abstracciones y de la semántica de los agregados, no solo del número de entidades.

```
// Repositorio específico
public interface IOrdenRepositorio
{
    Task<Orden?> GetByIdAsync(Guid id);
    Task AddAsync(Orden orden);
}

// Repositorio genérico
public interface IRepositorio<T> where T : class
{
    Task<T?> GetByIdAsync(Guid id);
    Task AddAsync(T entidad);
}
```

- **Línea 1:** `// Repositorio específico` → comentario de ubicación.
- **Línea 2:** `public interface IOrdenRepositorio` → interfaz específica.
- **Línea 3:** `{` → abre el cuerpo.
- **Línea 4:** `Task<Orden?> GetByIdAsync(Guid id);` → método específico.
- **Línea 5:** `Task AddAsync(Orden orden);` → método específico.
- **Línea 6:** `}` → cierra el cuerpo.
- **Línea 7:** `// Repositorio genérico` → comentario de ubicación.
- **Línea 8:** `public interface IRepositorio<T> where T : class` → interfaz genérica.
- **Línea 9:** `{` → abre el cuerpo.
- **Línea 10:** `Task<T?> GetByIdAsync(Guid id);` → método genérico.
- **Línea 11:** `Task AddAsync(T entidad);` → método genérico.
- **Línea 12:** `}` → cierra el cuerpo.

El repositorio específico tiene dos consecuencias prácticas. La primera es que la interfaz es legible: cada método describe una operación concreta. La segunda es que la implementación es testeable: cada repositorio se prueba de forma aislada. El repositorio genérico tiene dos consecuencias prácticas. La primera es que reduce la duplicación: una sola interfaz sirve para varias entidades. La segunda es que la interfaz es menos expresiva: los métodos son comunes a todas las entidades. La elección depende del contexto del proyecto.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Un repositorio específico usa vocabulario del agregado; uno genérico comparte operaciones estructurales. Ambos pueden ser útiles y ambos pueden volverse demasiado amplios.

Ejemplo concreto en AceriaApp. IOrdenTrabajoRepositorio permite expresar necesidades de órdenes sin obligar a todas las entidades futuras a tener el mismo identificador, paginación o política de borrado.

Riesgo, alternativa y límite. Una interfaz `IRepositorio<T>` con decenas de métodos y parámetros para incluir tablas puede terminar filtrando el ORM. Ahorrar clases no justifica perder el contrato.

Comprobación técnica y aprendizaje. Describe una operación válida para órdenes y otra prohibida. Verifica si la interfaz elegida permite distinguirlas o si ofrece accidentalmente operaciones que el dominio no admite.

Bloque 5 — Repositorio y puertos de salida

La interfaz del repositorio es el puerto de salida de persistencia; su implementación concreta es el adaptador. El puerto se define en la capa de aplicación. La implementación se sitúa en la capa de infraestructura. La interfaz del puerto se expresa en términos del dominio: recibe y devuelve entidades. La implementación conoce la tecnología de persistencia: usa EF Core, Dapper o el proveedor que corresponda. La separación entre puerto e implementación permite que el núcleo se pruebe con repositorios en memoria.

```
// Puerto de salida en la capa de aplicación
public interface IOrdenRepositorio
{
    Task AddAsync(Orden orden);
}

// Adaptador de salida en la capa de infraestructura
public class OrdenRepositorioSqlServer : IOrdenRepositorio
{
    private readonly AceriaDbContext _db;

    public OrdenRepositorioSqlServer(AceriaDbContext db)
    {
        _db = db;
    }

    public async Task AddAsync(Orden orden)
    {
        _db.Ordenes.Add(orden);
        await _db.SaveChangesAsync();
    }
}
```

- **Línea 1:** `// Puerto de salida en la capa de aplicación` → comentario de ubicación.
- **Línea 2:** `public interface IOrdenRepositorio` → puerto de salida.
- **Línea 3:** `{` → abre el cuerpo.
- **Línea 4:** `Task AddAsync(Orden orden);` → método del puerto.

- **Línea 5:** `}` → cierra el cuerpo.
- **Línea 6:** `// Adaptador de salida en la capa de infraestructura` → comentario de ubicación.
- **Línea 7:** `public class OrdenRepositorioSqlServer : IOrdenRepositorio` → adaptador.
- **Línea 8:** `{` → abre el cuerpo.
- **Línea 9:** `private readonly AceriaDbContext _db;` → dependencia del contexto.
- **Línea 10:** `public OrdenRepositorioSqlServer(AceriaDbContext db)` → constructor.
- **Línea 11:** `{` → abre el cuerpo del constructor.
- **Línea 12:** `_db = db;` → asigna el contexto.
- **Línea 13:** `}` → cierra el constructor.
- **Línea 14:** `public async Task AddAsync(Orden orden)` → implementación del método.
- **Línea 15:** `{` → abre el cuerpo del método.
- **Línea 16:** `_db.Ordenes.Add(orden);` → añade la orden al contexto.
- **Línea 17:** `await _db.SaveChangesAsync();` → persiste los cambios.
- **Línea 18:** `}` → cierra el método.
- **Línea 19:** `}` → cierra la clase.

La relación entre repositorio y puerto de salida tiene dos consecuencias prácticas. La primera es que el repositorio se define en el núcleo y se implementa en el exterior. La segunda es que el repositorio puede sustituirse por una implementación en memoria sin modificar el núcleo. La combinación de ambas permite que el núcleo se pruebe sin base de datos y que la tecnología de persistencia evolucione con independencia del núcleo.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El puerto pertenece al núcleo; el adaptador implementa el acceso. La inversión de dependencias se comprueba leyendo referencias de proyectos, no el recorrido de una petición.

Ejemplo concreto en AceriaApp. OrdenRepositorioSqlServer de M3 confirma cada alta. El adaptador de trabajo de M4 prepara el alta y deja que IUnitOfWork confirme el mismo HexagonalDbContext.

Riesgo, alternativa y límite. El ejemplo breve que guarda dentro del repositorio es una variante útil para una operación autónoma. No debe mezclarse con la confirmación conjunta de varios repositorios.

Comprobación técnica y aprendizaje. Al componer M4, comprueba que todos los participantes reciben la misma instancia scoped del contexto. Dos contextos independientes no forman una transacción por compartir una interfaz.

Bloque 6 — Repositorio y consultas de lectura

El repositorio devuelve entidades del dominio para las operaciones de escritura y para las operaciones de lectura que necesitan la entidad completa. Cuando la operación de lectura solo necesita una proyección, el repositorio puede ofrecer métodos que devuelvan DTOs. La decisión entre devolver entidades o DTOs depende del caso de uso. Las escrituras que ejecutan reglas de un agregado suelen cargar el agregado necesario; las actualizaciones directas exigen garantizar de otro modo sus invariantes. Las operaciones de lectura pueden necesitar solo una proyección. La separación entre lectura y escritura se profundiza en el módulo 5 con CQRS.

```
public interface IOrdenRepositorio
{
    Task<Orden?> GetByIdAsync(Guid id);
```

```
    Task<OrdenResumenDto?> GetResumenAsync(Guid id);
}
```

- **Línea 1:** `public interface IOrdenRepositorio` → interfaz del repositorio.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `Task<Orden?> GetByIdAsync(Guid id);` → devuelve la entidad completa.
- **Línea 4:** `Task<OrdenResumenDto?> GetResumenAsync(Guid id);` → devuelve una proyección.
- **Línea 5:** `}` → cierra el cuerpo.

La distinción entre entidad y proyección tiene dos consecuencias prácticas. La primera es que las operaciones con reglas de agregado usan el modelo que protege esas reglas. La segunda es que las operaciones de lectura pueden usar proyecciones para reducir el tamaño de la respuesta. La combinación de ambas mejora el rendimiento y mantiene la coherencia del modelo. La decisión se documenta en el ADR del punto y se revisa al final del módulo 5.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Una proyección de lectura expresa los datos que necesita la pantalla o consumidor. No obliga a materializar una entidad con todas sus relaciones.

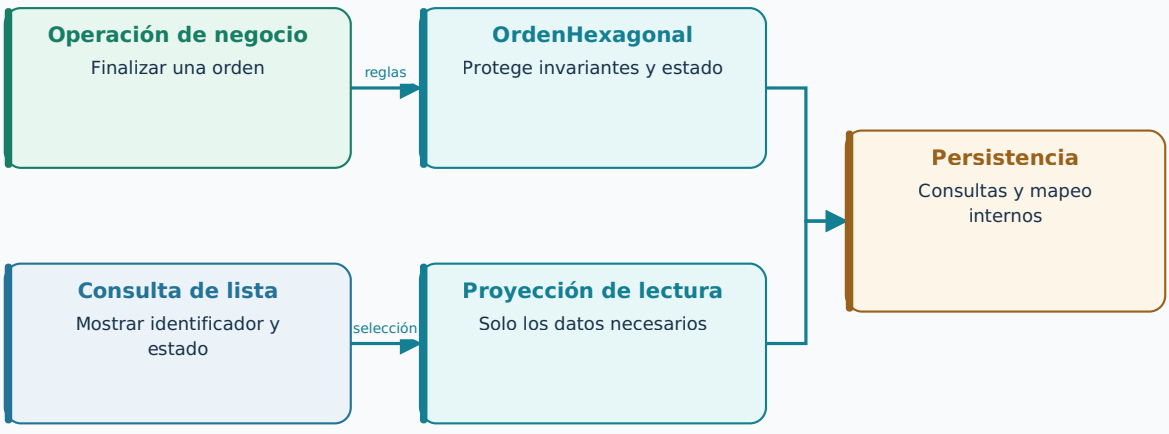
Ejemplo concreto en AceriaApp. Una lista de órdenes puede seleccionar identificador, cliente y estado. La operación Finalizar necesita recuperar la entidad sobre la que ejecuta la regla de transición.

Riesgo, alternativa y límite. Si el contrato está en dominio no debe depender de un DTO definido en API. Ubica proyecciones en aplicación o en un puerto de lectura separado.

Comprobación técnica y aprendizaje. Comprueba que la paginación se ejecuta antes de materializar grandes colecciones y que las lecturas sin edición usan AsNoTracking cuando no necesitan seguimiento.

Entidad y proyección responden preguntas distintas

La forma del contrato depende de las necesidades de cada caso de uso.



Un JOIN puede servir al contrato. Una proyección de aplicación no debe depender de DTOs definidos en la API.

Interpretación del gráfico: Distingue el modelo necesario para ejecutar reglas de una proyección de lectura; cada contrato tiene un propósito. La figura se distribuye en SVG vectorial y PNG.

Bloque 7 — Repositorio y pruebas con dobles

La capa de aplicación se prueba con dobles del repositorio; eso no prueba el SQL ni el mapeo del adaptador real. El doble sustituye la implementación real y permite verificar el comportamiento del caso de uso sin base de datos. Los dobles más usados son el mock y el stub. El mock verifica que se ha llamado a un método concreto. El stub devuelve un valor predefinido. La combinación de ambos permite probar los casos de uso de forma aislada.

```
var esperada = new OrdenHexagonal("ACX-001", "CHAPA", 10);
var repositorioMock = new Mock<IOrdenRepositorio>();
repositorioMock
    .Setup(r => r.ObtenerAsync(esperada.Id, It.IsAny<CancellationToken>()))
    .ReturnsAsync(esperada);
var servicio = new OrdenesCasoUso(repositorioMock.Object);
var orden = await servicio.ObtenerAsync(esperada.Id);
Assert.Same(esperada, orden);
repositorioMock.Verify(
    r => r.ObtenerAsync(esperada.Id, It.IsAny<CancellationToken>()), Times.Once);
```

- **Línea 1:** `var esperada = new OrdenHexagonal("ACX-001", "CHAPA", 10);` → crea una entidad válida con la misma identidad que buscará el test.
- **Línea 2:** `var repositorioMock = new Mock<IOrdenRepositorio>();` → crea un doble Moq del puerto real.
- **Línea 3:** `repositorioMock` → inicia la configuración del stub.
- **Línea 4:** `.Setup(r => r.ObtenerAsync(esperada.Id, It.IsAny<CancellationToken>()))` → acepta la identidad esperada y cualquier token.
- **Línea 5:** `.ReturnsAsync(esperada);` → configura el valor que devolverá el doble.
- **Línea 6:** `var servicio = new OrdenesCasoUso(repositorioMock.Object);` → construye el caso de uso sin contenedor ni base de datos.
- **Línea 7:** `var orden = await servicio.ObtenerAsync(esperada.Id);` → ejecuta la consulta por la identidad configurada.
- **Línea 8:** `Assert.Same(esperada, orden);` → comprueba el resultado, además de la interacción.
- **Línea 9:** `repositorioMock.Verify(` → inicia la verificación de la llamada.
- **Línea 10:** `r => r.ObtenerAsync(esperada.Id, It.IsAny<CancellationToken>()), Times.Once);` → exige una única consulta del identificador esperado.

Las pruebas con dobles tienen dos consecuencias prácticas. La primera es que los casos de uso se prueban de forma aislada. La segunda es que las pruebas se ejecutan sin base de datos y sin infraestructura. La combinación de ambas reduce el tiempo de ejecución de la suite y facilita el mantenimiento. El ejemplo muestra Moq; también se pueden escribir fakes pequeños sin una biblioteca adicional.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Un mock comprueba interacciones; un stub fija respuestas; un fake ofrece una implementación reducida. Elegir el doble depende de la pregunta que responde la prueba.

Ejemplo concreto en AceriaApp. El ejemplo usa el identificador de la misma OrdenHexagonal que devuelve el doble. Así detecta una consulta con una identidad incorrecta en lugar de aceptar cualquier GUID.

Riesgo, alternativa y límite. Una prueba del caso de uso no demuestra que SQL traduzca un filtro, respete claves foráneas o haga rollback. Para eso se necesita una prueba del adaptador.

Comprobación técnica y aprendizaje. Comprueba tanto el resultado como las interacciones relevantes. Evita verificar cada detalle interno: exigir un orden accidental de llamadas vuelve frágil la prueba sin mejorar el contrato.

Bloque 8 — Repositorio y unidad de trabajo

El repositorio y la unidad de trabajo se complementan. El repositorio gestiona las operaciones sobre una entidad. La unidad de trabajo coordina las operaciones de varios repositorios en una única transacción. La separación entre ambos permite que cada pieza tenga una responsabilidad clara. En esta separación, el repositorio no confirma la operación de negocio por su cuenta. El puerto de unidad de trabajo no necesita exponer entidades, aunque su implementación puede conocerlas a través del contexto. La combinación de ambos permite que la aplicación gestione transacciones complejas sin acoplar los repositorios entre sí.

```
Relación entre Repository y Unit of Work:

Repositorio A → operaciones sobre entidad A.
Repositorio B → operaciones sobre entidad B.
Unidad de trabajo → coordina las operaciones de A y B en una transacción.
```

- **Línea 1:** `Repositorio A → operaciones sobre entidad A.` → primer repositorio.
- **Línea 2:** `Repositorio B → operaciones sobre entidad B.` → segundo repositorio.
- **Línea 3:** `Unidad de trabajo → coordina las operaciones de A y B en una transacción.` → coordinación.

La relación entre repositorio y unidad de trabajo tiene dos consecuencias prácticas. La primera es que el repositorio se mantiene simple. La segunda es que la transacción se gestiona en la capa de aplicación. La combinación de ambas permite que la aplicación controle la atomicidad de las operaciones sin acoplar los repositorios. La unidad de trabajo se estudia en detalle en el punto 4.2.

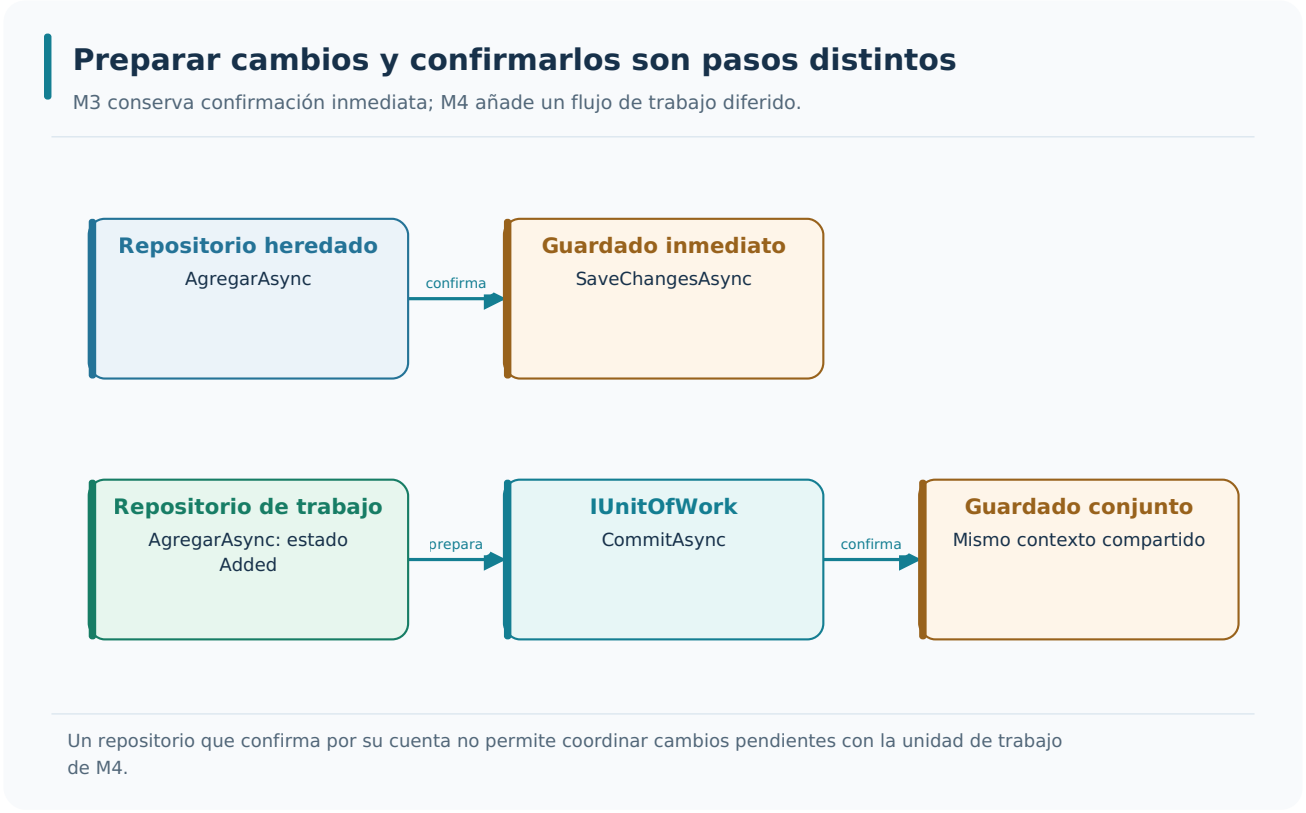
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Separar preparación y confirmación permite agrupar cambios. El requisito crucial es compartir un ámbito transaccional real, no añadir una clase llamada UnitOfWork.

Ejemplo concreto en AceriaApp. IOrdenTrabajoRepositorio.AgregarAsync prepara la entidad; IUnitOfWork.CommitAsync confirma. OrdenesTransaccionales coordina esas dos acciones sin depender directamente de EF Core.

Riesgo, alternativa y límite. Si AgregarAsync ejecutara SaveChanges antes de otro paso que falla, aparecería persistencia parcial. Cambiar nombres de clases no corrige esa frontera equivocada.

Comprobación técnica y aprendizaje. Comprueba que antes del commit una consulta desde otro contexto no ve la nueva orden, y que después del commit sí aparece. Ese test observa persistencia real.



Interpretación del gráfico: Compara confirmación inmediata con preparación de cambios. El flujo transaccional de M4 usa el segundo recorrido. La figura se distribuye en SVG vectorial y PNG.

Bloque 9 — Cuándo usar el patrón Repository

El patrón Repository se usa cuando el sistema accede a una base de datos y el dominio debe permanecer independiente de la tecnología de persistencia. El patrón se usa cuando varias capas necesitan acceder a los mismos datos. El patrón se usa cuando se quiere probar la capa de aplicación con repositorios en memoria. En un CRUD simple o con acceso trivial puede no compensar añadir un envoltorio sobre EF Core. También puede justificarse por una frontera de pruebas, seguridad o consistencia; la decisión depende del beneficio real.

```
Cuándo usar Repository:

Sistema con dominio complejo     → suele aportar valor.
Sistema con varios repositorios  → revisar coordinación.
Sistema con pruebas unitarias    → facilita dobles.
Sistema con CRUD simple          → evaluar coste.
Sistema sin lógica de dominio    → puede no compensar.
```

- **Línea 1:** `Sistema con dominio complejo → suele aportar valor.` → el repositorio aporta valor.
- **Línea 2:** `Sistema con varios repositorios → revisar coordinación.` → el repositorio aporta cohesión.
- **Línea 3:** `Sistema con pruebas unitarias → facilita dobles.` → el repositorio facilita los dobles.
- **Línea 4:** `Sistema con CRUD simple → evaluar coste.` → el envoltorio puede aportar poco sobre el ORM.
- **Línea 5:** `Sistema sin lógica de dominio → puede no compensar.` → el beneficio debe justificar la abstracción.

La decisión de usar el patrón tiene dos consecuencias prácticas. La primera es que el sistema gana independencia de la tecnología de persistencia. La segunda es que el sistema añade una capa de indirección. La combinación de ambas se valora según el contexto del proyecto. El curso aplica el patrón porque el dominio es complejo y se quiere mantener independiente de la base de datos.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Repository compensa cuando el contrato protege una frontera útil. En un CRUD pequeño, usar DbContext directamente en una aplicación acoplada a EF Core puede ser una decisión consciente.

Ejemplo concreto en AceriaApp. AceriaApp lo usa para estudiar puertos y pruebas aisladas. Ese objetivo pedagógico explica la abstracción sin afirmar que todo proyecto necesite la misma cantidad de capas.

Riesgo, alternativa y límite. La posibilidad teórica de cambiar de base de datos no basta para justificar cualquier envoltorio. Valora las necesidades actuales y el coste de conservar dos APIs equivalentes.

Comprobación técnica y aprendizaje. Escribe qué dependencia desaparece del caso de uso y qué pruebas habilita el puerto. Si no puedes explicar ninguno de esos beneficios, revisa la abstracción.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso conserva el repositorio de órdenes de M3 y añade `IOrdenTrabajoRepositorio`, con `ObtenerAsync` y `AgregarAsync`, para el trabajo que confirmará la unidad de trabajo. Los puertos están en aplicación y sus adaptadores de EF Core están en infraestructura. El catálogo también permite estudiar un posible repositorio de inventario, sin presentarlo como persistencia conectada en este módulo. El contrato incorpora las operaciones necesarias; las cinco operaciones CRUD del ejemplo son un catálogo didáctico, no cinco métodos obligatorios. Las consultas complejas se delegan en servicios específicos. La validación combina el test de coherencia de los repositorios con pruebas del adaptador y de los casos de uso conectados.

```
Aplicación al proyecto del curso:

Puertos activos      → IOrdenRepositorio e IOrdenTrabajoRepositorio.
Adaptadores activos  → repositorio inmediato y repositorio de trabajo de órdenes.
Operaciones posibles → GetById, Listar, Add, Update, Delete.
Consultas complejas  → servicios específicos.
Validación           → test de coherencia de repositorios.
```

- **Línea 1:** `Puertos activos` → interfaces de los dos recorridos de órdenes.
- **Línea 2:** `Adaptadores activos` → implementaciones EF Core en infraestructura.
- **Línea 3:** `Operaciones posibles` → el contrato selecciona los métodos que necesita.
- **Línea 4:** `Consultas complejas` → delegadas en servicios específicos.
- **Línea 5:** `Validación` → test de coherencia.

La aplicación del patrón Repository prepara el proyecto para los puntos 4.2, 4.3, 4.4 y 4.5. El punto 4.2 estudiará la unidad de trabajo. El punto 4.3 estudiará el patrón Mediator. El punto 4.4 estudiará el Result Pattern. El punto 4.5 estudiará la inyección de dependencias. El punto 4.1 establece la base que se aplicará en los cuatro puntos siguientes.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El punto mantiene dos evidencias complementarias: el catálogo describe decisiones arquitectónicas y el código ejecuta operaciones que deben respetarlas.

Ejemplo concreto en AceriaApp. El baseline es M3, punto 3.4, con AceriaApp.Api y las tres capas hexagonales. Los tipos de inventario del catálogo son ejemplos de extensión, no un adaptador conectado automáticamente.

Riesgo, alternativa y límite. Un test que valida nombres o metadatos no prueba almacenamiento. Tampoco una respuesta 201 demuestra que la orden pueda consultarse desde una sesión nueva.

Comprobación técnica y aprendizaje. Relaciona cada puerto activo con su registro DI, adaptador y prueba. Usa los nombres y rutas de la práctica para ejecutar la versión concreta del punto.

Resumen rápido de la teoría

- El patrón Repository es una abstracción que media entre la capa de dominio o aplicación y la capa de persistencia.
- Un repositorio tiene tres responsabilidades: recuperar, añadir y modificar entidades.
- El repositorio no decide reglas de negocio ni respuestas HTTP; puede usar consultas complejas y proyecciones acordes con su contrato. La confirmación compartida corresponde a la unidad de trabajo.
- La elección entre repositorio específico y genérico depende de la semántica de los contratos y del coste de mantenerlos.
- La interfaz del repositorio es el puerto de salida de persistencia; su implementación concreta es el adaptador.
- Los repositorios se prueban con dobles en la capa de aplicación.
- El repositorio y la unidad de trabajo se complementan: el repositorio gestiona una entidad, la unidad de trabajo coordina varias.
- El patrón Repository se usa cuando el dominio es complejo y debe permanecer independiente de la tecnología de persistencia.

Punto 4.2 — Unit of Work

Módulo 4: Patrones de arquitectura

Objetivos de aprendizaje:

- Comprender el patrón Unit of Work y su propósito en una arquitectura hexagonal.
- Identificar la relación entre Unit of Work y Repository.
- Comprender la gestión de transacciones como responsabilidad de la unidad de trabajo.
- Comprender el seguimiento de cambios y la coordinación de repositorios.
- Añadir al proyecto `AceriaApp` las clases de validación de la unidad de trabajo y un test unitario que verifique la validación.

Desarrollo teórico

Bloque 1 — Definición del patrón Unit of Work

El patrón Unit of Work es una abstracción que coordina las operaciones de varios repositorios en una única transacción. Su propósito es garantizar que un conjunto de cambios se aplica de forma atómica: o se aplican todos o no se aplica ninguno. La unidad de trabajo mantiene el seguimiento de los objetos que han sido modificados durante una operación y los persiste en bloque. La interfaz de la unidad de trabajo se define en la capa de aplicación. La implementación se sitúa en la capa de infraestructura.

```
Patrón Unit of Work:

Aplicación → IUnitOfWork (puerto)
                    ↑
                    | implementa
```

```
                    |
Infraestructura → EfUnitOfWork (adaptador)
```

- **Línea 1:** `Aplicación → IUnitOfWork (puerto)` → la aplicación define el puerto.
- **Línea 2:** `↑` → la flecha indica que la infraestructura implementa el puerto.
- **Línea 3:** `| implementa` → la infraestructura implementa el puerto.
- **Línea 4:** `Infraestructura → EfUnitOfWork (adaptador)` → la infraestructura contiene el adaptador.

La unidad de trabajo se caracteriza por tres propiedades. La primera es que mantiene el seguimiento de los cambios: sabe qué entidades han sido añadidas, modificadas o eliminadas durante una operación. La segunda es que coordina la persistencia: aplica todos los cambios en una única transacción. La tercera es que expone una operación de confirmación: el cliente decide cuándo se aplican los cambios. Las tres propiedades permiten que la aplicación controle la atomicidad de las operaciones sin acoplar los repositorios entre sí.

La aplicación del patrón Unit of Work tiene dos consecuencias prácticas. La primera es que las operaciones complejas que afectan a varias entidades se ejecutan de forma atómica. La segunda es que los repositorios no gestionan transacciones: solo añaden, modifican o eliminan entidades. La combinación de ambas permite que la aplicación controle la atomicidad sin acoplar los repositorios. La unidad de trabajo es la pieza que coordina los repositorios en una operación compleja.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Una unidad de trabajo reúne cambios relacionados durante un caso de uso. Atomicidad significa que la operación confirmada se aplica completa dentro del recurso transaccional cubierto.

Ejemplo concreto en AceriaApp. OrdenesTransaccionales registra órdenes mediante el puerto de trabajo y confirma por IUnitOfWork. Ambos adaptadores comparten HexagonalDbContext dentro del ámbito de DI.

Riesgo, alternativa y límite. Una transacción no garantiza por sí sola idempotencia de reintentos ni evita todas las carreras. Aislamiento, concurrencia y repetición son decisiones diferentes.

Comprobación técnica y aprendizaje. Formula una operación con dos cambios y provoca el fallo del segundo. Comprueba la base de datos desde un contexto nuevo para detectar cualquier escritura parcial.

Bloque 2 — Relación entre Unit of Work y Repository

El patrón Repository y el patrón Unit of Work se complementan. El repositorio gestiona las operaciones sobre una entidad concreta. La unidad de trabajo coordina las operaciones que participan en el mismo ámbito de persistencia. El repositorio no confirma los cambios: los deja pendientes. La unidad de trabajo confirma los cambios de todos los repositorios en una única transacción. La separación entre ambos permite que cada pieza tenga una responsabilidad clara.

```
public interface IUnitOfWork
{
    Task<int> CommitAsync(CancellationToken ct = default);
}
```

- **Línea 1:** `public interface IUnitOfWork` → interfaz de la unidad de trabajo.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `Task<int> CommitAsync(CancellationToken ct = default);` → método de confirmación.
- **Línea 4:** `}` → cierra el cuerpo.

La relación entre ambos patrones tiene dos consecuencias prácticas. La primera es que el repositorio se mantiene simple: no gestiona transacciones. La segunda es que el puerto se mantiene pequeño: no necesita enumerar las entidades, aunque el contexto de infraestructura las conozca. La combinación de ambas permite que la aplicación componga operaciones complejas a partir de operaciones simples. La composición se realiza en la capa de aplicación con la unidad de trabajo como coordinadora.

La ausencia de la unidad de trabajo produce tres síntomas reconocibles. El primero es la duplicación de la gestión de transacciones en cada repositorio. El segundo es la dificultad de coordinar varias operaciones en una única transacción. El tercero es el riesgo de confirmar cambios parciales. Los tres síntomas motivan revisar el límite transaccional; un DbContext compartido ya puede cubrirlo sin otra clase. La corrección se aplica en la capa de aplicación, donde se compone la operación compleja.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El repositorio expresa acceso a un agregado y la unidad expresa el límite de confirmación. La aplicación decide dónde termina la operación de negocio.

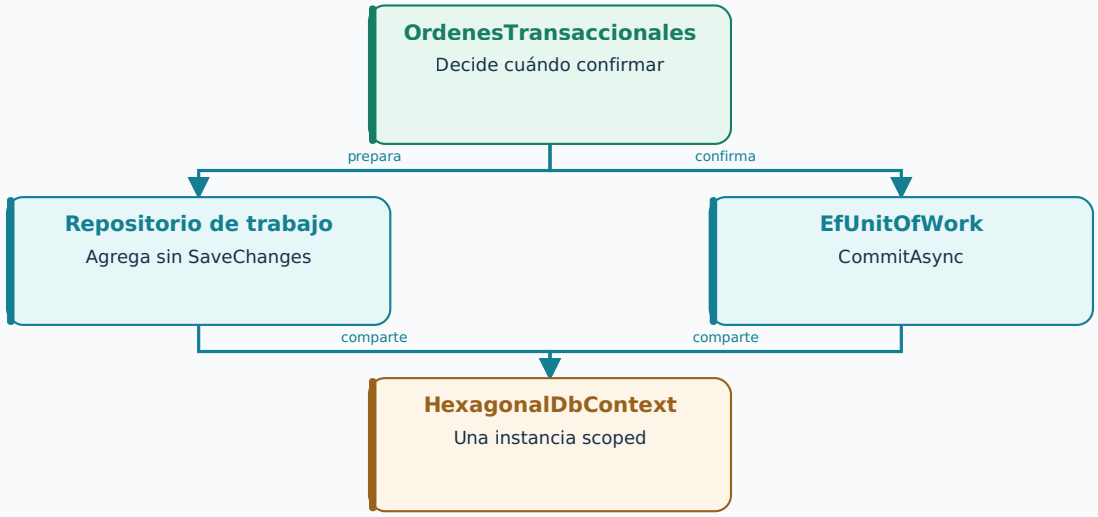
Ejemplo concreto en AceriaApp. El repositorio de trabajo no llama a SaveChanges. Su método AgregarAsync deja una entrada Added, y CommitAsync envía al proveedor todos los cambios del contexto compartido.

Riesgo, alternativa y límite. Un DbContext ya ofrece estas responsabilidades. El puerto adicional aporta independencia a aplicación; no debe duplicar manualmente el seguimiento que el ORM resuelve.

Comprobación técnica y aprendizaje. Busca SaveChanges en los adaptadores de M4 y verifica que la confirmación del flujo transaccional está centralizada. El repositorio heredado de M3 es una variante distinta.

Una unidad de trabajo requiere un contexto compartido

Instancias del mismo ámbito de petición o de operación.



Dos contextos independientes no comparten una transacción solo porque sus clases estén registradas como scoped.

Interpretación del gráfico: Los dos puertos comparten una misma instancia de contexto y una misma frontera de confirmación. La figura se distribuye en SVG vectorial y PNG.

Bloque 3 — Responsabilidades de la unidad de trabajo

La unidad de trabajo tiene tres responsabilidades principales. La primera es mantener el seguimiento de las entidades modificadas durante una operación. El ORM mantiene identidades y estados de seguimiento. El mapa de identidad evita materializar varias instancias seguidas con la misma clave; el seguimiento de cambios registra además qué debe persistirse. El estado puede ser añadido, modificado, eliminado o sin cambios. La segunda responsabilidad es coordinar la persistencia: aplicar todos los cambios en una única transacción. La tercera responsabilidad es exponer la operación de confirmación: el cliente decide cuándo se aplican los cambios.

Ejemplo de una **variante con reversión explícita**. Para implementarla se debe definir y conservar la transacción abierta; no se usa como sustituto del puerto mínimo de los bloques 5 y 7.

```
public interface IUnidadTransaccional
{
    Task<int> CommitAsync(CancellationToken ct = default);
    Task RollbackAsync();
}
```

- **Línea 1:** `public interface IUnidadTransaccional` → interfaz de la unidad de trabajo.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `Task<int> CommitAsync(CancellationToken ct = default);` → método de confirmación.
- **Línea 4:** `Task RollbackAsync();` → método de reversión.
- **Línea 5:** `}` → cierra el cuerpo.

La confirmación se expresa con `CommitAsync`. Algunas interfaces añaden `RollbackAsync`, pero ese nombre exige definir si revierte una transacción abierta o descarta cambios que todavía no se han enviado. No son equivalentes. Una interfaz mínima puede exponer solo confirmación y dejar que la eliminación del ámbito descarte el trabajo no confirmado.

La definición de las responsabilidades tiene dos consecuencias prácticas. La primera es que la unidad de trabajo es una abstracción de la transacción, no del dominio. La segunda es que el puerto puede ocultar las entidades concretas mientras la implementación gestiona sus cambios pendientes. La combinación de ambas permite que la unidad de trabajo sea simple y sustituible. La unidad de trabajo se limita a las operaciones básicas de confirmación y reversión.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Confirmar, deshacer una transacción abierta y abandonar cambios en memoria son operaciones diferentes. El contrato debe decir cuál ofrece, especialmente si usa la palabra Rollback.

Ejemplo concreto en AceriaApp. El puerto mínimo de M4 expone CommitAsync(ct). Si no se confirma y se desecha el ámbito, los cambios preparados desaparecen con el contexto sin haberse persistido.

Riesgo, alternativa y límite. Un RollbackAsync que solo espera Task.CompletedTask promete un efecto que no realiza. Marcar entradas Unchanged tampoco deshace escrituras ya confirmadas en SQL Server.

Comprobación técnica y aprendizaje. Comprueba por separado el estado del objeto y el estado almacenado. Después de una reversión, no reutilices ciegamente objetos que todavía contienen valores del intento fallido.

Bloque 4 — Lo que una unidad de trabajo no debe hacer

Una unidad de trabajo no debe contener reglas de negocio. Las reglas de negocio residen en las entidades del dominio. El contrato de unidad de trabajo no necesita enumerar entidades concretas; su

implementación sí puede conocerlas mediante el contexto. Una unidad de trabajo no debe transformar entidades en DTOs. Una unidad de trabajo no debe ejecutar consultas complejas. Una unidad de trabajo no debe gestionar la lógica de los repositorios.

```
Lo que una unidad de trabajo no debe hacer:

- Contener reglas de negocio.
- Exponer detalles de cada entidad en el puerto de confirmación.
- Transformar entidades en DTOs.
- Ejecutar consultas complejas.
- Gestionar la lógica de los repositorios.
```

- **Línea 1:** `- Contener reglas de negocio.` → las reglas residen en el dominio.
- **Línea 2:** `- Exponer detalles de cada entidad en el puerto de confirmación.` → la unidad de trabajo gestiona cambios pendientes.
- **Línea 3:** `- Transformar entidades en DTOs.` → la transformación reside en la aplicación.
- **Línea 4:** `- Ejecutar consultas complejas.` → las consultas residen en los repositorios.
- **Línea 5:** `- Gestionar la lógica de los repositorios.` → los repositorios gestionan su propia lógica.

La delimitación de responsabilidades tiene dos consecuencias prácticas. La primera es que la unidad de trabajo se mantiene simple y cohesionada. La segunda es que las reglas de negocio y las validaciones no se duplican entre la unidad de trabajo y el dominio. La combinación de ambas permite que la unidad de trabajo sea sustituible sin afectar a la lógica de negocio. La disciplina en la delimitación es lo que distingue una unidad de trabajo correcta de una que acumula responsabilidades.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La unidad debe coordinar persistencia; no decidir precios, cantidades máximas ni respuestas HTTP. La política de cuándo confirmar corresponde al caso de uso.

Ejemplo concreto en AceriaApp. OrdenHexagonal protege cantidad positiva; OrdenesTransaccionales coordina el alta; el adaptador EfUnitOfWork delega la escritura en EF Core. Son responsabilidades distintas aunque compartan una petición.

Riesgo, alternativa y límite. El contexto conoce el modelo y las relaciones. Eso no vulnera por sí mismo el patrón: la abstracción del núcleo puede seguir siendo una operación de confirmación pequeña.

Comprobación técnica y aprendizaje. Lee los parámetros de CommitAsync. No debería recibir HttpContext ni instrucciones de negocio ajenas a persistencia; sí puede recibir un CancellationToken.

Bloque 5 — Seguimiento de cambios

El seguimiento de cambios es la primera responsabilidad de la unidad de trabajo. La unidad de trabajo registra cada operación que modifica una entidad. El registro incluye el tipo de operación y la entidad afectada. Cuando el cliente confirma los cambios, la unidad de trabajo aplica todas las operaciones en el orden adecuado. EF Core calcula el orden según el grafo de dependencias, claves foráneas y restricciones. No existe una regla general que ordene siempre todas las inserciones antes de todas las eliminaciones.

```
public sealed class EfUnitOfWork(HexagonalDbContext db) : IUnitOfWork
{
    public Task<int> CommitAsync(CancellationToken ct = default) =>
```

```
        db.SaveChangesAsync(ct);
}
```

- **Línea 1:** `public sealed class EfUnitOfWork(HexagonalDbContext db) : IUnitOfWork` → recibe el mismo contexto scoped que usan los repositorios.
- **Línea 2:** `{` → abre la implementación del puerto mínimo.
- **Línea 3:** `public Task<int> CommitAsync(CancellationToken ct = default) =>` → expone la confirmación y propaga cancelación.
- **Línea 4:** `db.SaveChangesAsync(ct);` → delega el seguimiento y la escritura atómica en EF Core.
- **Línea 5:** `}` → cierra la clase.

El seguimiento de cambios tiene dos consecuencias prácticas. La primera es que la unidad de trabajo conoce el estado de cada entidad. La segunda es que la confirmación aplica todos los cambios en una única transacción. La combinación de ambas permite que la aplicación controle la atomicidad de las operaciones. El seguimiento de cambios se delega en el `DbContext` de EF Core cuando se usa Entity Framework Core.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El seguimiento compara valores y estados para producir las operaciones necesarias. No es una lista artesanal de SQL que la aplicación deba mantener.

Ejemplo concreto en AceriaApp. Una OrdenHexagonal nueva se añade al contexto y queda pendiente. Una entidad recuperada con seguimiento puede modificarse mediante un método de dominio y guardarse en el siguiente commit.

Riesgo, alternativa y límite. AsNoTracking evita ese seguimiento normal. Además, DbContext no permite operaciones concurrentes: no ejecutes consultas paralelas con la misma instancia por usar métodos asíncronos.

Comprobación técnica y aprendizaje. Inspecciona ChangeTracker.Entries durante una prueba y contrasta el estado antes y después de SaveChanges. No confundas ese estado de memoria con la durabilidad de una transacción externa.

Bloque 6 — EF Core como unidad de trabajo

Entity Framework Core implementa el patrón Unit of Work en su clase `DbContext`. El `DbContext` sigue las entidades cargadas con seguimiento y las adjuntadas o añadidas. Las consultas `AsNoTracking` no incorporan sus resultados al seguimiento normal. El método `SaveChanges` confirma los cambios pendientes. El método `SaveChangesAsync` confirma los cambios de forma asíncrona. Los `DbSet` ofrecen una abstracción de colección con responsabilidades semejantes a Repository. La implementación de ambos patrones en la misma clase es una decisión de diseño de EF Core.

Contexto del ejemplo: se supone un modelo didáctico con conjuntos `Ordenes` y `Movimientos`. Ilustra cambios en dos agregados; el contexto de órdenes de la práctica no incorpora automáticamente un `DbSet` de movimientos. `_db`, `orden` y `movimiento` son el contexto y los objetos preparados por el caso de uso.

```
await using var tx = await _db.Database.BeginTransactionAsync();
try
{
    _db.Ordenes.Add(orden);
    _db.Movimientos.Add(movimiento);
    await _db.SaveChangesAsync();
    await tx.CommitAsync();
}
catch
{
    await tx.RollbackAsync();
```

```
    throw;
}
```

- **Línea 1:** `await using var tx = await _db.Database.BeginTransactionAsync();` → abre una transacción.
- **Línea 2:** `try` → inicia el bloque protegido.
- **Línea 3:** `{` → abre el bloque.
- **Línea 4:** `_db.Ordenes.Add(orden);` → añade la orden.
- **Línea 5:** `_db.Movimientos.Add(movimiento);` → añade el movimiento.
- **Línea 6:** `await _db.SaveChangesAsync();` → persiste los cambios.
- **Línea 7:** `await tx.CommitAsync();` → confirma la transacción.
- **Línea 8:** `}` → cierra el bloque.
- **Línea 9:** `catch` → captura cualquier error.
- **Línea 10:** `{` → abre el bloque de error.
- **Línea 11:** `await tx.RollbackAsync();` → revierte la transacción.
- **Línea 12:** `throw;` → propaga la excepción.
- **Línea 13:** `}` → cierra el bloque de error.

La implementación de EF Core tiene dos consecuencias prácticas. La primera es que el `DbContext` ya implementa el patrón Unit of Work de forma nativa. La segunda es que los `DbSet` ya cubren muchas operaciones de un repositorio. La combinación de ambas permite que el proyecto use EF Core directamente sin necesidad de envoltorios adicionales. Sin embargo, la separación entre los patrones facilita las pruebas y la sustitución de la tecnología.

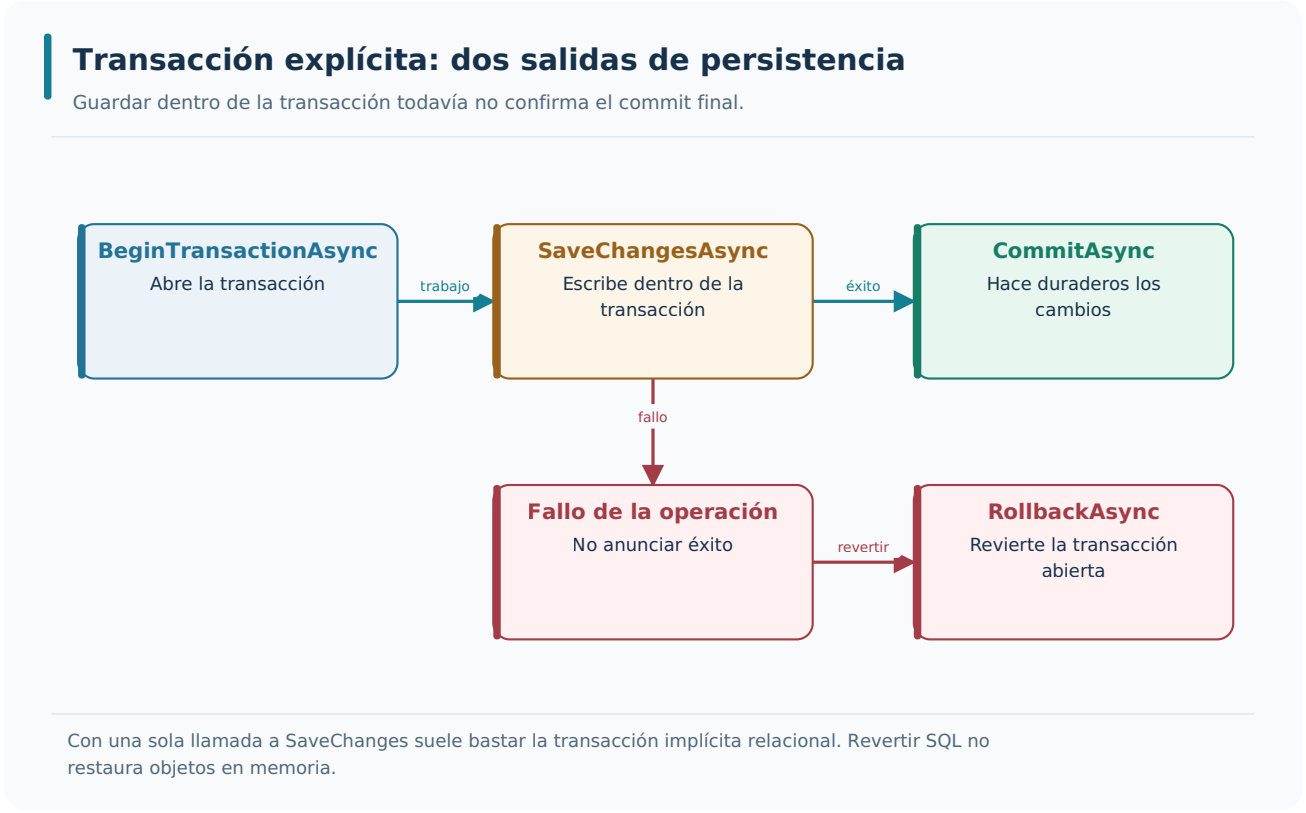
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Una llamada a SaveChanges usa atomicidad del proveedor relacional por defecto. Una transacción explícita es útil si necesitas cubrir varias llamadas u otras operaciones compatibles.

Ejemplo concreto en AceriaApp. El ejemplo muestra begin, guardado y commit explícitos. Con una sola llamada de guardado, suele bastar la transacción implícita del proveedor de EF Core.

Riesgo, alternativa y límite. SaveChanges dentro de una transacción abierta todavía no equivale a commit final. Reintentos configurados requieren ejecutar la transacción completa mediante la estrategia correspondiente.

Comprobación técnica y aprendizaje. La práctica usa `UseSqlServer` y un motor SQL Server real para comprobar restricciones y persistencia. EF Core InMemory no reproduce transacciones relacionales ni demuestra que el rollback SQL funcione; SQLite tiene transacciones, pero no valida el comportamiento específico de SQL Server.



Interpretación del gráfico: Guardar dentro de una transacción abierta y confirmar esa transacción son pasos distintos. La rama de fallo cancela su persistencia. La figura se distribuye en SVG vectorial y PNG.

Bloque 7 — Unidad de trabajo personalizada

Una unidad de trabajo personalizada envuelve el `DbContext` y expone solo las operaciones necesarias para la aplicación. La personalización tiene dos objetivos. El primero es desacoplar la aplicación de EF Core: la aplicación solo conoce la interfaz `IUnitOfWork`. El segundo es exponer solo las operaciones que la aplicación necesita: la interfaz es más pequeña que la del `DbContext`. La personalización facilita las pruebas y la sustitución de la tecnología de persistencia.

```
public interface IUnitOfWork
{
    Task<int> CommitAsync(CancellationToken ct = default);
}
public sealed class EfUnitOfWork(HexagonalDbContext db) : IUnitOfWork
{
    public Task<int> CommitAsync(CancellationToken ct = default) =>
        db.SaveChangesAsync(ct);
    public void DescartarCambiosPendientes() => db.ChangeTracker.Clear();
}
```

- **Línea 1:** `public interface IUnitOfWork` → declara un puerto mínimo en aplicación.
- **Línea 2:** { → abre el contrato.
- **Línea 3:** `Task<int> CommitAsync(CancellationToken ct = default);` → confirma los cambios pendientes del contexto compartido.
- **Línea 4:** } → cierra el contrato.
- **Línea 5:** `public sealed class EfUnitOfWork(HexagonalDbContext db) : IUnitOfWork` → implementa el puerto en infraestructura con un contexto externo.
- **Línea 6:** { → abre la implementación.

- **Línea 7:** `public Task<int> CommitAsync(CancellationToken ct = default) =>` → acepta cancelación desde el caso de uso.
- **Línea 8:** `db.SaveChangesAsync(ct);` → guarda las entradas modificadas; no confirma una transacción externa.
- **Línea 9:** `public void DescartarCambiosPendientes() => db.ChangeTracker.Clear();` → abandona el seguimiento local; este método no revierte datos ya confirmados.
- **Línea 10:** `}` → cierra la implementación.

La unidad de trabajo personalizada tiene dos consecuencias prácticas. La primera es que la aplicación solo conoce la interfaz `IUnitOfWork`. La segunda es que la implementación puede sustituirse sin modificar la aplicación. La combinación de ambas permite que el núcleo se pruebe con una unidad de trabajo en memoria y que la tecnología de persistencia evolucione con independencia del núcleo.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Una interfaz personalizada debe ser más pequeña y más clara que el detalle que encapsula. Ocultar DbContext no obliga a reimplementar todos sus métodos.

Ejemplo concreto en AceriaApp. CommitAsync devuelve el número de entradas de estado guardadas por EF Core. No representa necesariamente el número exacto de filas SQL afectadas ni es un identificador de negocio.

Riesgo, alternativa y límite. ChangeTracker.Clear abandona seguimiento. No restaura propiedades de los objetos que conserva el llamador, no hace rollback de una transacción abierta y no revierte un commit anterior.

Comprobación técnica y aprendizaje. Tras abandonar o revertir una operación fallida, termina el ámbito y usa un contexto nuevo para continuar cuando sea necesario. Haz explícita esa política de ciclo de vida.

Bloque 8 — Unidad de trabajo y transacciones distribuidas

Una unidad de trabajo coordina las operaciones de varios repositorios en una única transacción. En los ejemplos del curso la transacción es local y usa una base de datos y una conexión compartida. Cuando las operaciones afectan a varias bases de datos o a varios servicios, la transacción local no es suficiente. Entre servicios pueden usarse sagas con compensaciones y outbox para publicación fiable. Two-Phase Commit es un protocolo distinto que requiere participantes compatibles y tiene costes operativos; una saga no proporciona la misma atomicidad que una transacción ACID. La unidad de trabajo se limita a la transacción local.

```
Alcance de la unidad de trabajo:

Transacción local    → una única base de datos.
Transacción distribuida → varias bases de datos o servicios.
```

- **Línea 1:** `Transacción local → una única base de datos.` → alcance de la unidad de trabajo.
- **Línea 2:** `Transacción distribuida → varias bases de datos o servicios.` → requiere patrones adicionales.

El alcance de la unidad de trabajo tiene dos consecuencias prácticas. La primera es que la implementación puede garantizar atomicidad dentro de la transacción local que realmente comparte. La segunda es que la atomicidad entre bases de datos requiere patrones adicionales. La combinación de ambas define el ámbito de aplicación de la unidad de trabajo. La unidad de trabajo se aplica a las transacciones locales y se complementa con patrones distribuidos cuando es necesario.

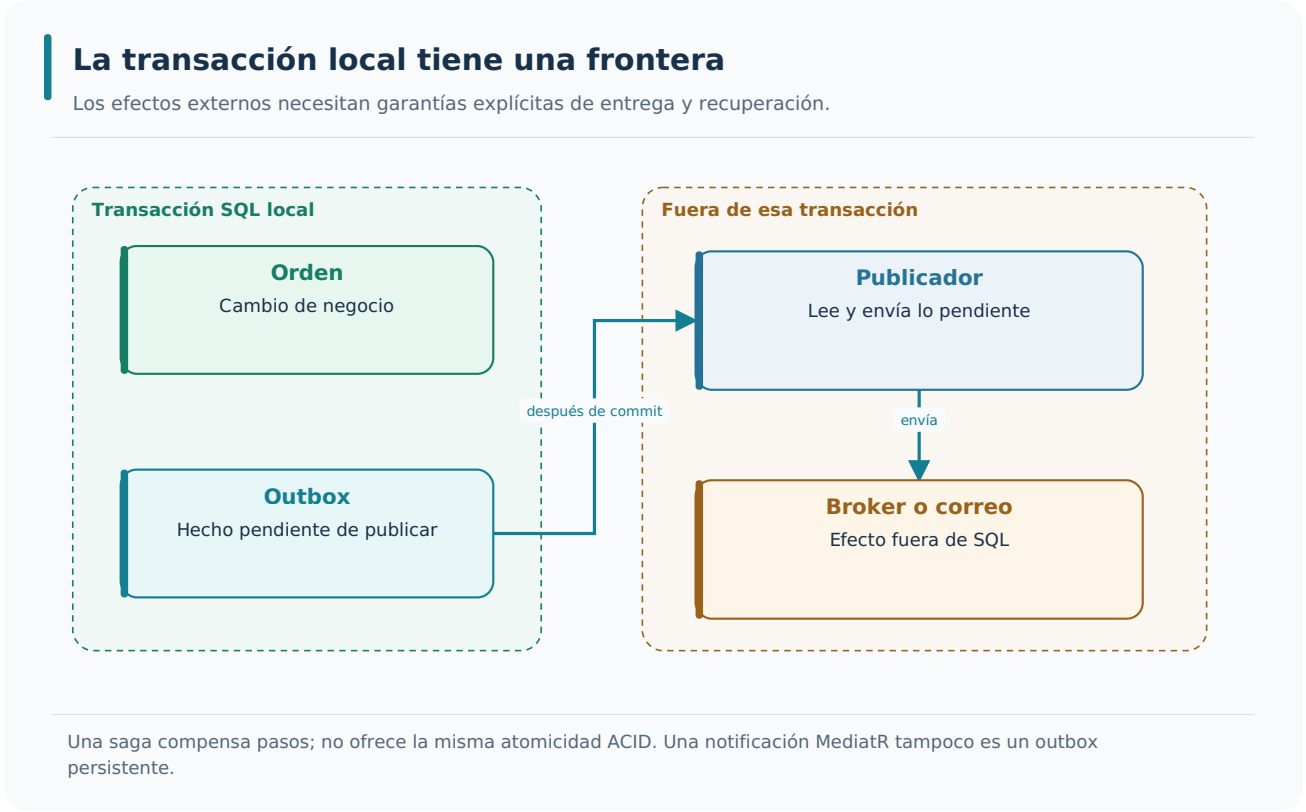
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La atomicidad local termina donde acaba la transacción compartida. Enviar un correo o publicar en un broker introduce efectos que no revierten con un rollback de SQL.

Ejemplo concreto en AceriaApp. Si se confirma una orden y después falla la publicación, la orden sigue guardada. Un outbox conserva el hecho pendiente junto con la orden para enviarlo posteriormente.

Riesgo, alternativa y límite. Una saga compensa pasos ya aplicados y admite estados intermedios; no convierte toda la conversación entre servicios en una transacción ACID única.

Comprobación técnica y aprendizaje. Dibuja qué recursos cubre la transacción y qué sucede si cae el proceso tras confirmar. Una notificación MediatR en memoria no elimina esa ventana de fallo.



Interpretación del gráfico: Identifica qué recursos cubre la transacción local y qué efectos necesitan un diseño adicional de entrega o compensación. La figura se distribuye en SVG vectorial y PNG.

Bloque 9 — Cuándo usar el patrón Unit of Work

El patrón Unit of Work se usa cuando una operación afecta a varias entidades y debe ejecutarse de forma atómica. El patrón se usa cuando la aplicación necesita coordinar varios repositorios. El patrón se usa cuando se quiere desacoplar la aplicación de la tecnología de transacciones. Una operación sobre una sola entidad también puede formar una unidad de trabajo. Lo que puede ser innecesario es crear un envoltorio adicional cuando el DbContext ya satisface el requisito de confirmación y aislamiento.

```
Cuándo usar Unit of Work:

Operación con varias entidades → sí.
Coordinación de repositorios   → sí.
Desacoplamiento de transacciones → sí.
Operación con una entidad      → DbContext puede bastar.
Sin requisito de atomicidad    → evaluar abstracción.
```

- **Línea 1:** `Operación con varias entidades → sí.` → el patrón aporta atomicidad.

- **Línea 2:** `Coordinación de repositorios → sí.` → el patrón coordina las operaciones.
- **Línea 3:** `Desacoplamiento de transacciones → sí.` → el patrón desacopla la aplicación.
- **Línea 4:** `Operación con una entidad → DbContext puede bastar.` → no siempre hace falta otro envoltorio.
- **Línea 5:** `Sin requisito de atomicidad → evaluar abstracción.` → el beneficio debe justificar la abstracción.

La decisión de usar el patrón tiene dos consecuencias prácticas. La primera es que la aplicación gana independencia de la tecnología de transacciones. La segunda es que la aplicación añade una capa de indirección. La combinación de ambas se valora según el contexto del proyecto. El curso aplica el patrón porque el dominio tiene operaciones que afectan a varias entidades.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El patrón puede existir aunque solo haya una entidad modificada. La decisión relevante es si necesitas otra abstracción sobre la unidad que ya proporciona EF Core.

Ejemplo concreto en AceriaApp. AceriaApp añade IUnitOfWork para que aplicación ordene la confirmación sin importar DbContext. El valor pedagógico es ver y probar esa frontera.

Riesgo, alternativa y límite. No mantengas transacciones abiertas mientras esperas al usuario o a un servicio remoto lento. El bloqueo de recursos debe corresponder al trabajo de persistencia necesario.

Comprobación técnica y aprendizaje. Documenta quién inicia la operación, quién confirma y quién termina el ámbito. Si hay más de un responsable de confirmar, revisa el diseño antes de añadir más repositorios.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica el patrón Unit of Work en el módulo 4. La unidad de trabajo se define como puerto en la capa de aplicación. Se implementa como adaptador en la capa de infraestructura. Los repositorios no confirman los cambios: los dejan pendientes. La unidad de trabajo confirma los cambios de todos los repositorios en una única transacción. La validación combina el catálogo con pruebas de confirmación y ausencia de persistencia parcial en una base relacional.

```
Aplicación al proyecto del curso:

Puerto de salida    → IUnitOfWork.
Adaptador de salida → EfUnitOfWork.
Repositorios        → no confirman cambios.
Unidad de trabajo   → confirma los cambios.
Validación          → test de coherencia.
```

- **Línea 1:** `Puerto de salida` → interfaz de la unidad de trabajo en la aplicación.
- **Línea 2:** `Adaptador de salida` → implementación en la infraestructura.
- **Línea 3:** `Repositorios` → no confirman cambios.
- **Línea 4:** `Unidad de trabajo` → confirma los cambios.
- **Línea 5:** `Validación` → test de coherencia.

La aplicación del patrón Unit of Work prepara el proyecto para los puntos 4.3, 4.4 y 4.5. El punto 4.3 estudiará el patrón Mediator. El punto 4.4 estudiará el Result Pattern. El punto 4.5 estudiará la inyección de dependencias. El punto 4.2 establece la base que se aplicará en los tres puntos siguientes.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El catálogo describe el patrón y las pruebas relacionales demuestran el comportamiento. Ambas piezas deben hablar del mismo límite de confirmación.

Ejemplo concreto en AceriaApp. La progresión conserva el comportamiento previo y añade el flujo de trabajo con IOrdenTrabajoRepositorio, IUnitOfWork y OrdenesTransaccionales. No atribuyas staging al repositorio heredado que guarda inmediatamente.

Riesgo, alternativa y límite. Una prueba con fake que cuenta commits demuestra orquestación, pero no atomicidad SQL. Una prueba real de restricciones detecta fallos que el fake nunca produciría.

Comprobación técnica y aprendizaje. Ejecuta un alta válida y un fallo que deje la base sin cambios parciales. El punto siguiente reutiliza esos puertos desde un manejador MediatR.

Resumen rápido de la teoría

- El patrón Unit of Work es una abstracción que coordina las operaciones de varios repositorios en una única transacción.
- Una unidad de trabajo tiene tres responsabilidades: mantener el seguimiento de cambios, coordinar la persistencia y exponer la operación de confirmación.
- La unidad de trabajo coordina persistencia sin asumir reglas de negocio ni respuestas HTTP; su contrato no necesita enumerar entidades, aunque su implementación las conozca.
- EF Core implementa el patrón Unit of Work en su clase `DbContext`.
- Una unidad de trabajo personalizada envuelve el `DbContext` y expone solo las operaciones necesarias.
- La unidad de trabajo se limita a la transacción local: la transacción distribuida requiere patrones adicionales.
- El patrón se usa cuando una operación afecta a varias entidades y debe ejecutarse de forma atómica.
- El patrón se complementa con el patrón Repository para coordinar las operaciones de varios repositorios.

Estudio integrador 4.2 — Demostrar atomicidad en SQL Server

Preparar cambios, enviarlos y confirmarlos son momentos distintos

Un repositorio de trabajo puede agregar dos órdenes al `DbContext` sin enviarlas todavía a la base. EF Core mantiene esas entidades en su seguimiento de cambios. En ese momento, cerrar el contexto descarta los cambios pendientes: no es un rollback de SQL porque aún no se ha ejecutado la escritura. Al llamar a `SaveChangesAsync`, el proveedor prepara y ejecuta los comandos necesarios. Si ese conjunto necesita una transacción, EF Core usa el comportamiento transaccional predeterminado del proveedor relacional.

En la implementación de `EfUnitOfWork`, `CommitAsync` delega en un único `SaveChangesAsync`. Ese es el límite de confirmación elegido para el caso de uso. Si una operación requiriera varios `SaveChanges` dentro de una única unidad atómica, habría que diseñar una transacción explícita que los abarcase. Añadir un método llamado `RollbackAsync` que no accione una transacción no proporciona esa garantía. La documentación de transacciones de EF Core explica el comportamiento de una confirmación y el control explícito.

Qué pregunta responde cada clase de prueba

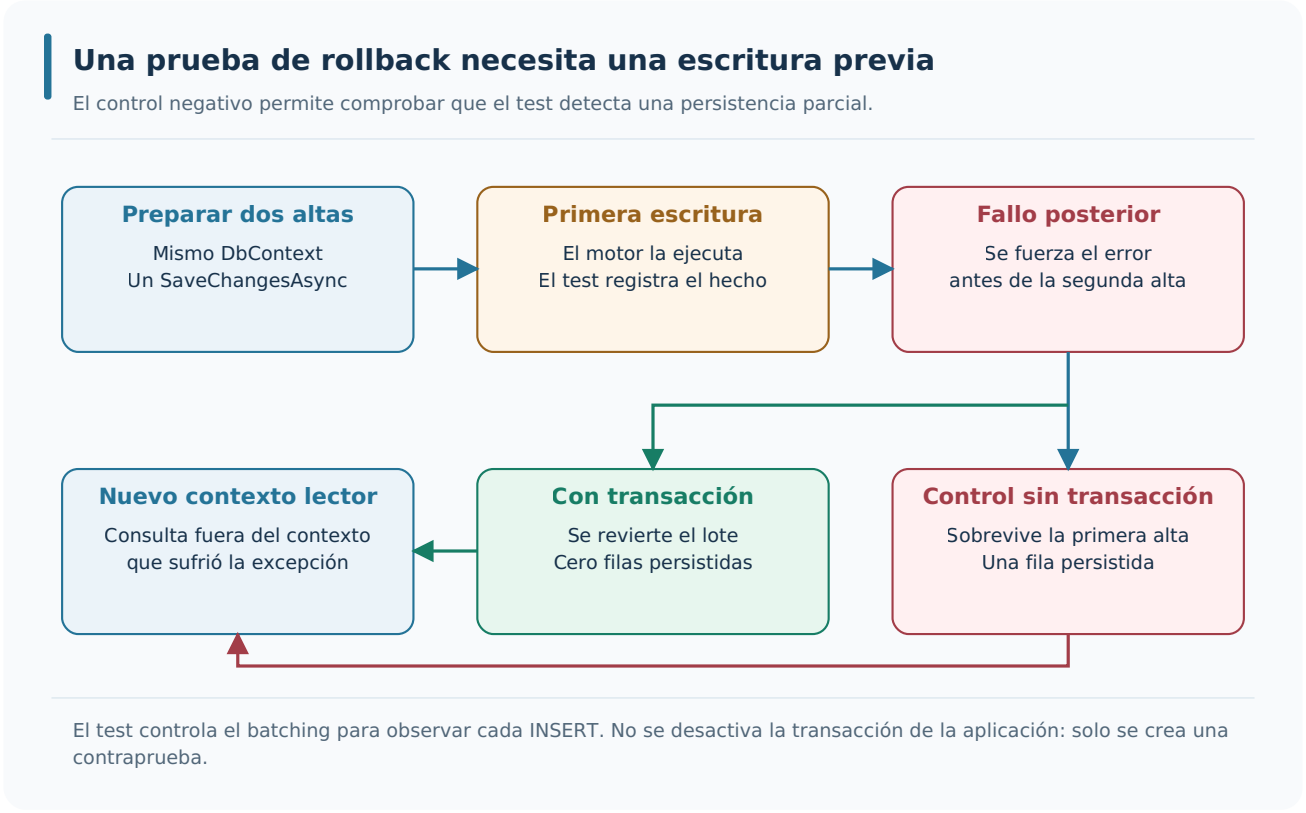
Técnica	Sirve para observar	Límite de lo demostrado
Doble de `IUnitOfWork`	Si el caso de uso solicita confirmar	Contar llamadas no ejecuta ni revierte SQL.
Contexto con EF InMemory	Operaciones sobre su almacenamiento no relacional	No demuestra una transacción relacional.
SQLite	Transacciones y reglas de SQLite	Su comportamiento no certifica el proveedor SQL Server.
SQL Server aislado	Escrituras, restricciones y rollback con `UseSqlServer`	Demuestra los escenarios ejecutados en esa configuración.

El laboratorio usa el último camino para comprobar persistencia y atomicidad. `BaseSqlServer.Crear()` devuelve una conexión a una base única; Testcontainers obtiene SQL Server real cuando no se proporciona `ACERIA_TEST_SQLSERVER`. La misma prueba puede usar LocalDB en Windows mediante esa variable. Cambiar el servidor no convierte una prueba de SQL en una prueba de dominio: sigue dependiendo de un recurso real y debe fallar si ese recurso no está disponible.

Por qué una excepción no basta para demostrar rollback

Supongamos que se preparan dos órdenes y el test provoca un error antes de enviar la primera. Después consulta la base y encuentra cero filas. Ese resultado sería idéntico aunque no existiera transacción, porque nunca llegó a producirse una escritura. La prueba no ha demostrado que se deshaga nada.

Para distinguir ambos casos se necesita **una escritura real anterior al fallo**. La prueba de `UnitOfWorkFuncionalesTests` provoca el error después de observar el primer INSERT y antes de completar la siguiente escritura. La configuración `sql.MaxBatchSize(1)` impide que el proveedor SQL Server agrupe ambas altas de una forma que impida observar ese punto. El interceptor busca `INSERT INTO` dentro del comando, porque SQL Server puede anteponer otras instrucciones como `SET`. Esta configuración pertenece al escenario de prueba; la aplicación no necesita usar ese tamaño de lote para funcionar correctamente.



Interpretación del gráfico: Las dos ramas repiten el mismo fallo controlado. La rama transaccional deja cero filas; el control sin transacción conserva la primera. La diferencia demuestra que la prueba es capaz de detectar una persistencia parcial. La figura se distribuye en SVG y PNG.

El test utiliza `AutoTransactionBehavior.WhenNeeded`, el comportamiento habitual, y lo contrasta con `AutoTransactionBehavior.Never` en una base independiente. La segunda variante es un **control negativo**: se elimina deliberadamente la protección que se está comprobando para ver si cambia el resultado. No es una recomendación para la configuración de la API.

Este extracto muestra la parte que observa el resultado; el montaje de los repositorios y del interceptor aparece completo en las prácticas:

```
db.Database.AutoTransactionBehavior = transacciones;
await Assert.ThrowsAsync<DbUpdateException>(
    () => new EfUnitOfWork(db).CommitAsync());
Assert.Equal(1, fallo.InsertsEjecutados);
await using var verificacion = Contexto(store);
var filasEsperadas = transacciones == AutoTransactionBehavior.Never ? 1 : 0;
Assert.Equal(filasEsperadas, await verificacion.Ordenes.CountAsync());
```

- **Línea 1:** Selecciona la política que se está examinando en ese caso aislado. Las entidades ya se han preparado en el contexto.
- **Línea 2:** Exige que el fallo de persistencia se manifieste como la excepción esperada. Un test que no alcanza el error no debe pasar.
- **Línea 3:** Invoca la misma implementación de unidad de trabajo que utiliza el caso de uso.
- **Línea 4:** Comprueba que una escritura se ejecutó antes del fallo. Sin esta observación, cero filas no demostraría una reversión.
- **Línea 5:** Abre otro contexto contra la misma base de ese caso. La comprobación no depende del seguimiento del contexto que falló.
- **Línea 6:** Expresa la diferencia esperada: el control sin transacción conserva una fila; el caso protegido no conserva ninguna.
- **Línea 7:** Consulta SQL Server y contrasta las filas persistidas con esa expectativa.

Recuperar la aplicación después de un fallo

El rollback se refiere al estado de la base de datos. Las instancias de objetos que ya se han modificado en memoria pueden conservar esos valores después de la excepción. `ChangeTracker.Clear()` abandona el seguimiento de EF Core, pero tampoco rebobina las propiedades de los objetos que aún conserva el llamador. Tras un fallo, el curso termina el ámbito afectado y verifica el resultado con otro contexto. Ese procedimiento hace visible la diferencia entre memoria de aplicación y almacenamiento confirmado.

También hay que situar la frontera: dos repositorios comparten atomicidad cuando participan en el mismo contexto y transacción. Tener dos objetos llamados `EfUnitOfWork` conectados a dos bases diferentes no crea una transacción distribuida. Enviar un mensaje o modificar un archivo tampoco se deshace por revertir SQL. Esos problemas requerirían otra coordinación y quedan fuera de la garantía que se prueba en este punto.

Comprobación de comprensión. Explica por qué «no hice commit», «falló antes de escribir» y «escribí y se revirtió» son tres hechos distintos. Después identifica en el test la observación que descarta el segundo caso, la consulta que verifica el tercero y el control que haría visible la ausencia de transacción. Esa lectura permite defender el resultado sin apoyarse únicamente en un contador de pruebas.

Punto 4.3 — Mediator (MediatR)

Módulo 4: Patrones de arquitectura

Objetivos de aprendizaje:

- Comprender el patrón Mediator y su propósito en una arquitectura hexagonal.
- Identificar la relación entre Mediator y los casos de uso.
- Comprender la diferencia entre comandos, consultas y notificaciones.
- Comprender el papel de MediatR como implementación del patrón.
- Añadir al proyecto `AceriaApp` las clases de validación del patrón Mediator y un test unitario que verifique la validación.

Desarrollo teórico

Bloque 1 — Definición del patrón Mediator

El patrón Mediator es un patrón de comportamiento que reduce el acoplamiento entre los componentes de un sistema al centralizar la comunicación en un objeto mediador. En lugar de que los componentes se conozcan entre sí, cada componente envía mensajes al mediador y el mediador los distribuye al componente adecuado. El patrón se aplica en sistemas donde varios componentes necesitan comunicarse sin conocerse. El mediador actúa como intermediario único. Los componentes solo conocen al mediador, no a los demás componentes.

```
Patrón Mediator:

Componente A ──┐
               │
Componente B ──┼──► Mediator ──► Componente destino
               │
Componente C ──┘
```

- **Línea 1:** `Componente A ──┐` → primer componente que envía mensajes.

- **Línea 2:** `  │` → línea de conexión al mediador.
- **Línea 3:** `Componente B ──┼──► Mediator` → segundo componente conectado al mediador.
- **Línea 4:** `  │` → línea de conexión al mediador.
- **Línea 5:** `Componente C ──┘` → tercer componente conectado al mediador.
- **Línea 6:** `Mediator ──► Componente destino` → el mediador distribuye al destino.

El patrón Mediator se caracteriza por tres propiedades. La primera es que reduce el acoplamiento: los componentes no se conocen entre sí. La segunda es que centraliza la comunicación: el mediador conoce todos los componentes. La tercera es que facilita la evolución: añadir un componente nuevo solo requiere registrarlo en el mediador. Las tres propiedades permiten que el sistema evolucione sin modificar los componentes existentes.

La aplicación del patrón Mediator tiene dos consecuencias prácticas. La primera es que los componentes se vuelven más simples: solo envían mensajes y no conocen a los receptores. La segunda es que el mediador se convierte en el punto central de la comunicación. La combinación de ambas permite que el sistema tenga un acoplamiento bajo entre los componentes. El mediador se usa en sistemas con muchos componentes que necesitan comunicarse de forma flexible.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El mediador sustituye dependencias directas entre emisores y receptores por contratos de mensajes. El acoplamiento al contrato sigue existiendo y debe diseñarse.

Ejemplo concreto en AceriaApp. Un endpoint conoce CrearOrdenCommand e ISender. El contenedor resuelve CrearOrdenHandler, que conoce los puertos necesarios para coordinar la orden.

Riesgo, alternativa y límite. Desacoplar no equivale a ejecutar en otro proceso. MediatR despacha en memoria dentro de la aplicación y una llamada esperada con await sigue formando parte del flujo.

Comprobación técnica y aprendizaje. Sigue un comando desde el endpoint hasta su manejador. Debe encontrarse una respuesta tipada y un único manejador apropiado para esa solicitud.

Bloque 2 — Mediator en la capa de aplicación

En una arquitectura hexagonal, el patrón Mediator se aplica en la capa de aplicación para desacoplar los adaptadores de entrada de los casos de uso. Los adaptadores de entrada envían comandos y consultas al mediador. El mediador los distribuye al manejador correspondiente. Los manejadores implementan los casos de uso y coordinan el dominio. La separación entre adaptador y manejador permite que un mismo caso de uso se invoque desde distintos adaptadores sin modificar el manejador.

```
public record CrearOrdenCommand(string Cliente, string Material, int Cantidad) : IRequest<Guid>;
public sealed class CrearOrdenHandler(
    IOrdenTrabajoRepositorio repositorio, IUnitOfWork unidad)
    : IRequestHandler<CrearOrdenCommand, Guid>
{
    public async Task<Guid> Handle(CrearOrdenCommand request, CancellationToken ct)
    {
        var orden = new OrdenHexagonal(request.Cliente, request.Material, request.Cantidad);
        await repositorio.AgregarAsync(orden, ct);
        await unidad.CommitAsync(ct);
        return orden.Id;
    }
}
```

- **Línea 1:** `public record CrearOrdenCommand(string Cliente, string Material, int Cantidad) : IRequest<Guid>;` → define la solicitud y el tipo exacto de respuesta.
- **Línea 2:** `public sealed class CrearOrdenHandler(` → declara el manejador del caso de uso.

- **Línea 3:** `IOrdenTrabajoRepositorio repositorio, IUnitOfWork unidad)` → recibe el repositorio y la unidad del mismo ámbito.
- **Línea 4:** `: IRequestHandler<CrearOrdenCommand, Guid>` → vincula el manejador al contrato del comando.
- **Línea 5:** `{` → abre el cuerpo de la clase.
- **Línea 6:** `public async Task<Guid> Handle(CrearOrdenCommand request, CancellationToken ct)` → implementa la operación asíncrona con cancelación.
- **Línea 7:** `{` → abre el caso de uso.
- **Línea 8:** `var orden = new OrdenHexagonal(request.Cliente, request.Material, request.Cantidad);` → construye la entidad con sus invariantes.
- **Línea 9:** `await repositorio.AgregarAsync(orden, ct);` → deja la entidad pendiente en el repositorio de M4.
- **Línea 10:** `await unidad.CommitAsync(ct);` → confirma la operación antes de responder.
- **Línea 11:** `return orden.Id;` → devuelve la identidad creada.
- **Línea 12:** `}` → cierra el método.
- **Línea 13:** `}` → cierra la clase.

La aplicación del patrón en la capa de aplicación tiene dos consecuencias prácticas. La primera es que los adaptadores de entrada no conocen los manejadores: solo conocen el mediador. La segunda es que un mismo manejador puede invocarse desde distintos adaptadores. La combinación de ambas permite que el sistema se exponga en varios protocolos sin duplicar la lógica de los casos de uso.

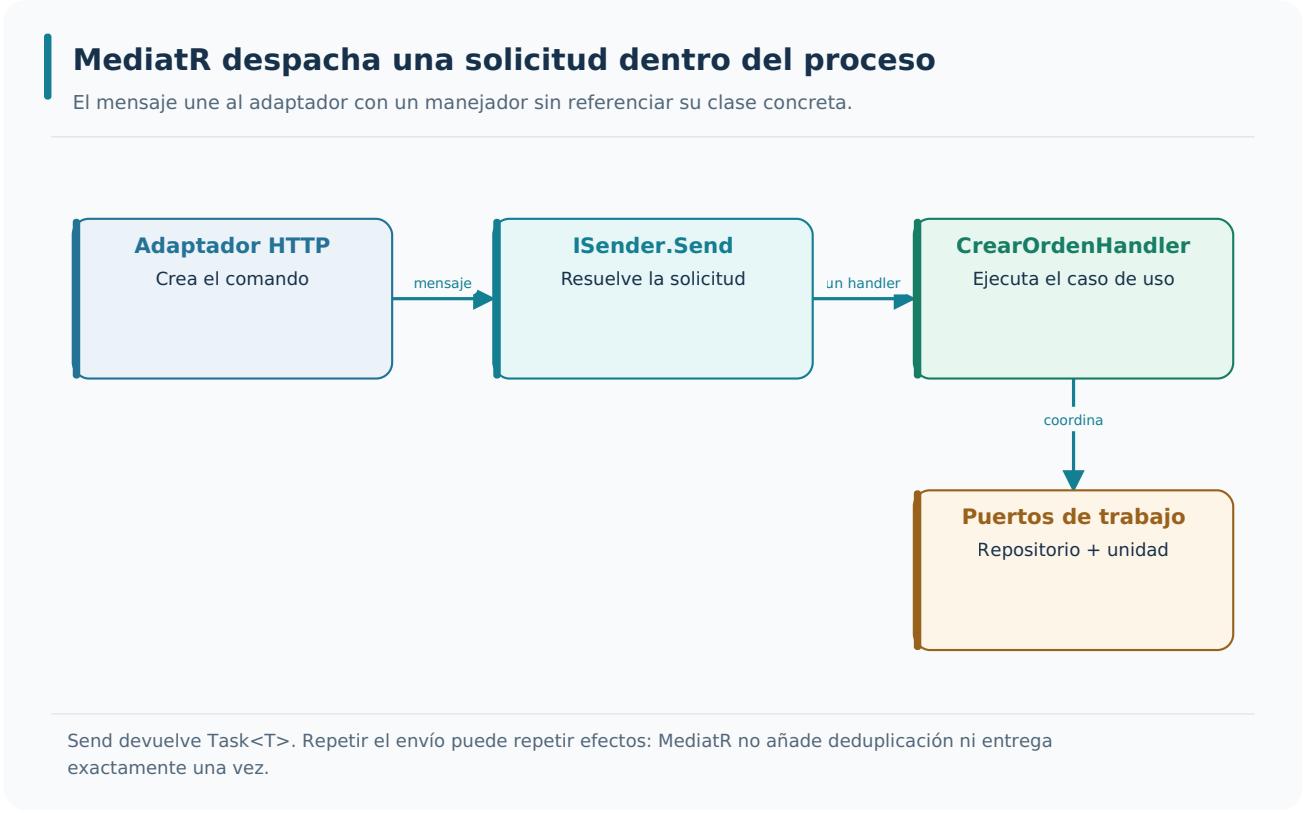
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El manejador puede implementar el caso de uso o delegar en uno existente. Ambas alternativas son válidas si evitan duplicar decisiones de negocio.

Ejemplo concreto en AceriaApp. El ejemplo muestra coordinación directa de entidad, repositorio de trabajo y unidad. La práctica puede delegar en OrdenesTransaccionales para reutilizar la operación del punto anterior.

Riesgo, alternativa y límite. Usar MediatR desde el dominio acoplaría sus reglas a una biblioteca de aplicación. Reserva los mensajes de transporte interno y sus manejadores para el nivel adecuado.

Comprobación técnica y aprendizaje. Prueba el manejador directamente para su lógica y mediante ISender para comprobar registro y pipeline. Son pruebas distintas con fallos distintos.



Interpretación del gráfico: El emisor conoce el mensaje y el mediador; el manejador conoce los puertos. El proceso continúa siendo local. La figura se distribuye en SVG vectorial y PNG.

Bloque 3 — Comandos

Un comando es un mensaje que solicita una acción que modifica el estado del sistema. Los comandos se nombran con un verbo en imperativo: `CrearOrden`, `CancelarOrden`, `ActualizarMaterial`. Un comando contiene los datos necesarios para ejecutar la acción. Un comando devuelve un resultado cuando la acción produce un valor: por ejemplo, el identificador de la entidad creada. Un comando devuelve un resultado vacío cuando la acción no produce un valor. `Send` dirige cada solicitud a un manejador. MediatR no ofrece deduplicación ni entrega exactamente una vez: reintentos del cliente pueden volver a ejecutar un comando. La idempotencia debe diseñarse cuando sea necesaria.

```
public record CrearOrdenCommand(string Cliente, string Material, int Cantidad) : IRequest<Guid>;
public record CancelarOrdenCommand(Guid Id) : IRequest;
public record ActualizarMaterialCommand(Guid Id, int Cantidad) : IRequest;
```

- **Línea 1:** `public record CrearOrdenCommand(string Cliente, string Material, int Cantidad) : IRequest<Guid>;` → comando de creación con retorno.
- **Línea 2:** `public record CancelarOrdenCommand(Guid Id) : IRequest;` → comando de cancelación sin retorno.
- **Línea 3:** `public record ActualizarMaterialCommand(Guid Id, int Cantidad) : IRequest;` → comando de actualización sin retorno.

Los comandos se caracterizan por tres propiedades. La primera es que expresan una intención de cambio: el nombre describe la acción. La segunda es que no son idempotentes por defecto: ejecutarlos varias veces puede producir efectos distintos. La tercera es que cada envío se resuelve con un manejador; esto no impide repetir el envío. Las tres propiedades permiten que los comandos sean la unidad de trabajo de las operaciones de escritura.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Un comando expresa intención: crear o finalizar son acciones. Una respuesta con identificador no lo convierte en consulta, porque el criterio es su efecto sobre el estado.

Ejemplo concreto en AceriaApp. CrearOrdenCommand contiene cliente, material y cantidad. Generar un nuevo GUID en cada ejecución hará dos órdenes si el cliente reintenta sin una estrategia de idempotencia.

Riesgo, alternativa y límite. MediatR no almacena mensajes ni identifica duplicados. Garantizar repetición segura puede requerir una clave de idempotencia y una restricción persistente sobre esa clave.

Comprobación técnica y aprendizaje. Envía dos veces el mismo comando en una prueba de exploración y observa el efecto. Decide después si el contrato de negocio admite duplicados o exige prevención.

Bloque 4 — Consultas

Una consulta es un mensaje que solicita información sin modificar el estado del sistema. Las consultas se nombran con un sustantivo o con un verbo en forma interrogativa: `ObtenerOrden`, `ListarOrdenes`, `BuscarMaterial`. Una consulta contiene los criterios de búsqueda. Una consulta devuelve un resultado con los datos solicitados. Una consulta no debe modificar el estado de negocio. Repetirla puede devolver datos distintos si cambió la base de datos; la ausencia de efectos no equivale a una respuesta idéntica. Las consultas no modifican el estado del sistema.

```
public record ObtenerOrdenQuery(Guid Id) : IRequest<OrdenDto?>;
public record ListarOrdenesQuery(int Page, int PageSize) : IRequest<List<OrdenDto>>;
public record BuscarMaterialQuery(string Nombre) : IRequest<List<MaterialDto>>;
```

- **Línea 1:** `public record ObtenerOrdenQuery(Guid Id) : IRequest<OrdenDto?>;` → consulta de detalle.
- **Línea 2:** `public record ListarOrdenesQuery(int Page, int PageSize) : IRequest<List<OrdenDto>>;` → consulta de lista paginada.
- **Línea 3:** `public record BuscarMaterialQuery(string Nombre) : IRequest<List<MaterialDto>>;` → consulta de búsqueda.

Las consultas se caracterizan por tres propiedades. La primera es que expresan una solicitud de información: el nombre describe qué se busca. La segunda es que repetirlas no debe añadir efectos de negocio, aunque el valor leído pueda variar. La tercera es que no modifican el estado del sistema. Las tres propiedades permiten que las consultas sean la unidad de trabajo de las operaciones de lectura.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Una consulta describe qué información necesita el consumidor. Separarla de comandos ayuda a revisar efectos, validación y optimización sin imponer bases de datos separadas.

Ejemplo concreto en AceriaApp. ObtenerOrdenQuery busca por identificador; una lista puede incorporar página y tamaño. La respuesta del manejador puede ser una proyección de aplicación adecuada para lectura.

Riesgo, alternativa y límite. Dos consultas iguales pueden devolver estados distintos porque otra petición finalizó una orden. No confundas seguridad de lectura con inmutabilidad del mundo observado.

Comprobación técnica y aprendizaje. Comprueba que una consulta no crea ni modifica órdenes. Si incorpora caché, explica su caducidad y consistencia sin esconder esas decisiones bajo el nombre Query.

Bloque 5 — Notificaciones

Una notificación es un mensaje que informa de un evento ocurrido en el sistema. Las notificaciones se nombran con un sustantivo en pasado: `OrdenCreada`, `MaterialActualizado`, `ClienteRegistrado`. Una notificación contiene los datos del evento. Una notificación no devuelve un resultado. Una notificación puede tener varios manejadores: cada manejador reacciona al evento de forma independiente. Las notificaciones se procesan después de que el evento haya ocurrido.

```
public record OrdenCreadaNotification(Guid Id, string Cliente) : INotification;
public interface ICorreoOrdenes
{
    Task EnviarConfirmacionAsync(Guid id, string cliente, CancellationToken ct);
}
public interface IAuditoriaOrdenes
{
    Task RegistrarCreacionAsync(Guid id, CancellationToken ct);
}
public sealed class EnviarEmailOrdenCreadaHandler(ICorreoOrdenes correo)
    : INotificationHandler<OrdenCreadaNotification>
{
    public Task Handle(OrdenCreadaNotification notification, CancellationToken ct) =>
        correo.EnviarConfirmacionAsync(notification.Id, notification.Cliente, ct);
}
public sealed class RegistrarAuditoriaOrdenCreadaHandler(IAuditoriaOrdenes auditoria)
    : INotificationHandler<OrdenCreadaNotification>
{
    public Task Handle(OrdenCreadaNotification notification, CancellationToken ct) =>
        auditoria.RegistrarCreacionAsync(notification.Id, ct);
}
```

- **Línea 1:** `public record OrdenCreadaNotification(Guid Id, string Cliente) : INotification;` → modela un hecho local sin tipo de respuesta.
- **Línea 2:** `public interface ICorreoOrdenes` → declara el puerto de correo del ejemplo.
- **Línea 3:** `{` → abre el contrato.
- **Línea 4:** `Task EnviarConfirmacionAsync(Guid id, string cliente, CancellationToken ct);` → define el efecto externo y la cancelación.
- **Línea 5:** `}` → cierra el contrato.
- **Línea 6:** `public interface IAuditoriaOrdenes` → declara el puerto de auditoría.
- **Línea 7:** `{` → abre el contrato.
- **Línea 8:** `Task RegistrarCreacionAsync(Guid id, CancellationToken ct);` → define la operación de registro.
- **Línea 9:** `}` → cierra el contrato.
- **Línea 10:** `public sealed class EnviarEmailOrdenCreadaHandler(ICorreoOrdenes correo)` → recibe el adaptador de correo por constructor.
- **Línea 11:** `: INotificationHandler<OrdenCreadaNotification>` → suscribe el manejador al tipo de notificación.
- **Línea 12:** `{` → abre la clase.
- **Línea 13:** `public Task Handle(OrdenCreadaNotification notification, CancellationToken ct) =>` → implementa el contrato de MediatR.
- **Línea 14:** `correo.EnviarConfirmacionAsync(notification.Id, notification.Cliente, ct);` → delegación completa al puerto, sin HTTP ni SMTP en el manejador.
- **Línea 15:** `}` → cierra el primer manejador.
- **Línea 16:** `public sealed class RegistrarAuditoriaOrdenCreadaHandler(IAuditoriaOrdenes auditoria)` → recibe el adaptador de auditoría por constructor.
- **Línea 17:** `: INotificationHandler<OrdenCreadaNotification>` → suscribe un segundo manejador al mismo hecho.

- **Línea 18:** `{` → abre la clase.
- **Línea 19:** `public Task Handle(OrdenCreadaNotification notification, CancellationToken ct) =>` → implementa el mismo contrato de manejo.
- **Línea 20:** `auditoria.RegistrarCreacionAsync(notification.Id, ct);` → registra el hecho mediante su propio puerto.
- **Línea 21:** `}` → cierra el segundo manejador.

Las notificaciones se caracterizan por tres propiedades. La primera es que informan de un evento ocurrido: el nombre describe el hecho. La segunda es que tienen varios manejadores: cada manejador reacciona de forma independiente. La tercera es que no devuelven resultado. Las tres propiedades permiten que las notificaciones sean el mecanismo de comunicación entre componentes desacoplados.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Una notificación comunica un hecho y admite cero o más receptores. Los efectos de los receptores no deben depender de un orden implícito de registro.

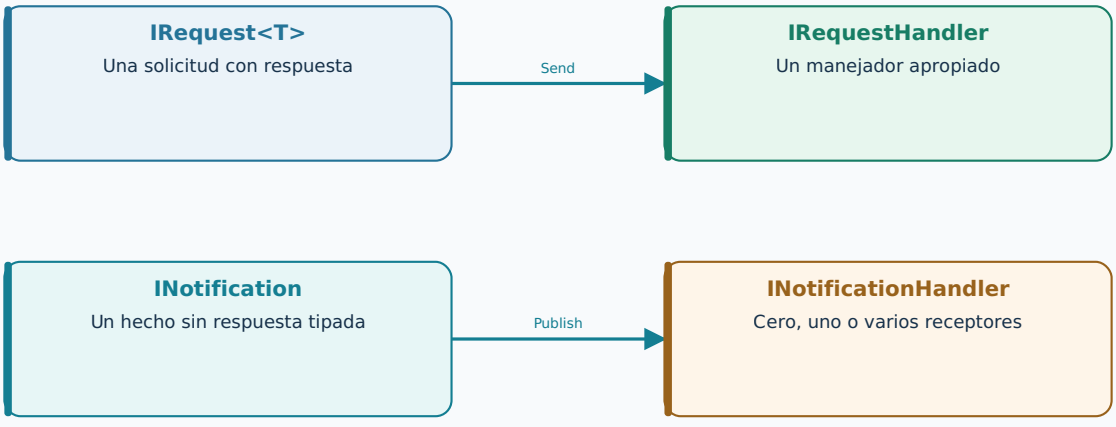
Ejemplo concreto en AceriaApp. La creación puede originar auditoría y confirmación por correo mediante puertos separados. Los manejadores completos del ejemplo delegan esos efectos sin conocer SMTP ni almacenamiento concreto.

Riesgo, alternativa y límite. Cada receptor puede fallar y la publicación puede quedar a medias. Para efectos imprescindibles después de commit, la fiabilidad necesita diseño adicional y pruebas de fallo.

Comprobación técnica y aprendizaje. Prueba cada receptor con un doble de su puerto y prueba la publicación completa por separado. No llames a un efecto finalizado antes de conocer cuándo se confirma su dato.

Solicitud y notificación tienen contratos diferentes

Command y Query usan Send; una notificación se distribuye con Publish.



La notificación es local al proceso. La estrategia predeterminada espera a los receptores secuencialmente y puede fallar.

Interpretación del gráfico: Solicitud y notificación tienen distinta cardinalidad de receptores y distinta forma de respuesta. La figura se distribuye en SVG vectorial y PNG.

Bloque 6 — MediatR como implementación del patrón

MediatR es una biblioteca de .NET que implementa el patrón Mediator. La biblioteca ofrece tres abstracciones principales: `IRequest<TResponse>` para comandos y consultas con retorno, `IRequest` para solicitudes sin valor de retorno, e `INotification` para notificaciones. La biblioteca ofrece `IRequestHandler<TRequest>` para solicitudes sin retorno y, además, `IRequestHandler<TRequest, TResponse>` para manejadores de comandos y consultas, e `INotificationHandler<TNotification>` para manejadores de notificaciones. La biblioteca se registra en el contenedor de dependencias con el método `AddMediatR`.

```
builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssemblyContaining<CrearOrdenHandler>();
});
```

- **Línea 1:** `builder.Services.AddMediatR(cfg =>` → registra MediatR en el contenedor.
- **Línea 2:** `{` → abre el bloque de configuración.
- **Línea 3:** `cfg.RegisterServicesFromAssemblyContaining<CrearOrdenHandler>();` → registra los manejadores del ensamblado.
- **Línea 4:** `});` → cierra el bloque.

MediatR se caracteriza por tres propiedades. La primera es que ofrece abstracciones para comandos, consultas y notificaciones. La segunda es que registra los manejadores en el contenedor de dependencias. La tercera es que ofrece `ISender` para solicitudes, `IPublisher` para notificaciones e `IMediator` como interfaz que reúne ambas capacidades. Las tres propiedades permiten que los adaptadores de entrada invoquen los casos de uso con una sola línea de código.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El módulo fija MediatR 12.4.1 para que código, paquetes y API sean reproducibles. Esa versión es una elección del curso, no una afirmación sobre la versión más reciente.

Ejemplo concreto en AceriaApp. Registra el ensamblado que contiene CrearOrdenHandler. Escanear Program solo encuentra manejadores de la API y dejaría sin descubrir los ubicados en aplicación.

Riesgo, alternativa y límite. AddMediatR registra manejadores, pero no inventa adaptadores ni configura automáticamente todos los validadores. Cada dependencia usada por un manejador también necesita registro.

Comprobación técnica y aprendizaje. Resuelve ISender dentro de un ámbito y ejecuta una solicitud real. Eso detecta ensamblados incorrectos y dependencias ausentes que una compilación no descubre.

Bloque 7 — Envío de comandos y consultas

El envío de comandos y consultas se realiza con el método asíncrono `Send` de la interfaz `IMediator`. El método recibe el comando o la consulta y devuelve el resultado del manejador. El mediador localiza el manejador adecuado según el tipo del mensaje. La localización se realiza en tiempo de ejecución. El adaptador de entrada no conoce el manejador: solo conoce el mediador. Esta separación permite que el adaptador se mantenga simple.

```
public static class OrdenesEndpoints
{
    public static void MapOrdenesEndpoints(this WebApplication app)
    {
        app.MapPost("/api/v1/ordenes", async (
            CrearOrdenCommand command,
            ISender mediator, CancellationToken ct) =>
```

```
        {
            var id = await mediator.Send(command, ct);
            return Results.Created($"/api/v1/ordenes/{id}", new { id });
        });
    }
}
```

- **Línea 1:** `public static class OrdenesEndpoints` → clase de endpoints.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `public static void MapOrdenesEndpoints(this WebApplication app)` → método de extensión.
- **Línea 4:** `{` → abre el cuerpo del método.
- **Línea 5:** `app.MapPost("/api/v1/ordenes", async (` → declara el endpoint POST.
- **Línea 6:** `CrearOrdenCommand command,` → recibe el comando.
- **Línea 7:** `ISender mediator, CancellationToken ct) =>` → inyecta el mediador.
- **Línea 8:** `{` → abre el cuerpo del lambda.
- **Línea 9:** `var id = await mediator.Send(command, ct);` → envía el comando al mediador.
- **Línea 10:** `return Results.Created($"/api/v1/ordenes/{id}", new { id });` → devuelve 201 con la ubicación.
- **Línea 11:** `});` → cierra el lambda.
- **Línea 12:** `}` → cierra el método.
- **Línea 13:** `}` → cierra la clase.

El envío de comandos y consultas tiene dos consecuencias prácticas. La primera es que el adaptador de entrada se mantiene simple: solo conoce el mediador. La segunda es que el manejador se mantiene independiente: no conoce al adaptador. La combinación de ambas permite que el sistema se exponga en varios protocolos sin modificar los manejadores.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El endpoint transforma protocolo y envía un mensaje. ISender limita su dependencia a Send cuando no necesita publicar notificaciones.

Ejemplo concreto en AceriaApp. La firma incluye CancellationToken y lo transmite a Send. El manejador recibe el mismo propósito de cancelación y lo transmite a EF Core mediante los puertos.

Riesgo, alternativa y límite. Un método de extensión debe estar dentro de una clase estática. Send devuelve Task; añadir Async al nombre por costumbre no crea un método existente.

Comprobación técnica y aprendizaje. Comprueba la respuesta HTTP y consulta la ubicación devuelta por una creación. El URI de Location debe corresponder a una ruta de consulta realmente expuesta por la API.

Bloque 8 — Publicación de notificaciones

La publicación de notificaciones se realiza con el método asíncrono `Publish` de la interfaz `IMediator`. El método recibe la notificación y la distribuye a todos los manejadores registrados. La estrategia de publicación decide la ejecución: la predeterminada espera secuencialmente a los manejadores. No debe dependerse de un orden implícito para reglas de negocio; una excepción puede interrumpir la publicación. La publicación no devuelve un resultado. La publicación se usa para comunicar eventos entre componentes desacoplados.

```
await mediator.Publish(new OrdenCreadaNotification(id, command.Cliente));
```

- **Línea 1:** `await mediator.Publish(new OrdenCreadaNotification(id, command.Cliente));` → publica la notificación.

La publicación de notificaciones tiene dos consecuencias prácticas. La primera es que los componentes se comunican sin conocerse entre sí. La segunda es que añadir un manejador nuevo no requiere modificar los manejadores existentes. La combinación de ambas permite que el sistema evolucione sin modificar el código existente. Las notificaciones de MediatR viven dentro del proceso y no son una cola persistente. La comunicación entre servicios del módulo 6 requiere un transporte y garantías adicionales.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Publish espera el trabajo de la estrategia de publicación configurada. Asíncrono describe cómo se espera; no implica una cola duradera ni un proceso separado.

Ejemplo concreto en AceriaApp. La estrategia predeterminada de esta versión recorre receptores secuencialmente. El diseño no debería usar su orden para ejecutar pasos de una operación de negocio obligatoria.

Riesgo, alternativa y límite. Si se necesita enviar a otro servicio, una notificación local puede iniciar un adaptador de mensajería, pero sus garantías pertenecen al transporte y al diseño de entrega.

Comprobación técnica y aprendizaje. Provoca un fallo en un receptor y observa cuáles llegaron a ejecutarse. Esa evidencia evita asumir aislamiento, ejecución completa o reintento automático que no se configuró.

Bloque 9 — MediatR y validación

MediatR se integra con FluentValidation mediante un comportamiento de pipeline. El comportamiento se ejecuta antes del manejador y valida el comando o la consulta. Si la validación falla, el comportamiento lanza una excepción que el adaptador de entrada traduce en una respuesta problem details. La integración aplica validación a las solicitudes que atraviesan el comportamiento registrado y para las que existen validadores. Un comportamiento genérico abierto se registra con `AddOpenBehavior`; `AddBehavior` también permite registrar comportamientos con tipos cerrados.

```
public sealed class ValidationBehavior<TRequest, TResponse>(
    IEnumerable<IValidator<TRequest>> validators)
    : IPipelineBehavior<TRequest, TResponse> where TRequest : notnull
{
    public async Task<TResponse> Handle(TRequest request,
        RequestHandlerDelegate<TResponse> next, CancellationToken ct)
    {
        var failures = new List<FluentValidation.Results.ValidationFailure>();
        foreach (var validator in validators)
        {
            var result = await validator.ValidateAsync(request, ct);
            failures.AddRange(result.Errors);
        }
        if (failures.Count > 0)
            throw new ValidationException(failures);
        return await next();
    }
}
```

- **Línea 1:** `public sealed class ValidationBehavior<TRequest, TResponse>(` → declara el comportamiento abierto.

- **Línea 2:** `IEnumerable<IValidator<TRequest>> validators)` → recibe todos los validadores registrados para la solicitud.
- **Línea 3:** `: IPipelineBehavior<TRequest, TResponse> where TRequest : notnull` → implementa el pipeline con la restricción de MediatR 12.4.1.
- **Línea 4:** `{` → abre la clase.
- **Línea 5:** `public async Task<TResponse> Handle(TRequest request,` → declara el método de la tubería.
- **Línea 6:** `RequestHandlerDelegate<TResponse> next, CancellationToken ct)` → recibe la continuación y la cancelación de la solicitud.
- **Línea 7:** `{` → abre el método.
- **Línea 8:** `var failures = new List<FluentValidation.Results.ValidationFailure>();` → prepara una colección de errores tipados.
- **Línea 9:** `foreach (var validator in validators)` → recorre los validadores uno a uno.
- **Línea 10:** `{` → abre la iteración.
- **Línea 11:** `var result = await validator.ValidateAsync(request, ct);` → ejecuta reglas síncronas y asíncronas sin concurrencia sobre recursos scoped.
- **Línea 12:** `failures.AddRange(result.Errors);` → acumula los errores del validador.
- **Línea 13:** `}` → cierra la iteración.
- **Línea 14:** `if (failures.Count > 0)` → detecta si debe detenerse la solicitud.
- **Línea 15:** `throw new ValidationException(failures);` → propaga errores esperados de validación al traductor configurado.
- **Línea 16:** `return await next();` → continúa con el siguiente comportamiento o manejador.
- **Línea 17:** `}` → cierra el método.
- **Línea 18:** `}` → cierra la clase.

La integración con FluentValidation tiene dos consecuencias prácticas. La primera es que la validación se realiza de forma automática. La segunda es que el manejador no contiene código de validación. La combinación de ambas permite que los manejadores se centren en la lógica del caso de uso.

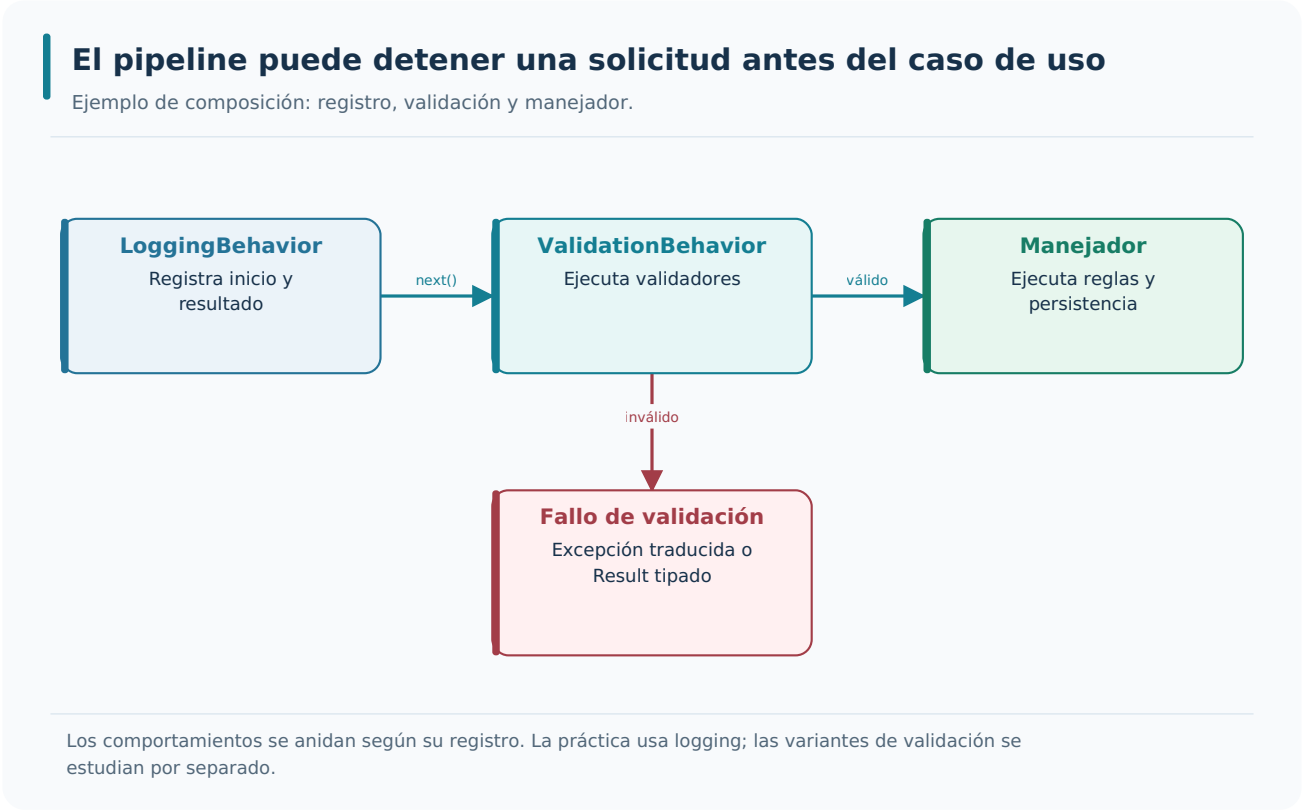
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Un pipeline aplica comportamiento transversal alrededor de la solicitud. Logging, validación y medición son ejemplos; el orden de registro decide cómo se anidan.

Ejemplo concreto en AceriaApp. La práctica conecta LoggingBehavior y los ejemplos desarrollan además una variante con FluentValidation. Los validadores se ejecutan secuencialmente para no compartir DbContext de forma concurrente.

Riesgo, alternativa y límite. La variante que lanza ValidationException necesita traducción HTTP explícita. En 4.4 se estudia otra variante que devuelve Result; no registres ambas para el mismo propósito sin diseñar su interacción.

Comprobación técnica y aprendizaje. Registra el comportamiento abierto con AddOpenBehavior y registra `IValidator<T>` para los mensajes que lo necesiten. Un pipeline sin validadores no valida por el solo hecho de existir.



Interpretación del gráfico: Los comportamientos se anidan; una validación fallida corta el recorrido antes del manejador. La figura se distribuye en SVG vectorial y PNG.

Registro del comportamiento de validación del ejemplo. En la raíz de composición, después de registrar los validadores concretos como `IValidator<TRequest>`, el comportamiento abierto se incorpora a MediatR así:

```
builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssemblyContaining<CrearOrdenHandler>();
    cfg.AddOpenBehavior(typeof(ValidationBehavior<,>));
});
```

- **Línea 1:** `builder.Services.AddMediatR(cfg =>` → configura la biblioteca fijada para el módulo.
- **Línea 2:** `{` → abre la configuración.
- **Línea 3:** `cfg.RegisterServicesFromAssemblyContaining<CrearOrdenHandler>();` → descubre manejadores en el ensamblado de aplicación.
- **Línea 4:** `cfg.AddOpenBehavior(typeof(ValidationBehavior<,>));` → incorpora el comportamiento para las solicitudes compatibles.
- **Línea 5:** `});` → cierra el registro.

Este es el registro de la variante con excepciones de validación. La práctica usa su comportamiento de logging; añadir validación exige registrar validadores y traducir `ValidationException` en el adaptador HTTP.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso conecta `CrearOrdenCommand` y `ObtenerOrdenQuery` con sus manejadores mediante MediatR 12.4.1. `LoggingBehavior` registra el recorrido de la solicitud, y los casos de uso conservan la validación que les corresponde. Los comandos y consultas implementan `IRequest<TResponse>`; sus manejadores implementan `IRequestHandler<TRequest, TResponse>`. El registro descubre el ensamblado de aplicación. Las notificaciones `INotification` y el pipeline de FluentValidation

desarrollados en los bloques anteriores son variantes didácticas adicionales: no se presentan como componentes conectados de este snapshot. Las pruebas del catálogo se complementan con pruebas de resolución y ejecución del mediador.

```
Aplicación al proyecto del curso:

Comando conectado   → CrearOrdenCommand.
Consulta conectada  → ObtenerOrdenQuery.
Pipeline conectado  → LoggingBehavior.
Manejadores         → implementan IRequestHandler.
MediatR             → registrado con AddMediatR.
Validación          → reglas de entidad y caso de uso.
```

- **Línea 1:** `Comando conectado` → solicitud de creación realmente registrada.
- **Línea 2:** `Consulta conectada` → solicitud de detalle realmente registrada.
- **Línea 3:** `Pipeline conectado` → logging transversal de solicitudes.
- **Línea 4:** `Manejadores` → implementan los contratos tipados de MediatR.
- **Línea 5:** `MediatR` → registrado en el contenedor con el ensamblado correcto.
- **Línea 6:** `Validación` → permanece en entidad y caso de uso en el recorrido ejecutable.

La aplicación del patrón Mediator prepara el proyecto para los puntos 4.4 y 4.5. El punto 4.4 estudiará el Result Pattern. El punto 4.5 estudiará la inyección de dependencias. El punto 4.3 establece la base que se aplicará en los dos puntos siguientes.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El catálogo conserva comandos, consultas y notificaciones como conceptos diferentes. La aplicación funcional debe indicar cuáles están realmente conectados al mediador.

Ejemplo concreto en AceriaApp. El punto introduce CrearOrdenCommand, ObtenerOrdenQuery y LoggingBehavior sobre los puertos de persistencia existentes. Las notificaciones amplían la teoría sin convertir el sistema en mensajería distribuida.

Riesgo, alternativa y límite. Registrar un manejador no prueba automáticamente que el endpoint use Send. Mantén una prueba de resolución y otra del recorrido HTTP cuando ese recorrido esté expuesto.

Comprobación técnica y aprendizaje. Comprueba que se conservan los casos anteriores y que una solicitud llega al manejador con su respuesta correcta. Después estudia cómo representar fallos esperados con Result.

Resumen rápido de la teoría

- El patrón Mediator reduce el acoplamiento entre los componentes al centralizar la comunicación en un objeto mediador.
- En una arquitectura hexagonal, el patrón Mediator se aplica en la capa de aplicación para desacoplar los adaptadores de entrada de los casos de uso.
- Un comando es un mensaje que solicita una acción que modifica el estado del sistema.
- Una consulta es un mensaje que solicita información sin modificar el estado del sistema.
- Una notificación es un mensaje que informa de un evento ocurrido en el sistema.
- MediatR es una biblioteca de .NET que implementa el patrón Mediator.
- El envío de comandos y consultas se realiza con el método asíncrono `Send`.
- La publicación de notificaciones se realiza con el método asíncrono `Publish`.
- MediatR se integra con FluentValidation mediante un comportamiento de pipeline.

Punto 4.4 — Result Pattern

Módulo 4: Patrones de arquitectura

Objetivos de aprendizaje:

- Comprender el patrón Result y su propósito como alternativa al uso de excepciones.
- Identificar la diferencia entre errores esperados y errores excepcionales.
- Comprender la estructura de un resultado y sus variantes.
- Comprender la integración del Result Pattern con los casos de uso y los adaptadores de entrada.
- Añadir al proyecto `AceriaApp` las clases de validación del Result Pattern y un test unitario que verifique la validación.

Desarrollo teórico

Bloque 1 — Definición del patrón Result

El patrón Result es un patrón de diseño que representa el resultado de una operación como un valor explícito en lugar de como una excepción. Su propósito es hacer visible en la firma del método que la operación puede fallar y qué tipo de fallo puede producir. El resultado encapsula dos estados posibles: éxito o fallo. En el estado de éxito, el resultado contiene el valor producido por la operación. En el estado de fallo, el resultado contiene la información del error. El patrón se aplica en operaciones donde el fallo es un resultado esperado y no una condición excepcional.

```
Patrón Result:

Result<T> → Éxito con valor T.
Result<T> → Fallo con error.
```

- **Línea 1:** `Result<T> → Éxito con valor T.` → estado de éxito.
- **Línea 2:** `Result<T> → Fallo con error.` → estado de fallo.

El patrón Result se caracteriza por tres propiedades. La primera es que hace explícito el fallo: la firma del método indica que la operación puede fallar. La segunda es que evita el uso de excepciones para errores esperados: los errores esperados se representan como valores. La tercera es que permite componer operaciones: el resultado de una operación se encadena con el de la siguiente. Las tres propiedades permiten que el flujo de errores sea explícito y controlado.

La aplicación del patrón Result tiene dos consecuencias prácticas. La primera es que el código refleja con precisión qué operaciones pueden fallar y por qué. La segunda es que se evita lanzar excepciones en rutas de fallo frecuentes; cualquier mejora de rendimiento debe medirse en el escenario real. La combinación favorece un flujo explícito y predecible. El patrón se usa en operaciones de negocio donde el fallo es parte del flujo normal.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Result vuelve visibles dos salidas de negocio: éxito y fallo esperado. La firma informa del contenedor, pero los códigos concretos de error también deben documentarse.

Ejemplo concreto en AceriaApp. Una orden con cantidad fuera de capacidad puede devolver un fallo conocido. Un timeout del proveedor no debe presentarse automáticamente como cliente no válido.

Riesgo, alternativa y límite. El patrón no impide excepciones internas ni sustituye cancelación. Tampoco obliga a envolver toda función, incluso aquellas que siempre devuelven un valor válido.

Comprobación técnica y aprendizaje. Escribe un test de éxito y otro de fallo esperado. En el fallo comprueba que no se escribió una orden y que no se devolvió un identificador de éxito por defecto.

Bloque 2 — Errores esperados frente a errores excepcionales

Los errores se clasifican en dos categorías: esperados y excepcionales. Los errores esperados son parte del flujo normal del negocio: un cliente que no existe, una cantidad fuera de rango, un material bloqueado. Los errores excepcionales son condiciones anómalas que no deberían ocurrir: un fallo de red, un fallo de disco, una violación de contrato. Los errores esperados se representan con el patrón Result. Los errores excepcionales se representan con excepciones. La distinción es la base del uso correcto del patrón.

```
Errores esperados          → Result.
Errores excepcionales      → excepción.
```

- **Línea 1:** `Errores esperados → Result.` → se representan como valores.
- **Línea 2:** `Errores excepcionales → excepción.` → se representan como excepciones.

La distinción entre errores esperados y excepcionales tiene dos consecuencias prácticas. La primera es que el código refleja con precisión la naturaleza del error. La segunda es que los errores esperados no se mezclan con los excepcionales en el flujo de control. La combinación de ambas permite que el sistema sea más predecible y que las excepciones se reserven para condiciones realmente anómalas.

La aplicación incorrecta del patrón produce dos síntomas reconocibles. El primero es el uso de excepciones para errores esperados, lo que produce un flujo de control costoso y difícil de seguir. El segundo es el uso del patrón Result para errores excepcionales, lo que oculta condiciones anómalas que deberían propagarse. Los dos síntomas se corrigen clasificando correctamente cada error antes de decidir cómo representarlo.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La clasificación depende del contrato del caso de uso. Un recurso ausente puede ser normal en una búsqueda y una violación de integridad puede revelar un defecto.

Ejemplo concreto en AceriaApp. OrdenesConResultado representa validación, ausencia y conflicto como tipos de error. El adaptador decide después si corresponde 400, 404 o 409.

Riesgo, alternativa y límite. No captures Exception para convertir cualquier problema en Failure con el texto del proveedor. Eso ocultaría defectos y podría publicar datos internos del sistema.

Comprobación técnica y aprendizaje. Simula un fallo esperado y una excepción inesperada por separado. El primero debe seguir el contrato público; la segunda debe llegar al mecanismo de observabilidad y error configurado.

Bloque 3 — Estructura de un resultado

Un resultado se estructura con dos propiedades principales: si la operación ha tenido éxito y el valor o el error asociado. La propiedad de éxito es un booleano. La propiedad de valor contiene el resultado de la operación cuando el éxito es verdadero. La propiedad de error contiene la información del error cuando el éxito es falso. La estructura se implementa con una clase o con un `record`. La estructura se acompaña de métodos estáticos para crear resultados de éxito y de fallo.

```
public sealed class Result<T>
{
    private readonly T? _value;
    public bool IsSuccess { get; }
```

```
    public string? Error { get; }
    public T Value => IsSuccess ? _value! :
        throw new InvalidOperationException("Un fallo no tiene valor de éxito.");
    private Result(bool success, T? value, string? error)
    {
        IsSuccess = success;
        _value = value;
        Error = error;
    }
    public static Result<T> Success(T value) => new(true, value, null);
    public static Result<T> Failure(string error)
    {
        ArgumentException.ThrowIfNullOrWhiteSpace(error);
        return new(false, default, error);
    }
}
```

- **Línea 1:** `public sealed class Result<T>` → declara un contenedor genérico que no permite herencia accidental.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `private readonly T? _value;` → almacena el valor incluso cuando T es un tipo por valor.
- **Línea 4:** `public bool IsSuccess { get; }` → expone el discriminante de éxito o fallo.
- **Línea 5:** `public string? Error { get; }` → expone el mensaje de fallo.
- **Línea 6:** `public T Value => IsSuccess ? _value! :` → permite leer Value únicamente si la operación tuvo éxito.
- **Línea 7:** `throw new InvalidOperationException("Un fallo no tiene valor de éxito.");` → detecta el uso incorrecto del resultado por parte del programador.
- **Línea 8:** `private Result(bool success, T? value, string? error)` → restringe la construcción a las factorías.
- **Línea 9:** `{` → abre el constructor.
- **Línea 10:** `IsSuccess = success;` → asigna el estado.
- **Línea 11:** `_value = value;` → asigna el almacenamiento del valor.
- **Línea 12:** `Error = error;` → asigna el error.
- **Línea 13:** `}` → cierra el constructor.
- **Línea 14:** `public static Result<T> Success(T value) => new(true, value, null);` → construye éxito; la nulabilidad de T forma parte del contrato.
- **Línea 15:** `public static Result<T> Failure(string error)` → declara la factoría de fallo.
- **Línea 16:** `{` → abre la factoría.
- **Línea 17:** `ArgumentException.ThrowIfNullOrWhiteSpace(error);` → rechaza un mensaje vacío que haría ambiguo el fallo.
- **Línea 18:** `return new(false, default, error);` → construye fallo sin valor accesible.
- **Línea 19:** `}` → cierra la factoría.
- **Línea 20:** `}` → cierra la clase.

La estructura del resultado se caracteriza por tres propiedades. La primera es que el constructor es privado: los resultados se crean con los métodos estáticos. La segunda es que sus propiedades son de solo lectura: el contenedor es inmutable, aunque T podría ser un objeto mutable. La tercera es que el estado controla el acceso: un fallo tiene error no vacío y no permite leer Value. El valor default almacenado para un fallo no representa un éxito. Las tres propiedades garantizan que el resultado sea consistente y seguro.

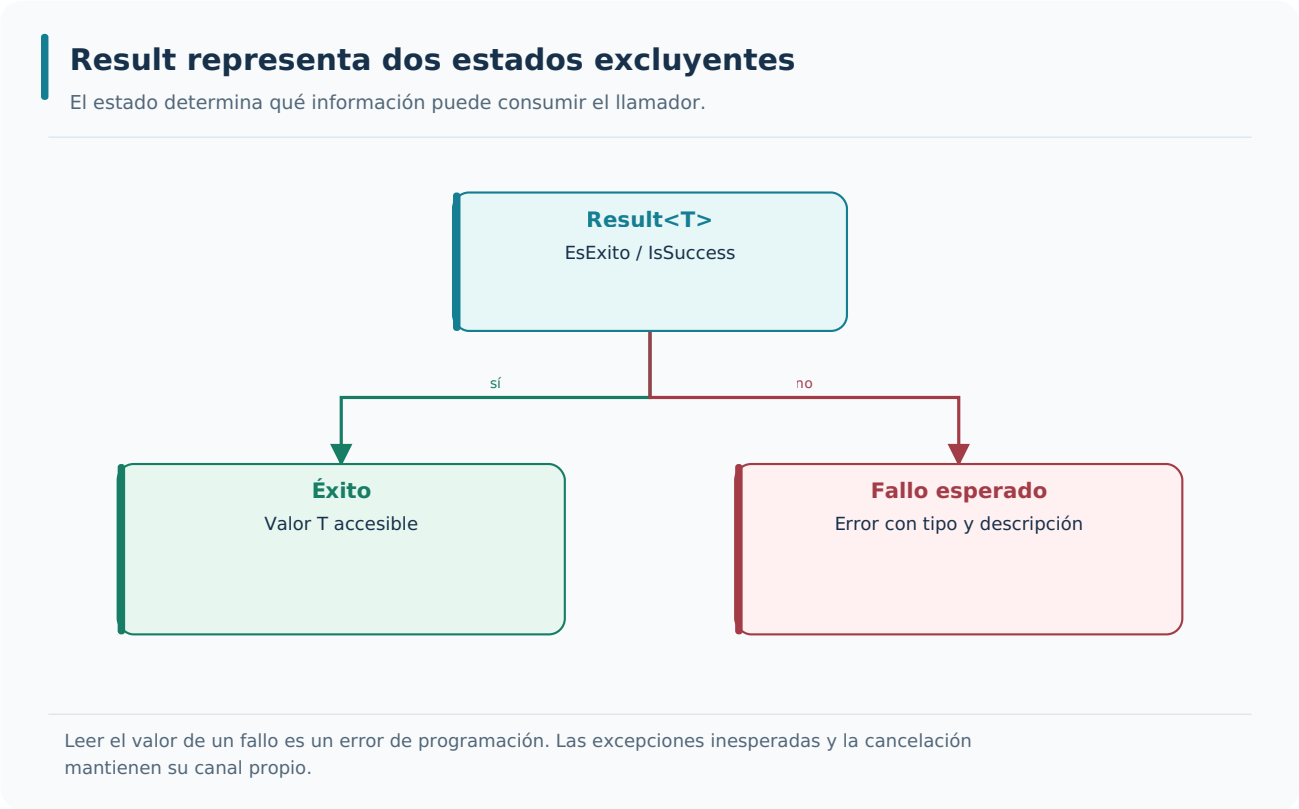
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Un resultado debe impedir estados contradictorios. Un constructor privado y factorías válidas son más claros que permitir modificar IsSuccess, Value y Error independientemente.

Ejemplo concreto en AceriaApp. El tipo simple mostrado prohíbe leer Value en fallo y exige un error no vacío. El tipo funcional del proyecto usa EsExito, Valor, Exito/Fallo, Error y TipoError para la traducción HTTP.

Riesgo, alternativa y límite. T? no convierte siempre un tipo de valor genérico en `Nullable<T>`. Un Guid.Empty almacenado en un fallo no es una identidad válida que pueda devolverse al consumidor.

Comprobación técnica y aprendizaje. Comprueba las invariantes del contenedor y sus dos factorías. Si permites éxito con null, documéntalo mediante T nullable y no lo confundas con la ausencia de éxito.



Interpretación del gráfico: Un resultado representa éxito o fallo. La lectura del valor depende del estado y los errores esperados tienen un tipo estable. La figura se distribuye en SVG vectorial y PNG.

Bloque 4 — Variantes del patrón Result

El patrón Result admite varias variantes según las necesidades del proyecto. La primera variante es `Result<T>` con un valor de éxito y un mensaje de error. La segunda variante es `Result<T, TError>` con un valor de éxito y un tipo de error específico. La tercera variante es `Result` sin valor de éxito para operaciones que no devuelven nada. La cuarta variante es `Result<T>` con una lista de errores para operaciones que pueden fallar por varios motivos. La elección de la variante depende del tipo de errores que maneja el proyecto.

```
Variantes del patrón Result:

Result<T>               → valor de éxito y mensaje de error.
Result<T, TError>       → valor de éxito y tipo de error específico.
```

```
Result                  → sin valor de éxito.
Result<T> con errores   → lista de errores.
```

- **Línea 1:** `Result<T>` → valor de éxito y mensaje de error.
- **Línea 2:** `Result<T, TError>` → valor de éxito y tipo de error específico.
- **Línea 3:** `Result` → sin valor de éxito.
- **Línea 4:** `Result<T> con errores` → lista de errores.

La elección de la variante tiene dos consecuencias prácticas. La primera es que la variante simple es más legible pero menos expresiva. La segunda es que la variante con tipo de error específico es más expresiva pero más compleja. La combinación de ambas se valora según el contexto del proyecto. El curso usa la variante simple con un mensaje de error como base y menciona las variantes avanzadas como opciones.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El mensaje explica un error a una persona; un código o categoría estable permite que otro componente decida sin analizar texto traducido.

Ejemplo concreto en AceriaApp. El proyecto usa TipoError.Validacion, TipoError.NoEncontrado y TipoError.Conflicto. Un Error conserva información del fallo sin incorporar el concepto de IResult al núcleo.

Riesgo, alternativa y límite. Añadir muchas variantes sin necesidad complica consumo y pruebas. Elige una forma coherente para el módulo y reserva ejemplos alternativos para comparar sus ventajas.

Comprobación técnica y aprendizaje. Construye una tabla de categorías y códigos HTTP en el adaptador. Cambiar el texto de un mensaje no debería modificar accidentalmente la respuesta de 404 a 400.

Bloque 5 — Result Pattern en los casos de uso

El patrón Result se aplica en los casos de uso de la capa de aplicación. Los casos de uso que necesitan expresar fallos esperados pueden devolver `Result<T>`; las excepciones inesperadas y la cancelación siguen teniendo su canal propio. El resultado encapsula el éxito o el fallo de la operación. El manejador construye el resultado según el flujo del caso de uso. Los errores esperados se representan con `Result.Failure`. El éxito se representa con `Result.Success`. La firma del caso de uso hace visible que la operación puede fallar.

```
public record CrearOrdenConResultadoCommand(string Cliente, string Material, int Cantidad)
    : IRequest<Result<Guid>>;
public sealed class CrearOrdenConResultadoHandler(
    IOrdenTrabajoRepositorio repositorio, IUnitOfWork unidad)
    : IRequestHandler<CrearOrdenConResultadoCommand, Result<Guid>>
{
    public async Task<Result<Guid>> Handle(CrearOrdenConResultadoCommand request, CancellationToken
ct)
    {
        if (string.IsNullOrWhiteSpace(request.Cliente))
            return Result<Guid>.Failure("El cliente es obligatorio.");
        if (string.IsNullOrWhiteSpace(request.Material))
            return Result<Guid>.Failure("El material es obligatorio.");
        if (request.Cantidad is <= 0 or > 500)
            return Result<Guid>.Failure("La cantidad debe estar entre 1 y 500.");
        var orden = new OrdenHexagonal(request.Cliente, request.Material, request.Cantidad);
        await repositorio.AgregarAsync(orden, ct);
        await unidad.CommitAsync(ct);
        return Result<Guid>.Success(orden.Id);
```

```
    }
}
```

- **Línea 1:** `public record CrearOrdenConResultadoCommand(string Cliente, string Material, int Cantidad)` → define una variante distinta del comando del punto 4.3.
- **Línea 2:** `: IRequest<Result<Guid>>;` → hace coincidir la respuesta del comando con la del manejador.
- **Línea 3:** `public sealed class CrearOrdenConResultadoHandler(` → declara el caso de uso con resultado.
- **Línea 4:** `IOrdenTrabajoRepositorio repositorio, IUnitOfWork unidad)` → recibe los puertos de persistencia del mismo ámbito.
- **Línea 5:** `: IRequestHandler<CrearOrdenConResultadoCommand, Result<Guid>>` → implementa exactamente IRequestHandler para ese mensaje y respuesta.
- **Línea 6:** `{` → abre la clase.
- **Línea 7:** `public async Task<Result<Guid>> Handle(CrearOrdenConResultadoCommand request, CancellationToken ct)` → recibe solicitud y cancelación.
- **Línea 8:** `{` → abre el método.
- **Línea 9:** `if (string.IsNullOrWhiteSpace(request.Cliente))` → identifica el primer fallo esperado.
- **Línea 10:** `return Result<Guid>.Failure("El cliente es obligatorio.");` → devuelve fallo sin modificar persistencia.
- **Línea 11:** `if (string.IsNullOrWhiteSpace(request.Material))` → comprueba la entrada de material.
- **Línea 12:** `return Result<Guid>.Failure("El material es obligatorio.");` → devuelve un segundo fallo esperado.
- **Línea 13:** `if (request.Cantidad is <= 0 or > 500)` → comprueba invariantes de entrada y política de capacidad.
- **Línea 14:** `return Result<Guid>.Failure("La cantidad debe estar entre 1 y 500.");` → devuelve el motivo conocido al llamador.
- **Línea 15:** `var orden = new OrdenHexagonal(request.Cliente, request.Material, request.Cantidad);` → crea una entidad válida; no captura errores inesperados.
- **Línea 16:** `await repositorio.AgregarAsync(orden, ct);` → registra la entidad para la unidad de trabajo.
- **Línea 17:** `await unidad.CommitAsync(ct);` → confirma antes de anunciar éxito.
- **Línea 18:** `return Result<Guid>.Success(orden.Id);` → devuelve el identificador confirmado.
- **Línea 19:** `}` → cierra el método.
- **Línea 20:** `}` → cierra la clase.

La aplicación del patrón en los casos de uso tiene dos consecuencias prácticas. La primera es que el flujo de errores es explícito: cada caso de uso documenta los errores que puede producir. La segunda es que el adaptador de entrada puede traducir el fallo en una respuesta HTTP específica. La combinación de ambas permite que el sistema sea más predecible y que los errores se gestionen de forma uniforme.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El caso de uso debe devolver éxito después de confirmar los cambios requeridos. Preparar una entidad en memoria todavía no demuestra que la operación haya terminado.

Ejemplo concreto en AceriaApp. El ejemplo incluye un comando con respuesta `IRequest<Result<Guid>>`, manejador compatible, comprobaciones de entrada, alta pendiente y commit. OrdenesConResultado aplica el mismo criterio en el proyecto.

Riesgo, alternativa y límite. Result no justifica duplicar reglas inconsistentes. Una validación de entrada puede evitar fallos previsibles mientras la entidad conserva sus invariantes para cualquier llamador.

Comprobación técnica y aprendizaje. Prueba que el fallo esperado no llama a AgregarAsync ni a CommitAsync. Prueba también que una excepción de guardado no se convierte en éxito con el GUID generado.

Bloque 6 — Result Pattern en los adaptadores de entrada

El patrón Result se traduce en los adaptadores de entrada en respuestas HTTP. El adaptador comprueba el resultado del caso de uso. Si el resultado es exitoso, devuelve una respuesta con el valor. Si el resultado es fallido, devuelve una respuesta con el código y el mensaje adecuados. La traducción se realiza en un método específico del adaptador. La traducción mantiene la independencia entre la lógica de negocio y el protocolo HTTP.

```
app.MapPost("/api/v1/ordenes-con-resultado", async (
    CrearOrdenConResultadoCommand command, ISender mediator, CancellationToken ct) =>
{
    var result = await mediator.Send(command, ct);
    IResult respuesta = result.IsSuccess
        ? Results.Created($"/api/v1/ordenes-con-resultado/{result.Value}", new { id = result.Value })
        : Results.Problem(title: "Solicitud no válida", detail: result.Error, statusCode: 400);
    return respuesta;
});
```

- **Línea 1:** `app.MapPost("/api/v1/ordenes-con-resultado", async (` → mapea la ruta didáctica de la variante de creación.
- **Línea 2:** `CrearOrdenConResultadoCommand command, ISender mediator, CancellationToken ct) =>` → recibe el mensaje tipado, el emisor y la cancelación.
- **Línea 3:** `{` → abre el adaptador HTTP.
- **Línea 4:** `var result = await mediator.Send(command, ct);` → ejecuta el caso de uso y espera el resultado.
- **Línea 5:** `IResult respuesta = result.IsSuccess` → elige una respuesta con tipo común IResult.
- **Línea 6:** `? Results.Created($"/api/v1/ordenes-con-resultado/{result.Value}", new { id = result.Value })` → representa éxito como 201 con una ubicación que la API debe exponer.
- **Línea 7:** `: Results.Problem(title: "Solicitud no válida", detail: result.Error, statusCode: 400);` → representa aquí los fallos de entrada como ProblemDetails 400.
- **Línea 8:** `return respuesta;` → devuelve la traducción sin filtrar detalles de infraestructura.
- **Línea 9:** `});` → cierra el endpoint.

La traducción del resultado tiene dos consecuencias prácticas. La primera es que el adaptador de entrada se mantiene simple: solo traduce el resultado a una respuesta. La segunda es que la lógica de negocio no conoce el protocolo HTTP: solo conoce el resultado. La combinación de ambas permite que el mismo caso de uso se exponga en distintos protocolos con traducciones específicas.

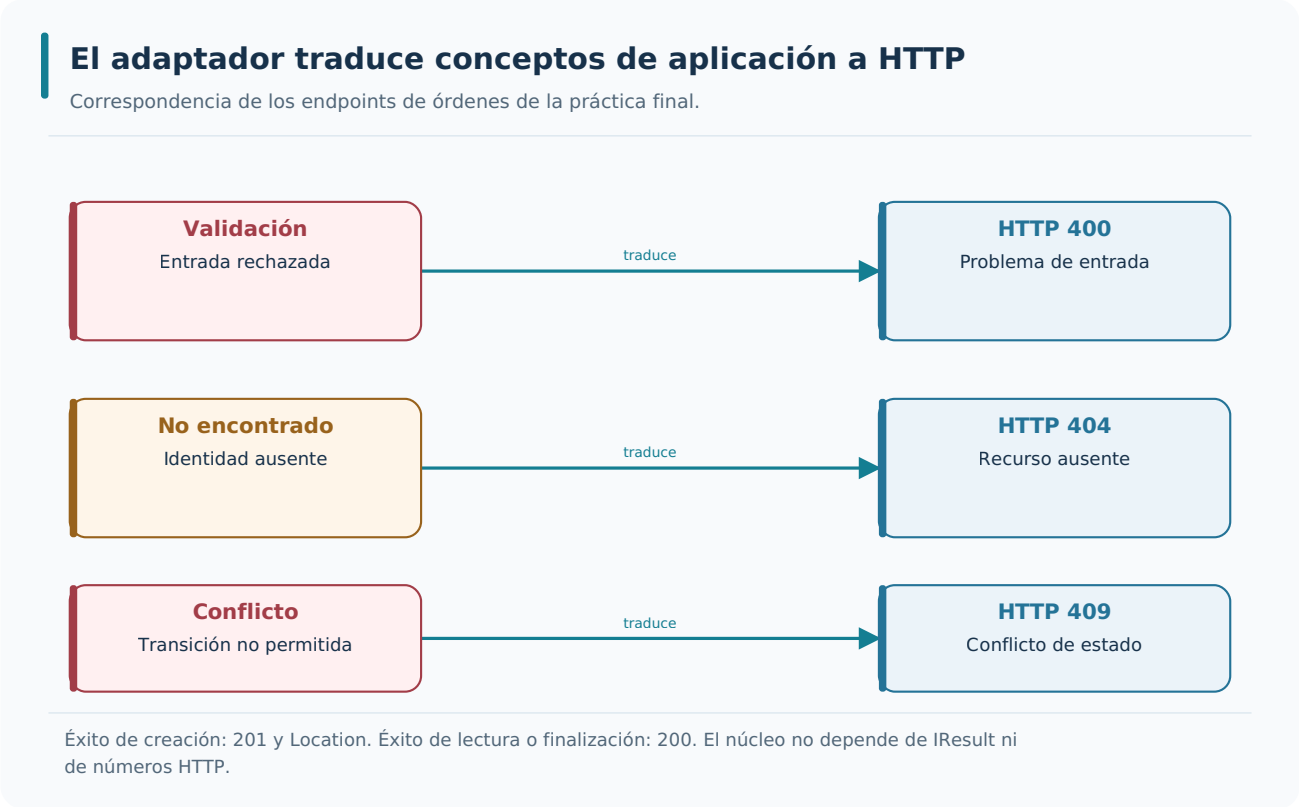
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La traducción a HTTP pertenece a API. El núcleo no devuelve Results.BadRequest, ProblemDetails ni números de estado como sustituto de conceptos de negocio.

Ejemplo concreto en AceriaApp. El ejemplo simple traduce fallos de entrada a 400. La implementación con Error tipado distingue además ausencia con 404 y conflicto con 409.

Riesgo, alternativa y límite. No conviertas todo Failure en BadRequest si el contrato distingue categorías. Tampoco devuelvas directamente mensajes de excepciones técnicas al cliente.

Comprobación técnica y aprendizaje. Comprueba el código, el cuerpo y el tipo de contenido en la prueba HTTP. Para 201 verifica que Location permite recuperar la entidad y no solo que contiene un GUID.



Interpretación del gráfico: El adaptador convierte categorías de aplicación en respuestas HTTP; el dominio no conoce los códigos. La figura se distribuye en SVG vectorial y PNG.

Bloque 7 — Composición de resultados

La composición de resultados permite encadenar varias operaciones donde cada una puede fallar. La composición se realiza con métodos como `Map`, `Bind` y `Match`. El método `Map` transforma el valor de éxito en otro valor. El método `Bind` encadena una operación que devuelve otro resultado. El método `Match` ejecuta una función según el estado del resultado. La composición evita el anidamiento de condicionales y mantiene el flujo de errores explícito.

```
public static class ResultExtensions
{
    public static Result<U> Map<T, U>(this Result<T> source, Func<T, U> map) =>
        source.IsSuccess ? Result<U>.Success(map(source.Value)) : Result<U>.Failure(source.Error!);
    public static Result<U> Bind<T, U>(this Result<T> source, Func<T, Result<U>> bind) =>
        source.IsSuccess ? bind(source.Value) : Result<U>.Failure(source.Error!);
    public static U Match<T, U>(this Result<T> source, Func<T, U> success, Func<string, U> failure)
=>
        source.IsSuccess ? success(source.Value) : failure(source.Error!);
    public static async Task<Result<U>> BindAsync<T, U>(this Result<T> source,
        Func<T, Task<Result<U>>> bind) => source.IsSuccess
```

```
        ? await bind(source.Value) : Result<U>.Failure(source.Error!);
}
```

- **Línea 1:** `public static class ResultExtensions` → declara extensiones para el tipo completo del bloque 3.
- **Línea 2:** `{` → abre la clase estática.
- **Línea 3:** `public static Result<U> Map<T, U>(this Result<T> source, Func<T, U> map) =>` → Map transforma un valor sin introducir otro contenedor.
- **Línea 4:** `source.IsSuccess ? Result<U>.Success(map(source.Value)) : Result<U>.Failure(source.Error!);` → solo ejecuta la transformación en éxito y conserva el fallo.
- **Línea 5:** `public static Result<U> Bind<T, U>(this Result<T> source, Func<T, Result<U>> bind) =>` → Bind recibe una operación que ya produce `Result<U>`.
- **Línea 6:** `source.IsSuccess ? bind(source.Value) : Result<U>.Failure(source.Error!);` → evita anidar `Result<Result<U>>` y mantiene el cortocircuito.
- **Línea 7:** `public static U Match<T, U>(this Result<T> source, Func<T, U> success, Func<string, U> failure) =>` → Match convierte los dos estados en un mismo tipo de salida.
- **Línea 8:** `source.IsSuccess ? success(source.Value) : failure(source.Error!);` → obliga al llamador a describir ambas ramas.
- **Línea 9:** `public static async Task<Result<U>> BindAsync<T, U>(this Result<T> source,` → BindAsync declara explícitamente la asincronía.
- **Línea 10:** `Func<T, Task<Result<U>>> bind) => source.IsSuccess` → recibe una operación asíncrona cuyo resultado conserva el tipo U.
- **Línea 11:** `? await bind(source.Value) : Result<U>.Failure(source.Error!);` → espera el éxito siguiente o propaga el error existente.
- **Línea 12:** `}` → cierra la clase.

La composición de resultados tiene dos consecuencias prácticas. La primera es que el flujo de errores es lineal: cada paso encadena con el siguiente. La segunda es que el código evita el anidamiento de condicionales. La combinación de ambas permite que el código sea más legible y que el flujo de errores sea explícito.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Map transforma un valor; Bind encadena otra operación con posibilidad de fallo; Match obliga a tratar ambos estados. No son métodos mágicos incorporados a cualquier Result.

Ejemplo concreto en AceriaApp. Las extensiones completas acompañan al tipo del bloque 3. BindAsync devuelve `Task<Result<U>>` y acepta una función compatible, evitando mezclar lambdas async con una firma síncrona.

Riesgo, alternativa y límite. Un método de repositorio que devuelve Task sin valor no puede producir por sí solo `Result<Guid>`. La composición debe devolver explícitamente el identificador tras confirmar.

Comprobación técnica y aprendizaje. Verifica el cortocircuito: si falla el primer resultado, el delegado siguiente no debe ejecutarse. Prueba también una excepción inesperada del delegado sin absorberla como error de negocio.



Interpretación del gráfico: Map transforma, Bind encadena y Match decide una salida; un fallo evita ejecutar los pasos siguientes. La figura se distribuye en SVG vectorial y PNG.

Ejemplo de composición sin efectos. Con las extensiones del bloque, el valor se transforma y el siguiente resultado se aplana. La salida es el texto `Cantidad válida: 20`.

```
var salida = Result<int>.Success(10)
    .Map(cantidad => cantidad * 2)
    .Bind(cantidad => cantidad <= 500
        ? Result<int>.Success(cantidad)
        : Result<int>.Failure("Capacidad superada."))
    .Match(valor => $"Cantidad válida: {valor}", error => $"Error: {error}");
```

- **Línea 1:** `var salida = Result<int>.Success(10)` → inicia la composición con éxito.
- **Línea 2:** `.Map(cantidad => cantidad * 2)` → transforma 10 en 20.
- **Línea 3:** `.Bind(cantidad => cantidad <= 500` → decide el resultado de la siguiente operación.
- **Línea 4:** `? Result<int>.Success(cantidad)` → conserva el valor admitido.
- **Línea 5:** `: Result<int>.Failure("Capacidad superada."))` → representa el fallo esperado cuando se supera la capacidad.
- **Línea 6:** `.Match(valor => $"Cantidad válida: {valor}", error => $"Error: {error}");` → convierte ambos estados en texto.

Bloque 8 — Result Pattern y validación

El patrón Result se integra con la validación de FluentValidation. El comportamiento de pipeline ejecuta los validadores antes del manejador. Si la validación falla, el comportamiento devuelve un `Result.Failure` con los errores de validación. Si la validación pasa, el comportamiento continúa con el manejador. La integración permite que la validación se realice de forma automática y que el resultado del caso de uso incluya los errores de validación.

```
public sealed class CrearOrdenResultValidationBehavior(
    IEnumerable<IValidator<CrearOrdenConResultadoCommand>> validators)
    : IPipelineBehavior<CrearOrdenConResultadoCommand, Result<Guid>>
```

```
{
    public async Task<Result<Guid>> Handle(CrearOrdenConResultadoCommand request,
        RequestHandlerDelegate<Result<Guid>> next, CancellationToken ct)
    {
        var mensajes = new List<string>();
        foreach (var validator in validators)
        {
            var validacion = await validator.ValidateAsync(request, ct);
            mensajes.AddRange(validacion.Errors.Select(e => e.ErrorMessage));
        }
        if (mensajes.Count > 0)
            return Result<Guid>.Failure(string.Join("; ", mensajes));
        return await next();
    }
}
```

- **Línea 1:** `public sealed class CrearOrdenResultValidationBehavior(` → define un comportamiento cerrado y fácil de comprobar.
- **Línea 2:** `IEnumerable<IValidator<CrearOrdenConResultadoCommand>> validators)` → recibe validadores para el comando correcto.
- **Línea 3:** `: IPipelineBehavior<CrearOrdenConResultadoCommand, Result<Guid>>` → fija `Result<Guid>` como respuesta del pipeline.
- **Línea 4:** `{` → abre el cuerpo.
- **Línea 5:** `public async Task<Result<Guid>> Handle(CrearOrdenConResultadoCommand request,` → implementa la solicitud con su tipo de resultado.
- **Línea 6:** `RequestHandlerDelegate<Result<Guid>> next, CancellationToken ct)` → recibe la continuación con el mismo `Result<Guid>`.
- **Línea 7:** `{` → abre el método.
- **Línea 8:** `var mensajes = new List<string>();` → prepara los mensajes de validación.
- **Línea 9:** `foreach (var validator in validators)` → recorre los validadores secuencialmente.
- **Línea 10:** `{` → abre la iteración.
- **Línea 11:** `var validacion = await validator.ValidateAsync(request, ct);` → valida con cancelación sin ejecutar el manejador todavía.
- **Línea 12:** `mensajes.AddRange(validacion.Errors.Select(e => e.ErrorMessage));` → conserva los mensajes de cada fallo.
- **Línea 13:** `}` → cierra la iteración.
- **Línea 14:** `if (mensajes.Count > 0)` → comprueba si la solicitud debe detenerse.
- **Línea 15:** `return Result<Guid>.Failure(string.Join("; ", mensajes));` → construye un fallo del tipo exacto, sin object ni casts genéricos.
- **Línea 16:** `return await next();` → continúa únicamente si toda la validación fue correcta.
- **Línea 17:** `}` → cierra el método.
- **Línea 18:** `}` → cierra el comportamiento.

La integración con la validación tiene dos consecuencias prácticas. La primera es que la validación se realiza antes del manejador. La segunda es que el resultado del caso de uso incluye los errores de validación cuando la validación falla. La combinación de ambas permite que el flujo de errores sea uniforme en todos los casos de uso.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El comportamiento tiene que construir exactamente el tipo que promete la solicitud. `Result<object>` y `Result<Guid>` son tipos distintos y un cast a object no los vuelve compatibles.

Ejemplo concreto en AceriaApp. El ejemplo usa un pipeline cerrado para CrearOrdenConResultadoCommand y `Result<Guid>`. Conserva mensajes de validación y omite el manejador cuando hay fallos.

Riesgo, alternativa y límite. Para generalizar a cualquier T puede diseñarse una factoría tipada o una abstracción apropiada; el ejemplo cerrado evita añadir reflexión y conversiones inseguras al aprendizaje inicial.

Comprobación técnica y aprendizaje. Regístralo como `IPipelineBehavior<CrearOrdenConResultadoCommand, Result<Guid>>` y prueba con un validador que falle. Comprueba que next no se ejecutó y que el retorno conserva su tipo.

Bloque 9 — Cuándo usar el patrón Result

El patrón Result se usa cuando los errores son parte del flujo normal del negocio. El patrón se usa cuando se quiere evitar el coste de las excepciones en operaciones frecuentes. El patrón se usa cuando se quiere hacer explícito en la firma del método que la operación puede fallar. El patrón no se usa cuando los errores son excepcionales. El patrón no se usa cuando la operación no puede fallar.

```
Cuándo usar Result:

Errores esperados            → sí.
Operaciones frecuentes       → medir, no asumir.
Firma explícita              → sí.
Errores excepcionales        → no.
Operación sin fallo esperado → no.
```

- **Línea 1:** `Errores esperados → sí.` → el patrón aporta claridad.
- **Línea 2:** `Operaciones frecuentes → medir, no asumir.` → la claridad del contrato es el criterio principal.
- **Línea 3:** `Firma explícita → sí.` → el patrón documenta los errores.
- **Línea 4:** `Errores excepcionales → no.` → las excepciones son más adecuadas.
- **Línea 5:** `Operación sin fallo esperado → no.` → el patrón añade indirección innecesaria.

La decisión de usar el patrón tiene dos consecuencias prácticas. La primera es que el código gana claridad sobre los errores esperados. La segunda es que el código añade una estructura adicional. La combinación de ambas se valora según el contexto del proyecto. El curso aplica el patrón porque el dominio tiene varios errores esperados en sus operaciones.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Result tiene sentido cuando el fallo es parte habitual del contrato y el consumidor debe decidir qué hacer. La legibilidad y la coherencia son los criterios principales.

Ejemplo concreto en AceriaApp. Finalizar una orden ya finalizada puede devolver Conflicto. Obtener una identidad desconocida puede devolver NoEncontrado en un caso de uso que exige una entidad existente.

Riesgo, alternativa y límite. No hace falta envolver una función aritmética pura que no tiene un fallo esperado en su contrato. Tampoco se gana seguridad ocultando defectos dentro de un Result genérico.

Comprobación técnica y aprendizaje. Enumera los fallos de cada operación y comprueba que el consumidor trata todos. Si hay un caso imposible de distinguir sin leer el texto, mejora el modelo de error.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso incorpora `Result<T>` en `OrdenesConResultado` y en las respuestas de los mensajes MediatR. La implementación concreta representa éxito con `Exito` y fallos esperados con `Fallo`, acompañado de `Error` y `TipoError`. Sus consumidores usan `EsExito`, `Valor` y `Error`. Los adaptadores de entrada traducen validación, ausencia y conflicto a HTTP 400, 404 y 409. El pipeline conectado continúa siendo `LoggingBehavior`; la validación de entrada y negocio se ejecuta en el caso de uso y en la entidad. El comportamiento FluentValidation con retorno Result del bloque 8 es una alternativa didáctica completa, no un registro activo de este snapshot. Las pruebas verifican invariantes del contenedor, errores esperados, persistencia y traducción HTTP.

```
Aplicación al proyecto del curso:

Caso de uso          → OrdenesConResultado devuelve Result<T>.
Errores esperados     → Fallo con Error y TipoError.
Éxito                → Exito con Valor.
Adaptadores entrada  → 201/200 o 400/404/409.
Validación           → reglas del caso de uso y entidad.
Pipeline             → LoggingBehavior.
```

- **Línea 1:** `Caso de uso` → coordina los puertos y representa el resultado tipado.
- **Línea 2:** `Errores esperados` → mantienen categoría, código y descripción.
- **Línea 3:** `Éxito` → permite consumir el valor confirmado.
- **Línea 4:** `Adaptadores entrada` → traducen las categorías del contrato a HTTP.
- **Línea 5:** `Validación` → se ejecuta en los componentes que contienen las reglas.
- **Línea 6:** `Pipeline` → registra la ejecución sin transformar genéricamente Result.

La aplicación del patrón Result prepara el proyecto para el punto 4.5. El punto 4.5 estudiará la inyección de dependencias. El punto 4.4 establece la base que se aplicará en el punto 4.5. La combinación de Mediator, Result y validación explícita del caso de uso mantiene visibles la intención, el resultado y las reglas de cada operación.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El resultado debe conservar su significado desde aplicación hasta el adaptador. El catálogo ayuda a revisar esa coherencia, y las pruebas funcionales demuestran sus consecuencias.

Ejemplo concreto en AceriaApp. OrdenesConResultado compone los puertos de trabajo y devuelve Result con Error tipado. La API aplica la correspondencia acordada de validación, ausencia y conflicto.

Riesgo, alternativa y límite. Los ejemplos de pipeline muestran alternativas para integrar validación; su presencia en teoría no significa que todo comportamiento esté registrado en el proyecto ejecutable.

Comprobación técnica y aprendizaje. Ejecuta una creación correcta y peticiones que provoquen cada error esperado. Verifica estado de base de datos y respuesta pública para impedir éxitos aparentes o persistencia parcial.

Resumen rápido de la teoría

- El patrón Result representa el resultado de una operación como un valor explícito en lugar de como una excepción.
- Los errores se clasifican en esperados y excepcionales. Los esperados se representan con Result, los excepcionales con excepciones.
- Un resultado se estructura con dos propiedades: si la operación ha tenido éxito y el valor o el error asociado.

- El patrón admite varias variantes según las necesidades del proyecto.
- El patrón se aplica en los casos de uso de la capa de aplicación.
- Los adaptadores de entrada traducen el resultado en respuestas HTTP.
- La composición de resultados permite encadenar operaciones con `Map`, `Bind` y `Match`.
- El patrón se integra con FluentValidation mediante un comportamiento de pipeline.
- El patrón se usa cuando los errores son parte del flujo normal del negocio.

Punto 4.5 — Inyección de dependencias

Módulo 4: Patrones de arquitectura

Objetivos de aprendizaje:

- Comprender el principio de inversión de dependencias y su relación con la inyección de dependencias.
- Comprender los tres ciclos de vida del contenedor de ASP.NET Core.
- Comprender la composición de servicios mediante métodos de extensión.
- Comprender las reglas de registro y las prohibiciones del contenedor.
- Añadir al proyecto `AceriaApp` las clases de validación de la inyección de dependencias y un test unitario que verifique la validación. Cerrar el módulo 4 con el resumen del estado del proyecto.

Desarrollo teórico

Bloque 1 — Principio de inversión de dependencias

El principio de inversión de dependencias establece que los módulos de alto nivel no deben depender de los módulos de bajo nivel. Ambos deben depender de abstracciones. Las abstracciones no deben depender de los detalles. Los detalles deben depender de las abstracciones. El principio es uno de los cinco principios SOLID y es la base sobre la que se construye la inyección de dependencias. Su aplicación permite que las capas interiores definan contratos y que las capas exteriores los implementen. La separación entre contrato e implementación es la que permite sustituir la implementación sin modificar el contrato.

```
Principio de inversión de dependencias:

Módulo de alto nivel → abstracción ← Módulo de bajo nivel.
```

- **Línea 1:** `Módulo de alto nivel → abstracción ← Módulo de bajo nivel.` → ambos módulos dependen de la abstracción.

La aplicación del principio tiene dos consecuencias prácticas. La primera es que los módulos de alto nivel son independientes de los detalles de implementación. La segunda es que los módulos de bajo nivel son sustituibles sin modificar los de alto nivel. La combinación de ambas permite que el sistema evolucione sin acoplamiento entre capas. El principio puede materializarse con inyección manual o con un contenedor que respete la dirección de los contratos.

La violación del principio produce tres síntomas reconocibles. El primero es la dependencia directa entre capas: un módulo de alto nivel instancia un módulo de bajo nivel. El segundo es la imposibilidad de sustituir la implementación: el módulo de alto nivel conoce la clase concreta. El tercero es la dificultad de las pruebas: no se puede sustituir la implementación por un doble. Los tres síntomas se corrigen invirtiendo la dependencia y definiendo abstracciones.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Inversión de dependencias decide hacia qué contratos apuntan las referencias; inyección decide cómo recibe un objeto sus colaboradores. Un contenedor automatiza la composición.

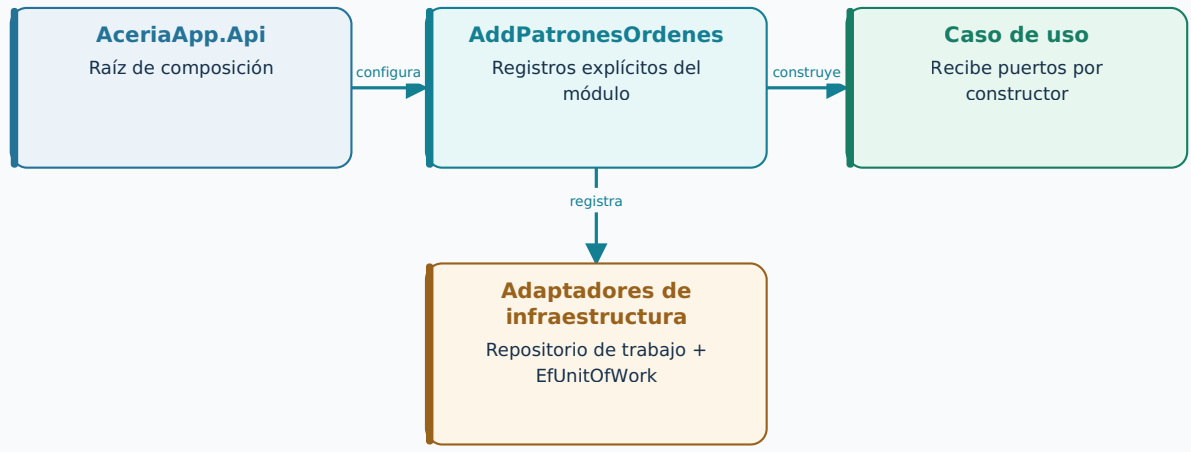
Ejemplo concreto en AceriaApp. Aplicación declara IOrdenTrabajoRepositorio e IUnitOfWork. Infraestructura los implementa; AceriaApp.Api participa en el arranque que conecta todas las piezas.

Riesgo, alternativa y límite. Añadir una interfaz junto a una clase no garantiza inversión si aplicación sigue dependiendo del ensamblado de infraestructura para conocerla.

Comprobación técnica y aprendizaje. Comprueba ProjectReference y namespaces además de constructores. El dominio no necesita conocer el contenedor para que sus objetos se construyan correctamente.

La composición conecta contratos con implementaciones

La API configura el grafo; el caso de uso recibe sus dependencias.



Dominio ignora IServiceProvider. El código de negocio no busca servicios arbitrariamente en un contenedor global.

Interpretación del gráfico: La raíz de composición conoce los adaptadores y enlaza puertos internos con implementaciones exteriores. La figura se distribuye en SVG vectorial y PNG.

Bloque 2 — Inyección de dependencias

La inyección de dependencias es una técnica que ayuda a aplicar la inversión de dependencias. No son sinónimos: se puede inyectar una clase concreta y seguir acoplado a un detalle. El mecanismo consiste en que un objeto no crea sus dependencias: las recibe desde el exterior. La recepción se realiza a través del constructor, de una propiedad o de un método. La inyección por constructor es la más común y la que el curso usa. El contenedor de dependencias se encarga de resolver las dependencias y de inyectarlas en el momento de crear el objeto.

```
public class OrdenServicio
{
    private readonly IOrdenRepositorio _repositorio;

    public OrdenServicio(IOrdenRepositorio repositorio)
    {
        _repositorio = repositorio;
```

```
    }
}
```

- **Línea 1:** `public class OrdenServicio` → clase que necesita una dependencia.
- **Línea 2:** `{` → abre el cuerpo.
- **Línea 3:** `private readonly IOrdenRepositorio _repositorio;` → campo de la dependencia.
- **Línea 4:** `public OrdenServicio(IOrdenRepositorio repositorio)` → constructor con la dependencia.
- **Línea 5:** `{` → abre el cuerpo del constructor.
- **Línea 6:** `_repositorio = repositorio;` → asigna la dependencia.
- **Línea 7:** `}` → cierra el constructor.
- **Línea 8:** `}` → cierra la clase.

La inyección de dependencias se caracteriza por tres propiedades. La primera es que el objeto no crea sus dependencias: las recibe. La segunda es que permite recibir contratos estables, sean interfaces o clases apropiadas. La tercera es que la construcción puede hacerse manualmente o mediante un contenedor; el objeto no necesita conocer cuál se ha usado. Las tres propiedades permiten que el objeto sea testeable y sustituible.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La inyección por constructor hace visibles las dependencias obligatorias y permite crear objetos completos desde el principio. La construcción manual también es inyección.

Ejemplo concreto en AceriaApp. Una prueba puede crear OrdenesTransaccionales con un fake de repositorio y otro de unidad. El caso de uso recibe colaboradores sin preguntar cómo fueron construidos.

Riesgo, alternativa y límite. Inyectar IServiceProvider en cada clase oculta dependencias y permite resolver servicios inesperados. Reserva factorías y ámbitos explícitos para responsabilidades que realmente los necesitan.

Comprobación técnica y aprendizaje. Lee el constructor como una declaración de necesidades. Si requiere demasiados servicios, examina la cohesión del caso de uso antes de esconderlos detrás del contenedor.

Bloque 3 — El contenedor de dependencias de ASP.NET Core

El contenedor de dependencias de ASP.NET Core registra servicios y construye sus grafos. Usarlo en la raíz de composición para inyectar dependencias no equivale al patrón Service Locator; este aparece cuando el código de negocio pide arbitrariamente sus dependencias al contenedor. El contenedor se configura en el arranque de la aplicación. Cada dependencia se registra con un tipo de servicio y un tipo de implementación. El contenedor resuelve las dependencias en tiempo de ejecución. El contenedor se encarga del ciclo de vida de las instancias.

```
builder.Services.AddScoped<IOrdenRepositorio, OrdenRepositorioSqlServer>();
builder.Services.AddScoped<IOrdenServicio, OrdenServicio>();
builder.Services.AddSingleton<TimeProvider>(TimeProvider.System);
```

- **Línea 1:** `builder.Services.AddScoped<IOrdenRepositorio, OrdenRepositorioSqlServer>();` → registra el repositorio con ciclo scoped.
- **Línea 2:** `builder.Services.AddScoped<IOrdenServicio, OrdenServicio>();` → registra el servicio con ciclo scoped.

- **Línea 3:** `builder.Services.AddSingleton<TimeProvider>(TimeProvider.System);` → registra un proveedor de tiempo seguro para compartir; un adaptador de broker requiere evaluar su concurrencia.

El contenedor se caracteriza por tres propiedades. La primera es que registra las dependencias en el arranque. La segunda es que resuelve las dependencias en tiempo de ejecución. La tercera es que gestiona el ciclo de vida de las instancias. Las tres propiedades permiten que el sistema se componga de forma declarativa y que las dependencias se resuelvan sin intervención manual.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. IServiceCollection describe registros; IServiceProvider resuelve instancias a partir de ellos. La configuración debe completarse antes de construir el proveedor que usará la aplicación.

Ejemplo concreto en AceriaApp. AddPatronesOrdenes agrupa los puertos, casos de uso y adaptadores del módulo. AddDbContext aporta HexagonalDbContext con el ciclo scoped habitual.

Riesgo, alternativa y límite. Evita llamar a BuildServiceProvider dentro de los registros para obtener servicios. Crear otro proveedor puede duplicar singletons y producir grafos con ciclos de vida inesperados.

Comprobación técnica y aprendizaje. Resuelve el caso de uso desde un ámbito real y comprueba todas sus dependencias. Una lista de ServiceDescriptor correcta visualmente no demuestra que el grafo se construya.

Bloque 4 — Ciclos de vida del contenedor

El contenedor de ASP.NET Core ofrece tres ciclos de vida: `Transient`, `Scoped` y `Singleton`. El ciclo `Transient` crea una instancia nueva en cada resolución. El ciclo `Scoped` crea una instancia por ámbito de resolución; ASP.NET Core crea normalmente un ámbito para cada petición HTTP y una tarea en segundo plano puede crear ámbitos explícitos. El ciclo `Singleton` reutiliza una instancia por proveedor de servicios; varios procesos o proveedores no comparten automáticamente esa instancia. La elección del ciclo depende del estado que mantiene la implementación y de la duración esperada. Un servicio sin estado puede usar `Singleton` si también sus dependencias, su concurrencia y su liberación de recursos son compatibles. Los servicios que comparten recursos por petición usan `Scoped`. Los servicios que deben ser independientes en cada resolución usan `Transient`.

```
Ciclos de vida del contenedor:

Transient → nueva instancia en cada resolución.
Scoped    → una instancia por ámbito (normalmente petición HTTP).
Singleton → una instancia por proveedor de servicios.
```

- **Línea 1:** `Transient` → nueva instancia en cada resolución.
- **Línea 2:** `Scoped` → una instancia por ámbito explícito o de petición.
- **Línea 3:** `Singleton` → una instancia por proveedor raíz.

La elección del ciclo tiene dos consecuencias prácticas. La primera es que el ciclo incorrecto produce errores de concurrencia o de memoria. La segunda es que el ciclo correcto mantiene el rendimiento y la coherencia. La combinación de ambas permite que el contenedor gestione los recursos de forma eficiente.

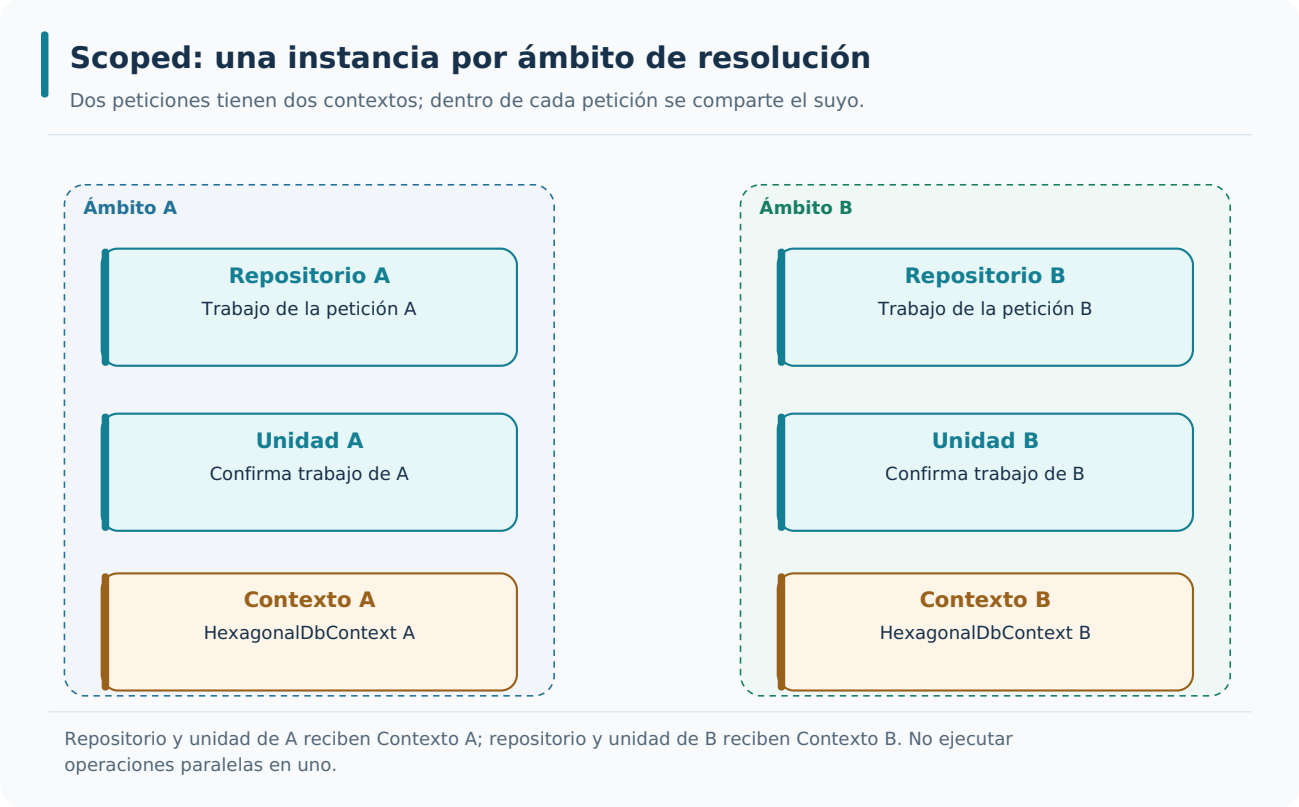
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La duración se elige por estado, propiedad de recursos y concurrencia. Scoped significa ámbito, no usuario, sesión del navegador ni proceso completo.

Ejemplo concreto en AceriaApp. Repositorio y unidad de trabajo comparten HexagonalDbContext dentro de una petición. Otra petición recibe otro contexto, evitando mezclar seguimiento de entidades de usuarios distintos.

Riesgo, alternativa y límite. Un singleton debe soportar acceso concurrente y no capturar scoped. Un servicio de correo o broker no se vuelve seguro como singleton por tener pocas propiedades.

Comprobación técnica y aprendizaje. Resuelve dos veces dentro de un ámbito y una vez desde otro. Usa identidad de objetos para comprobar igualdad dentro del scope y diferencia entre scopes cuando corresponde.



Interpretación del gráfico: Cada ámbito tiene su propio contexto; repositorio y unidad del mismo ámbito comparten ese contexto. La figura se distribuye en SVG vectorial y PNG.

Bloque 5 — Registro de servicios por convención

Una convención de organización agrupa los registros de una capa en un método de extensión. El ejemplo usa registros explícitos; no hace descubrimiento automático de clases. La agrupación reduce el tamaño del archivo `Program.cs` y facilita el mantenimiento. Cada grupo de servicios se registra en un archivo separado. Los métodos de extensión se aplican a `IServiceCollection`. La composición se realiza con llamadas encadenadas. La organización de los métodos de extensión sigue la organización de las capas.

```
public static class RepositoriosExtensions
{
    public static IServiceCollection AddRepositorios(this IServiceCollection services)
    {
        services.AddScoped<IOrdenRepositorio, OrdenRepositorioSqlServer>();
        services.AddScoped<IInventarioRepositorio, InventarioRepositorioSqlServer>();
        return services;
    }
}
```

- **Línea 1:** `public static class RepositoriosExtensions` → clase de extensión.
- **Línea 2:** { → abre el cuerpo.

- **Línea 3:** `public static IServiceCollection AddRepositorios(this IServiceCollection services)` → método de extensión.
- **Línea 4:** `{` → abre el cuerpo del método.
- **Línea 5:** `services.AddScoped<IOrdenRepositorio, OrdenRepositorioSqlServer>();` → registra el repositorio de órdenes.
- **Línea 6:** `services.AddScoped<IInventarioRepositorio, InventarioRepositorioSqlServer>();` → registra el repositorio de inventario.
- **Línea 7:** `return services;` → devuelve la colección para encadenar.
- **Línea 8:** `}` → cierra el método.
- **Línea 9:** `}` → cierra la clase.

El registro por convención tiene dos consecuencias prácticas. La primera es que el archivo `Program.cs` se mantiene corto. La segunda es que los registros de un mismo grupo se mantienen juntos. La combinación de ambas reduce el coste de mantenimiento y facilita la localización de los registros.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Los métodos de extensión organizan la raíz de composición por capacidades o capas. Su valor está en encapsular registros coherentes y dependencias necesarias.

Ejemplo concreto en AceriaApp. AddPatronesOrdenes permite que Program mantenga una llamada legible. La implementación registra puertos y adaptadores de M4 junto a sus casos de uso.

Riesgo, alternativa y límite. El método de extensión debe ubicarse donde sea legal conocer las implementaciones. Ponerlo en dominio obligaría a introducir referencias exteriores y rompería el propósito de la separación.

Comprobación técnica y aprendizaje. Llama al método desde una prueba de composición y resuelve la capacidad completa. Esa prueba protege mejor que verificar únicamente cuántas líneas contiene Program.

Bloque 6 — Registro de servicios por ensamblado

El registro de servicios por ensamblado agrupa los registros según la convención del nombre del tipo. La técnica se usa para registrar automáticamente todos los servicios que implementan una interfaz específica. La técnica se apoya en la reflexión para explorar el ensamblado. La reflexión tiene un coste de arranque que se acepta por la reducción del código de registro. La técnica se aplica cuando hay muchos servicios con la misma convención.

```
var assembly = typeof(OrdenServicio).Assembly;
var tipos = assembly.GetTypes()
    .Where(t => t.IsClass && !t.IsAbstract && !t.ContainsGenericParameters
        && t.Name.EndsWith("Servicio", StringComparison.Ordinal))
    .ToList();

foreach (var tipo in tipos)
{
    var interfaz = tipo.GetInterfaces().FirstOrDefault(i => i.Name == "I" + tipo.Name);
    if (interfaz != null)
    {
        services.AddScoped(interfaz, tipo);
    }
}
```

- **Línea 1:** `var assembly = typeof(OrdenServicio).Assembly;` → elige el ensamblado de una clase de aplicación y no necesariamente el de Program.
- **Línea 2:** `var tipos = assembly.GetTypes()` → obtiene los tipos del ensamblado.

- **Línea 3:** `.Where(t => t.IsClass && !t.IsAbstract && !t.ContainsGenericParameters` → descarta tipos que no sean clases concretas cerradas.
- **Línea 4:** `&& t.Name.EndsWith("Servicio", StringComparison.Ordinal))` → aplica la convención de nombre con comparación explícita.
- **Línea 5:** `.ToList();` → materializa los tipos seleccionados.
- **Línea 6:** `foreach (var tipo in tipos)` → recorre las implementaciones.
- **Línea 7:** `{` → abre la iteración.
- **Línea 8:** `var interfaz = tipo.GetInterfaces().FirstOrDefault(i => i.Name == "I" + tipo.Name);` → busca el contrato cuyo nombre coincide con la convención.
- **Línea 9:** `if (interfaz != null)` → comprueba que se encontró ese contrato.
- **Línea 10:** `{` → abre la condición.
- **Línea 11:** `services.AddScoped(interfaz, tipo);` → registra la pareja contrato e implementación como scoped.
- **Línea 12:** `}` → cierra la condición.
- **Línea 13:** `}` → cierra la iteración.

El registro por ensamblado tiene dos consecuencias prácticas. La primera es que el código de registro se reduce. La segunda es que la convención debe ser estricta para evitar registros accidentales. La combinación de ambas permite que el sistema tenga muchos servicios sin un archivo de registro extenso.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El escaneo por reflexión automatiza una convención. Debe elegir ensamblados concretos y filtrar clases abstractas, genéricas abiertas y tipos ajenos a la capacidad.

Ejemplo concreto en AceriaApp. MediatR busca manejadores en el ensamblado de aplicación mediante un tipo marcador. El ejemplo de servicios busca interfaces cuyo nombre coincide con una convención explícita.

Riesgo, alternativa y límite. Renombrar una clase puede quitarla del registro sin impedir compilar. El trimming y AOT también pueden exigir conservación explícita de metadatos o registros estáticos.

Comprobación técnica y aprendizaje. Prueba el conjunto que se espera descubrir. Ante pocos servicios, los registros explícitos son fáciles de revisar y suelen ser suficientes para el módulo.

Bloque 7 — Reglas de registro

El registro de servicios sigue cinco reglas prácticas. La primera es que cada contrato debe tener una resolución intencional. El contenedor admite varias implementaciones: `IEnumerable<T>` devuelve todas; resolver T directamente elige el último registro no indexado. La segunda es que la implementación se registra con el ciclo adecuado a su estado. La tercera es que las dependencias se resuelven por constructor. La cuarta es que la resolución manual se limita a composición, factorías justificadas y creación explícita de ámbitos. La quinta es que los servicios no capturan el contenedor. La aplicación de las cinco reglas produce un contenedor coherente y mantenible.

```
Reglas de registro:

1. Resolución intencional de cada contrato.
2. Ciclo adecuado al estado.
3. Inyección por constructor.
4. Resolución manual solo en los bordes justificados.
5. Sin capturar el contenedor.
```

- **Línea 1:** `1. Resolución intencional de cada contrato.` → primera regla.

- **Línea 2:** `2. Ciclo adecuado al estado.` → segunda regla.
- **Línea 3:** `3. Inyección por constructor.` → tercera regla.
- **Línea 4:** `4. Resolución manual solo en los bordes justificados.` → cuarta regla.
- **Línea 5:** `5. Sin capturar el contenedor.` → quinta regla.

La aplicación de las reglas tiene dos consecuencias prácticas. La primera es que se reduce el riesgo de registros ausentes o incompatibles; sigue siendo necesario comprobar el grafo real. La segunda es que los servicios se mantienen testeables. La combinación de ambas permite que el sistema se componga de forma predecible y que las pruebas se ejecuten sin infraestructura.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El contenedor admite varios registros del mismo contrato. La política debe distinguir resolver uno, resolver `IEnumerable<T>` o seleccionar servicios con clave cuando corresponda.

Ejemplo concreto en AceriaApp. Varios `IValidator<CrearOrdenCommand>` son intencionales: el comportamiento puede recorrerlos. Dos adaptadores de persistencia registrados sin política de selección pueden esconder una configuración equivocada.

Riesgo, alternativa y límite. El último registro simple prevalece al resolver un único servicio. El orden de registro no debería ser una decisión de negocio escondida que cambie por un método de extensión.

Comprobación técnica y aprendizaje. Comprueba la resolución individual y la colección cuando haya varios registros. Activa ValidateScopes y ValidateOnBuild en pruebas, y resuelve los grafos representativos.

Bloque 8 — Prohibiciones del contenedor

El contenedor tiene cuatro prohibiciones principales. La primera es que un servicio no debe resolver sus dependencias manualmente con el contenedor. La segunda es que un servicio no debe capturar el contenedor en un campo. La tercera es que un servicio no debe registrar servicios en el contenedor en tiempo de ejecución. La cuarta es que un singleton no debe capturar un servicio scoped, directa ni indirectamente. Un scoped sí puede recibir un transient; un transient capturado por un singleton permanece retenido y exige revisar su estado y seguridad concurrente. Las dependencias cíclicas son otro problema del grafo, no la consecuencia automática de cualquier diferencia de duración.

```
Prohibiciones del contenedor:

1. No resolver manualmente.
2. No capturar el contenedor.
3. No registrar en tiempo de ejecución.
4. No capturar scoped desde singleton.
```

- **Línea 1:** `1. No resolver manualmente.` → primera prohibición.
- **Línea 2:** `2. No capturar el contenedor.` → segunda prohibición.
- **Línea 3:** `3. No registrar en tiempo de ejecución.` → tercera prohibición.
- **Línea 4:** `4. No capturar scoped desde singleton.` → cuarta prohibición.

La aplicación de las prohibiciones tiene dos consecuencias prácticas. La primera es que se evitan varias causas de fallo en ejecución, sin sustituir pruebas de composición. La segunda es que las dependencias son predecibles y testeables. La combinación de ambas permite que el sistema se comporte de forma coherente en producción.

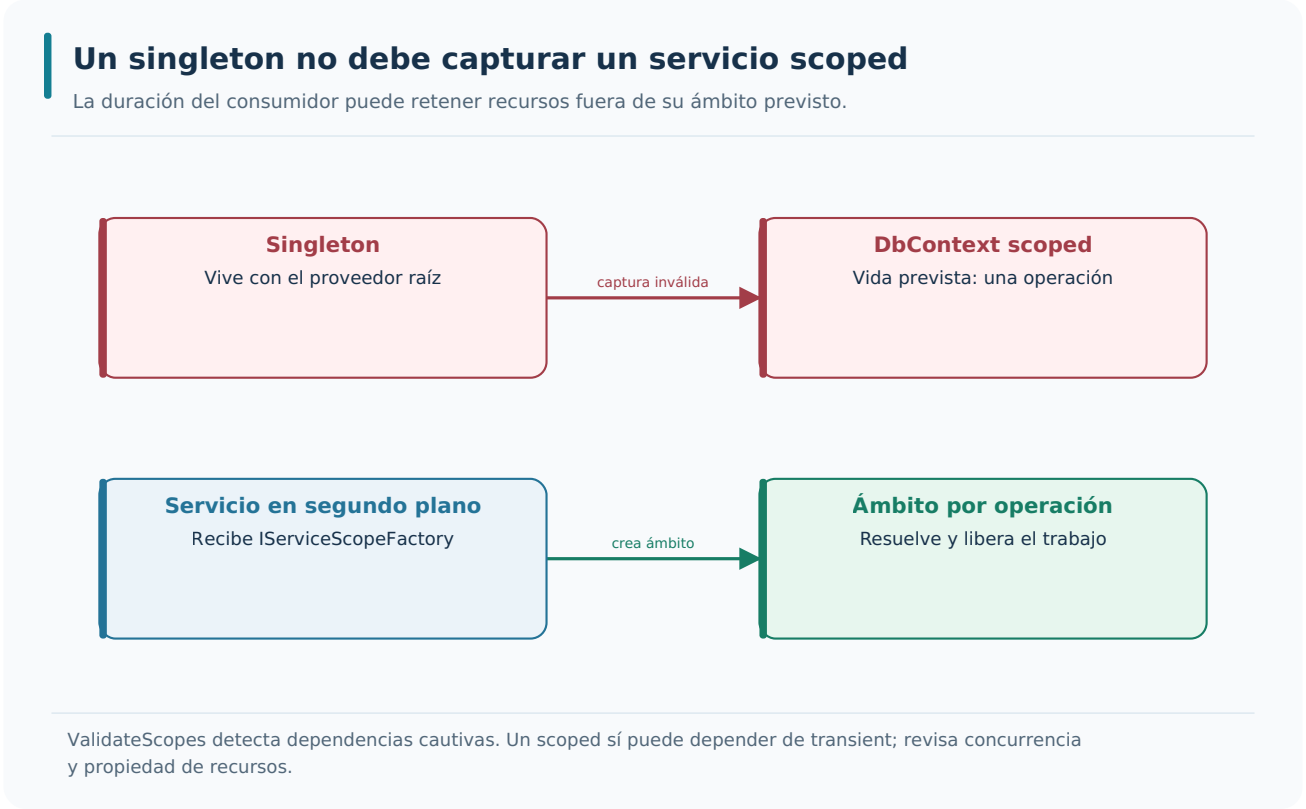
Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. La dependencia cautiva aparece cuando un consumidor retiene un colaborador más allá de su ámbito previsto. El caso crítico es singleton que captura DbContext scoped.

Ejemplo concreto en AceriaApp. Un proceso en segundo plano puede inyectar IServiceScopeFactory, crear un ámbito por operación y resolver dentro de él los servicios de trabajo. Ese uso es composición explícita.

Riesgo, alternativa y límite. No prohíbas todos los transient dentro de scoped: esa relación es válida. En un singleton un transient retenido requiere revisar seguridad concurrente y liberación de recursos.

Comprobación técnica y aprendizaje. Haz una prueba negativa que registre un consumidor singleton con una dependencia scoped y comprueba el fallo de validación. No desactives ValidateScopes para hacer desaparecer el síntoma.



Interpretación del gráfico: La duración del consumidor no debe extender accidentalmente la del servicio scoped que conserva. La figura se distribuye en SVG vectorial y PNG.

Bloque 9 — Inyección de dependencias y tests

La inyección de dependencias facilita los tests porque las dependencias se sustituyen por dobles. El test crea una instancia del servicio con las dependencias sustituidas. Para probar una clase suele bastar construir sus dependencias manualmente. Las pruebas de composición sí levantan el contenedor real y verifican los registros y ámbitos. La separación entre configuración de producción y configuración de pruebas permite que los tests se ejecuten sin infraestructura.

```
var repositorioMock = new Mock<IOrdenRepositorio>();
repositorioMock
    .Setup(r => r.AgregarAsync(It.IsAny<OrdenHexagonal>(), It.IsAny<CancellationToken>()))
    .Returns(Task.CompletedTask);
var servicio = new OrdenesCasoUso(repositorioMock.Object);
var resultado = await servicio.CrearAsync("ACX-001", "CHAPA", 10);
Assert.Equal("ACX-001", resultado.Cliente);
```

```
repositorioMock.Verify(
    r => r.AgregarAsync(It.IsAny<OrdenHexagonal>(), It.IsAny<CancellationToken>()), Times.Once);
```

- **Línea 1:** `var repositorioMock = new Mock<IOrdenRepositorio>();` → crea un mock del puerto de M3.
- **Línea 2:** `repositorioMock` → inicia la configuración.
- **Línea 3:** `.Setup(r => r.AgregarAsync(It.IsAny<OrdenHexagonal>(), It.IsAny<CancellationToken>()))` → acepta la entidad que generará el caso de uso y su cancelación.
- **Línea 4:** `.Returns(Task.CompletedTask);` → simula la terminación de la escritura.
- **Línea 5:** `var servicio = new OrdenesCasoUso(repositorioMock.Object);` → construye directamente el caso de uso heredado.
- **Línea 6:** `var resultado = await servicio.CrearAsync("ACX-001", "CHAPA", 10);` → ejecuta una creación válida sin servidor ni SQL Server.
- **Línea 7:** `Assert.Equal("ACX-001", resultado.Cliente);` → comprueba el dato devuelto, no solo que hubo una llamada.
- **Línea 8:** `repositorioMock.Verify(` → inicia la comprobación de interacción.
- **Línea 9:** `r => r.AgregarAsync(It.IsAny<OrdenHexagonal>(), It.IsAny<CancellationToken>()), Times.Once);` → exige un alta de la entidad; las pruebas de M4 añaden la confirmación correspondiente.

La inyección de dependencias en los tests tiene dos consecuencias prácticas. La primera es que los tests se ejecutan sin base de datos ni infraestructura. La segunda es que las dependencias se sustituyen por dobles específicos. La combinación de ambas permite que los tests sean rápidos y fiables.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. Las pruebas unitarias construyen la clase con colaboradores controlados; las pruebas de composición verifican que el contenedor de producción sabe construirla.

Ejemplo concreto en AceriaApp. El ejemplo usa OrdenesCasoUso heredado para mostrar constructor manual. Los nuevos casos de M4 necesitan además el doble de IUnitOfWork y las expectativas de confirmación.

Riesgo, alternativa y límite. Un fake de memoria compartido entre tests puede introducir contaminación. Crea estado nuevo por prueba y reserva recursos compartidos para fixtures con aislamiento definido.

Comprobación técnica y aprendizaje. Combina pruebas de resultado, ausencia de efectos en fallo y registro real. No necesitas arrancar HTTP para verificar una regla que puede comprobarse directamente en aplicación.

Bloque 10 — Aplicación al proyecto del curso

El proyecto del curso aplica la inyección de dependencias en el módulo 4. Los puertos se registran con sus adaptadores en el contenedor. Los servicios se registran con el ciclo adecuado. La configuración se organiza con métodos de extensión. Las prohibiciones se respetan. La validación del contenedor se realiza con un test que verifica los registros. El módulo 4 se cierra con el resumen del estado del proyecto.

```
Aplicación al proyecto del curso:

Registro de puertos     → interfaces con adaptadores.
Ciclo de vida           → Scoped para la mayoría.
Configuración           → métodos de extensión.
```

```
Prohibiciones           → respetadas.
Validación              → test de coherencia.
Cierre del módulo       → resumen del estado.
```

- **Línea 1:** `Registro de puertos` → interfaces con adaptadores.
- **Línea 2:** `Ciclo de vida` → Scoped para la mayoría.
- **Línea 3:** `Configuración` → métodos de extensión.
- **Línea 4:** `Prohibiciones` → respetadas.
- **Línea 5:** `Validación` → test de coherencia.
- **Línea 6:** `Cierre del módulo` → resumen del estado.

La aplicación de la inyección de dependencias cierra el módulo 4 y prepara el terreno para el módulo 5. El módulo 5 estudiará la introducción a DDD y CQRS. La inyección de dependencias es la base sobre la que se componen los servicios del dominio. El punto 4.5 cierra el módulo 4 con el resumen del estado del proyecto.

Ampliación didáctica aplicada a AceriaApp

La decisión arquitectónica. El cierre del módulo enlaza contratos, efectos y composición. No consiste solo en agregar cinco nombres de patrones a un catálogo.

Ejemplo concreto en AceriaApp. Repository expresa persistencia; Unit of Work confirma; MediatR despacha solicitudes; Result representa fallos esperados; DI conecta sus implementaciones respetando ámbitos.

Riesgo, alternativa y límite. Los patrones resuelven problemas concretos y añaden costes. Ninguno convierte automáticamente un monolito modular en microservicios ni hace fiable una operación distribuida.

Comprobación técnica y aprendizaje. Desde el snapshot final, ejecuta la suite completa, recorre los endpoints de patrones y revisa dependencias. Esa base deja preparados los conceptos de agregados, DDD y CQRS del módulo siguiente.

Resumen rápido de la teoría

- El principio de inversión de dependencias establece que los módulos de alto nivel y de bajo nivel deben depender de abstracciones.
- La inyección de dependencias es el mecanismo que permite aplicar el principio.
- El contenedor de ASP.NET Core registra y resuelve las dependencias en tiempo de ejecución.
- Los ciclos de vida del contenedor son `Transient`, `Scoped` y `Singleton`.
- El registro de servicios por convención agrupa los registros en métodos de extensión.
- El registro por ensamblado agrupa los registros según la convención del nombre del tipo.
- Las reglas de registro y sus pruebas ayudan a mantener un contenedor coherente.
- Las dependencias cautivas, ciclos y registros ausentes requieren comprobaciones distintas.
- La inyección de dependencias facilita los tests al permitir la sustitución de dependencias por dobles.

Documentación técnica de consulta

- Transacciones en EF Core: transacción implícita, control explícito y límites del proveedor.
- Ciclo de vida de DbContext: ámbito de unidad de trabajo y ausencia de seguridad para acceso concurrente.
- MediatR 12.4.1: versión fijada para los ejemplos del módulo y registro de manejadores y comportamientos.
- Validación asíncrona en FluentValidation: uso de ValidateAsync para reglas síncronas y asíncronas.
- Directrices de inyección de dependencias en .NET: duración, composición y propiedad de los recursos.